

NUMBER THEORY

数论 v2.0

教师版

麓山国际实验学校 Federico Prask · 2026

阅读说明与课程地图

适用范围与阅读边界

本资料面向已经学习基本代数、递归、复杂度与 C++ 的信息学竞赛学生，不能把整本资料定位成“小学二年级难度”。同余、线性不定方程可以作为入门；二维积性函数、洲阁筛、AGC031F 属于选修提高内容。

题目这里只给教学性摘要，不替代评测网站的完整输入输出说明。标为“研究拓展”的 Bonus 或最优复杂度改进，只给出可验证的数学路线，不冒充可直接提交的完整程序。实现模板也不等于各原题的完整提交代码。

导读：不认识的字去哪查

本书第 0 章专门解释词与符号。读到某个词不知道它在说什么，先回第 0 章查；查不到的，就在正文里看旁边的绿色 tip。

统一记号

记号	含义与约定
$\mathbb{P}, \mathbb{Z}, \mathbb{N}$	分别为素数集、整数集、非负整数集；数论函数的定义域为正整数。
m, M	一般模数，默认为正整数；讨论单位群与逆元时通常 $m \geq 2$ 。 $m = 1$ 的模运算结果一律为 0。
p, q	素数； $Q = p^\alpha$ 表示素数幂，避免把合数也一律记成 p 。
$[P]$	Iverson 括号，命题成立为 1，否则为 0。
$\nu_p(x)$	非零整数 x 中 p 的指数，即 $\nu_p(x)$ ；需要时约定 $\nu_p(0) = +\infty$ 。
φ, μ, τ	欧拉函数、莫比乌斯函数、约数个数函数。原稿的 $d(n)$ 统一写作 $\tau(n)$ 。
$1, \varepsilon, \text{id}_t$	常一函数、卷积单位函数、 $\text{id}_t(n) = n^t$ 。原稿 I 统一写作 1 。
$f * g$	狄利克雷卷积； fg 为逐点乘积。卷积除法不使用普通分数线混写。
$S_f(n)$	$\sum_{i=1}^n f(i)$ ，并约定 $S_f(0) = 0$ 。
$(n!)_p$	将 $n!$ 中的全部 p 因子去掉后的整数；不是仅删除 p 的倍数。
$\text{ord}_m(a)$	a 模 m 的乘法阶；与原稿 $\delta_m(a)$ 相同。

说明：这张速查表怎么用

上面这张表是速查表，用术语解释术语，适合已经会的人。第 0 章把每一行都翻译成了大白话，并说明这个词为什么会出现在哪里。

所有复杂度默认采用竞赛中的字长运算模型。高精度题目的“多项式时间”以输入位数 $\log n$

为变量； $O(n^{1/4})$ 并不是关于输入位数的多项式时间。哈希表复杂度默认期望，随机算法的经验性能与严格最坏界会明确区分。

词义：字长模型

字长模型 = 假设 64 位整数的一次加减乘比较都算一步。它不是真实机器模型（内存不连续时会慢），也不是高精度模型（那时乘法是 $O(n^2)$ ）。凡是遇到 10^9 以上相乘、 10^{18} 以上相加，先想会不会溢出 long long。

建议的教学分层

层次	主线	可检查的学习产出
A：基础	整除、gcd、exgcd、逆元、CRT、快速幂	能写出条件、通解、最小可行解；能说明一次取模是否丢信息。
B：核心	阶、BSGS、LTE、组合数取模、筛法与反演	能完成质因数拆分、交换求和、解释分母为何可逆。
C：提高	杜教筛、PN、洲阁筛、Miller–Rabin、Pollard–Rho	能区分预处理和查询成本，证明递归状态集的大小。
D：专题	二维积性函数、图上的同余、已知特殊信息分解、韦达跳跃	能识别稀疏支撑、状态等价与证明缺口；不只记最终公式。

建议按每讲 90 分钟安排：A 层 5–6 讲，B 层 6–8 讲，C 层 5–6 讲，D 层按选题增加 4–6 讲；上机训练另计。课时仅作备课建议，不应在一节课内从逆元跳到二维卷积。

每个定理的四步讲法

1. 先提出一个朴素算法或具体算例，明确真正昂贵的对象。
2. 说明不变量与适用条件，再给定理；要求学生主动说出“不满足条件会怎样”。
3. 证明至少解释关键一步：为什么没有漏解、为什么可以约分、为什么每个合数只处理一次。
4. 把数学变成接口：输入、输出、边界、复杂度、溢出风险、最小反例。

教师版的“课堂追问”可先遮住答案；例题解析不建议原样念完，应在关键换元前暂停。

目录

阅读说明与课程地图	iii
第 0 章 数学用词与使用方法说明	1
0.1 怎么读这本书	1
0.2 集合、元素与逻辑用语	1
0.2.1 集合与元素	2
0.2.2 “或”“且”“当且仅当”	2
0.2.3 量词：任意与存在	2
0.2.4 Iverson 括号与“示性函数”	2
0.3 求和、取整、下标：把式子读成程序	3
0.3.1 求和与求积	3
0.3.2 取整	3
0.3.3 下标、递推与“记号说明”	3
0.4 整数：约数、倍数、最大公约数与素因数分解	3
0.5 同余：按余数把整数分堆	4
0.6 幂、阶乘与组合数	5
0.7 数论函数、前缀和与卷积	5
0.8 “群、环、域、同态”：这一页只需听懂，不必学会	6
0.9 算法行话：复杂度、预处理、在线、对拍、溢出	6
0.10 高中叫法 $\leftrightarrow$ 本书叫法	7
0.11 证明的三种句式，以及“不跳步”长什么样	8
0.11.1 双向证明	8
0.11.2 反证与最小反例	8
0.11.3 归纳	8
0.11.4 示范：把一段跳步的证明补全	8
0.12 自查小测	9
第 1 章 整除、最大公约数与线性不定方程	11
1.1 从质因数分解理解整除	11
1.2 欧几里得算法及其复杂度	11
1.3 裴蜀定理与扩展欧几里得	12

1.4	从一个解得到全部解	15
1.5	环上的固定步长	17
第 2 章	同余、逆元与指数降幂	19
2.1	同余是整数等式的压缩	19
2.2	乘法逆元与正确约分	19
2.3	费马小定理与欧拉定理	20
2.4	扩展欧拉定理：为什么要保留“大于阈值”的信息	21
2.5	五类逆元问题	22
2.5.1	前四类：应掌握的基础方法	23
2.5.2	在线方案一：每步至少减半	23
2.5.3	在线方案二： $O(m^{2/3})$ 预处理与常数次查询运算	24
第 3 章	中国剩余定理与同余约束的合并	27
3.1	互质模数：构造局部为一、其余为零的系数	27
3.2	不互质模数：合并两个等差数列	28
第 4 章	乘法阶、原根与离散对数	31
4.1	周期与乘法阶	31
4.2	有限域上的根数与原根	32
4.3	BSGS：用平方根空间换时间	33
4.4	exBSGS：剥离非互质部分	34
第 5 章	升幂引理：把巨大幂变成指数的加法	39
5.1	p -进赋值与使用前的检查	39
5.2	与素数互质的指数	40
5.3	奇素数的完整形式	40
5.4	素数二的特殊形式	41
5.5	分层例题与解析	41
第 6 章	阶乘、素数幂与组合数取模	45
6.1	Wilson 定理及单位群乘积	45
6.2	Legendre 公式：阶乘中的素数指数	46
6.3	Kummer 定理：进位就是损失的数位和	47
6.4	Lucas 定理：逐位计算组合数	48
6.5	去掉素因子后的阶乘递推	49
6.6	扩展 Lucas：先计算素数幂，再 CRT	50
6.7	原稿例题的完整建模	50
第 7 章	数论函数、狄利克雷卷积与筛法	53
7.1	先分清积性与完全积性	53

7.2	欧拉函数的公式与恒等式	54
7.3	狄利克雷卷积与逆元	54
7.4	线性筛：唯一的最小素因子分解	55
7.5	整除分块：相同商的最大区间	56
7.6	狄利克雷前缀和、后缀和与差分	57
第 8 章	莫比乌斯反演与 gcd 型求和	59
8.1	两种基础“拆贡献”恒等式	59
8.2	先处理单变量与互质计数	60
8.3	多询问与不规则预处理表	62
第 9 章	积性函数求和：杜教筛与 Powerful Number	65
9.1	先看规模，再看倍增估计	65
9.2	用卷积把求和变成递归	65
9.2.1	无预处理与带预处理的复杂度	66
9.3	Powerful Number：稀疏的高次素因子结构	68
9.4	把差异留给 PN 支撑	68
第 10 章	洲阁筛与素数贡献的分层统计	71
10.1	先把适用条件说清楚	71
10.2	第一阶段：把合数从简单前缀和中筛掉	71
10.2.1	只保留商集与滚动数组	72
10.3	第二阶段：用素数幂重建一般积性函数	72
10.3.1	复杂度说明与备课时的取舍	73
第 11 章	二维积性函数：消去主要贡献以得到稀疏支撑	75
11.1	问题、定义与二维卷积	75
11.2	算法一：先消去坐标轴	76
11.2.1	局部系数怎样实际求？	76
11.2.2	为何只有 $O(n)$ 个位置？一个更直接的证明	76
11.3	算法二：再消去局部对角线	77
11.3.1	稀疏支撑数为 $O((AB)^{1/3})$	78
11.3.2	将单次商块成本求和	78
11.4	推广：第一个非零局部系数决定主导规模	78
11.5	算法三：去掉枚举阶段的对数（研究拓展）	79
第 12 章	综合专题：循环、指数向量、图与期望	81
12.1	CF516E：快乐传播不是只保留最小起点	81
12.1.1	传播时间的精确形式	81
12.1.2	修正原稿的关键反例	81
12.1.3	把步长变成加一	82

12.1.4 环上的带权多源扫线	82
12.2 CF571E: 把等比数列交集转到指数空间	82
12.2.1 为什么不能对每个素数独立 CRT?	83
12.2.2 两条射线的完整分类	83
12.3 AGC031F: 把巨大的状态图缩成每点六个状态	84
12.3.1 第一步: 反向读路径消去长度变量	84
12.3.2 第二步: 推出同顶点的平移等价	84
12.3.3 第三步: 统一边权余数并只保留六类形式	84
12.3.4 第四步: 把询问数值匹配到形式	85
12.4 WC2020 猜数游戏: 把期望拆成分量贡献	85
12.4.1 先分清题目中的最优含义	86
12.4.2 期望线性性给出封闭计数式	86
12.4.3 不用真的建出平方规模的图	86
第 13 章 素性判定与因数分解	89
13.1 试除、费马测试与 Carmichael 数	89
13.2 Miller–Rabin 的强伪素数测试	89
13.3 随机误差与确定性底数	90
13.3.1 课堂可证明的较弱上界: $1/2$	90
13.4 Pollard–Rho: 在未知模素因子下找碰撞	91
13.4.1 生日悖论如何给出规模估计	91
13.4.2 批量 gcd 与故障回退	92
13.4.3 应怎样表述复杂度	92
13.5 位数模型与大整数题的边界	92
第 14 章 特殊信息如何把分解变成随机多项式问题	93
14.1 QOJ 4920: 已知 n 与 $\varphi(n)$	93
14.1.1 先理解一个二素数例子	93
14.1.2 保持原始的群阶倍数	93
14.1.3 一次随机尝试	94
14.2 QOJ 1860: 由 $M = N\varphi(N)$ 还原 N	94
14.2.1 第一步: 解确实唯一	94
14.2.2 第二步: 不必完全分解原数	95
14.2.3 一次递归怎样处理?	95
14.2.4 第三步: 为什么仍有常数成功概率?	95
14.2.5 最后按降序恢复答案	95
第 15 章 韦达跳跃与构造不定方程的解	97
15.1 把一个变量看成二次方程的根	97
15.1.1 推导跳跃公式而不是记变换	97

15.1.2 严格递降与停止条件	98
15.1.3 从基本解向上生成：两个孩子	98
15.1.4 一个适用于所有合法 k 的种子	98
15.1.5 BFS 与全局去重的可实现策略	99
15.1.6 正确性证明与规模证据分开陈述	99
附录 A 分层训练与参考解析	A1
A.1 A 组：整除、通解与基础建模	A1
A.2 B 组：指数、赋值与组合数	A3
A.3 C 组：反演与亚线性求和	A6
A.4 D 组：证明辨析与综合拓展	A7
附录 B 核心模板与接口	B1
B.1 模板使用说明	B1
B.2 安全模乘、快速幂与逆元	B2
B.3 最小非负离散对数	B2
B.4 素数幂模数下的组合数	B3
B.5 完整无符号 64 位的 Miller–Rabin	B4
B.6 怎样复现验证	B4
附录 C 教师使用：课堂设计、追问与评量	C1
C.1 如何安排一轮而不是一节“数论课”	C1
C.2 课堂包一：exgcd 从“能解”到“求最小解”	C1
C.3 课堂包二：指数取模的三个不同问题	C2
C.4 课堂包三：组合数中不能取逆的部分去哪了？	C2
C.5 课堂包四：反演是为了分离变量	C3
C.6 课堂包五：PN 与二维积性函数的“消主项”	C4
C.7 课堂包六：概率算法的答案为什么可信？	C4
C.8 一次完整作业的评分框架	C5
C.9 上机作业的最低测试清单	C5

第 0 章

数学用词与使用方法说明

章节导读

后面的数论章节会默认使用约数、同余、可逆、赋值、卷积、双射等术语。先把这些词和记号统一起来，后面遇到新算法时就可以把注意力放在数学本身，而不是被术语卡住。

这一章解决什么问题

作者在后面的章节默认你会说一种“数学黑话”：约数、模、可逆、赋值、卷积、双射、记忆化、阈值……这些词每一个似乎都能用一句话解释清楚，但正文一般不会专门说。本章把全书会用到、而高中课本不一定会说的词与记号一次交代清楚，并在末尾给出一张“高中叫法 $\leftrightarrow$ 本书叫法”的对照表和一段的证明示范。

本章不教新的数学知识。你以后在阅读正文遇到哪个词把你卡住了，就可以回到这里查它到底是什么意思。

0.1 怎么读这本书

正文里有四种带颜色的框，它们分工不同：

- **定义/定理/引理/推论/例**：黑色小标题，是必须记住的结论。看书时先问自己“条件是什么、结论是什么”，再往下看。
- **题目框**（深蓝色标题）：题目与它的解析。标“统一练习”的不是原题，而是把几道题共用的想法抽出来。
- **易错框**（橙棕色标题，标注“易错点”）：这里写的是“大多数人会错在哪”，以及一个具体反例。跳过它最省时间，也最亏。
- **本章新增的绿色 tip 框**：只解释一个词、一步跳步或一个例子。它可以单独读懂，不需要你先看懂上下文。

一句话学习方法

每读完一节，合上书回答三个问题：这一节要解决什么问题？它的条件少一条会怎样（举出反例）？我能把结论写成几行代码吗？三个问题答不上来，就说明只是“看过了”，不是“会了”。

0.2 集合、元素与逻辑用语

高中课本已经给了你绝大部分记号，这里只把本书用得多的补齐。

0.2.1 集合与元素

集合就是“一些确定的东西放在一起”，记 $a \in A$ 读作“ a 是 A 的元素”， $a \notin A$ 读作“不是”； $A \subseteq B$ 读作“ A 的每个元素都在 B 里”（ A 含于 B ）； $\emptyset$ 是空集，即不含任何元素的集合。两个集合相等，当且仅当它们互相含于。

本书最常用的几个数集：

$$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}, \quad \mathbb{N} = \{0, 1, 2, \dots\}, \quad \mathbb{P} = \{2, 3, 5, 7, 11, \dots\},$$

分别是整数集、非负整数集、素数集。 $\mathbb{Q}$ 、 $\mathbb{R}$ 是有理数集与实数集，本书只在说明“范围放宽了”时提到。注意本书约定 $\mathbb{N}$ 含 0；如果你所在的教材不含 0，读到 $\mathbb{N}$ 时就按本书的约定来。

描述法的写法 $\{x \in \mathbb{Z} : 0 \leq x < m\}$ 表示“满足后面条件的那些 x ”，它等于 $\{0, 1, \dots, m-1\}$ 。冒号也可写成竖线。 $\{0, 1, \dots, m-1\}$ 有 m 个元素，别把“从 0 到 $m-1$ ”数成 $m-1$ 个。

0.2.2 “或”“且”“当且仅当”

- “ P 且 Q ”：两者同时成立，符号 $P \wedge Q$ 。
- “ P 或 Q ”：至少一个成立（可以两个都成立），符号 $P \vee Q$ 。数学里的“或”不是“二选一”。
- “ $P \Rightarrow Q$ ”： P 成立时 Q 必须成立，读作“ P 推出 Q ”。这时也说“ P 是 Q 的充分条件”“ Q 是 P 的必要条件”。
- “ $P \iff Q$ ”（ $P \Leftrightarrow Q$ ）：互相推出，读作“ P 当且仅当 Q ”，也读作“ P 是 Q 的充要条件”。

“必要性来自……充分性来自……”是在说什么

这是正文证明里最常见的句式，翻译成人话就是：“两个方向分开证”。写 $P \iff Q$ 时，**必要性**指“已知 P 成立，去证 Q ”（不做到的话 P 就不成立，所以 Q 是必须的）；**充分性**指“已知 Q 成立，去造出 P ”（ Q 一条就足够了）。一个方向没证，结论就还只是猜测。本书第 0 章第 11 节会完整示范一次双向证明，第 1 章的裴蜀定理也给了不跳步的版本。

0.2.3 量词：任意与存在

$\forall$ 读“对任意（所有）”， $\exists$ 读“存在（至少有一个）”， $\exists!$ 读“存在且唯一”。顺序改变含义：

$$\forall n \in \mathbb{N}, \exists p \in \mathbb{P}, p > n \quad (\text{素数无穷多个}),$$

$\forall$ 在前表示“先给我 n ，我再找 p ”， p 可以依赖 n 。若写成 $\exists p, \forall n$ ，就变成“有一个固定素数比所有数都大”，荒谬。

否定规则要背熟： $\neg(\forall x, P(x))$ 等价于“存在一个 x 使 $P(x)$ 不成立”，即 $\exists x, \neg P(x)$ 。因此要推翻“所有情况都好”，只需举一个反例；反之，要反驳“存在坏情况”，必须证明“全都好”。

0.2.4 Iverson 括号与“示性函数”

本书写 $[\gcd(i, n) = 1]$ ，读作“Iverson 括号”：方括号里是一句话（命题），成立时这个式子的值为 1，不成立时为 0。例如 $[5 > 3] = 1$ ， $[4 \text{ 是奇数}] = 0$ 。

它的好处是把“只在某些情况下计入”写成算式，从而能自由交换求和顺序：

$$\sum_{i=1}^n [i \text{ 与 } n \text{ 互质}] = \text{“1 到 } n \text{ 中与 } n \text{ 互质的数的个数”}.$$

看到 $[\cdot]$ 别慌，它只是一个“条件成立记 1、否则记 0”的开关。

0.3 求和、取整、下标：把式子读成程序

0.3.1 求和与求积

$\sum_{i=1}^n a_i = a_1 + a_2 + \cdots + a_n$, $\prod_{i=1}^n a_i = a_1 a_2 \cdots a_n$ 。下标从 1 还是 0 开始、到 n 还是 $n-1$ 结束，务必按式子读；差一项在很多题里就是 100 分与 0 分的差距。空和（一项都没有）约定为 0，空积约定为 1，这是为了让递推式在边界也成立。

换序（“交换求和”）是本书用得最多的技术，合法条件是：项数有限。有限项相加，先算哪一项都行。示例：

$$\sum_{d=1}^n \sum_{\substack{k \geq 1 \\ dk \leq n}} 1 = \sum_{m=1}^n \sum_{d|m} 1,$$

左边是“对每个 d 数它有多少个不超过 n/d 的倍数 k ”，右边是“对每个 m 数它有多少个约数 d ”；两边数的是同一批满足 $dk \leq n$ 的整数对 (d, k) ，所以相等。遇到换序，先像这样把“数的是什么”说清楚，比背公式可靠。

0.3.2 取整

$\lfloor x \rfloor$ 是“不超过 x 的最大整数”（向下取整，floor 是地板）， $\lceil x \rceil$ 是“不小于 x 的最小整数”（向上取整）。例如 $\lfloor 7/2 \rfloor = 3$, $\lceil 7/2 \rceil = 4$, $\lfloor -7/2 \rfloor = -4$ （注意不是 -3 ： -4 才是不超过 -3.5 的最大整数）。

两条最常用性质：对整数 a 与正整数 b ,

$$\left\lfloor \frac{a}{b} \right\rfloor = q \iff qb \leq a < (q+1)b, \quad \left\lceil \frac{a}{b} \right\rceil = q \iff (q-1)b < a \leq qb.$$

第二条也可以写成 $\lceil a/b \rceil = \lfloor (a+b-1)/b \rfloor$ ($a \geq 0$ 时)。C++ 的 `‘/’` 是**向零取整**（截断），不是向下取整： $-7/2 = -3 \neq \lfloor -3.5 \rfloor$ 。所以本书模板里所有“整除取商”都写成 `floor_div`（见附录“核心模板与接口”与第 1 章）。

0.3.3 下标、递推与“记号说明”

a_i 读“ a 下标 i ”，是数列的第 i 项（本书默认从 a_1 开始，除非写了 a_0 ）。“递推式”是用前一项表示后一项，例如 $T_h = a^{T_{h-1}}$ ；要它可用，必须同时给**起点** ($T_0 = 1$) 和**方向** (h 递增)。看到“类似地”“同理”“不难验证”，但你自己验不出来，就把它当成一个待填的空。

0.4 整数：约数、倍数、最大公约数与素因数分解

- **整除**： $a \mid b$ 读“ a 整除 b ”，意思是“存在整数 k 使 $b = ka$ ”。此时 a 是 b 的**约数**（高中课本常叫“因数”）， b 是 a 的**倍数**。规定 $0 \mid 0$ 成立（取 k 任意），而 $a \mid b$ 中若 $a = 0, b \neq 0$ 则不成立。符号 $\nmid$ 表示“不整除”。
- **公约数与最大公约数**：同时整除 a, b 的正整数叫公约数，其中最大的叫 $\gcd(a, b)$ (greatest common divisor)。本书也写 (a, b) 表示同一个东西。
- **公倍数与最小公倍数**：同时被 a, b 整除的正整数中最小的叫 $\text{lcm}(a, b)$ 。有 $\gcd(a, b) \text{lcm}(a, b) = ab$ ($a, b > 0$)。
- **互质**： $\gcd(a, b) = 1$ ，即“公约数只有 1”。注意：1 与 1 互质；0 与 1 互质；0 与 5 不互质 ($\gcd(0, 5) = 5$)。两个互质的数不必都是素数 (8 与 9)。

- **素数与合数**：大于 1 且约数只有 1 与自身的叫素数（本书与“质数”混用，同义）；大于 1 又不是素数的叫合数。1 既不是素数也不是合数。
- **唯一分解定理**：每个大于 1 的整数都能写成素数的乘积，且不计顺序时写法唯一， $n = p_1^{e_1} \cdots p_t^{e_t}$ 。于是“ n 的约数”与“指数向量 $(f_1, \dots, f_t)$, $0 \leq f_j \leq e_j$ ”一一对应，约数个数为 $\tau(n) = \prod_j (e_j + 1)$ 。本书把原稿用的 $d(n)$ 统一记作 $\tau(n)$ ，避免与 gcd 的括号写法混淆。
- **素因数（本书常写“素因子”）**：出现在分解式里的素数。 $\text{rad}(n) = \prod_{p|n} p$ 是所有不同素因子的乘积。
- **平方自由**：分解式中每个指数都 ≤ 1 ，即不被 $4, 9, 25, \dots$ 这类平方数整除。例如 $30 = 2 \cdot 3 \cdot 5$ 平方自由， $12 = 2^2 \cdot 3$ 不是。
- **素数幂**：形如 $Q = p^\alpha$ 的数（ p 为素数， $\alpha \geq 1$ ）。本书大量结论先在素数幂下证明，再用中国剩余定理拼回一般模数，这条路线第 3 章起会反复出现。

“素因子”与“质因数”是同一个东西

中学课本说“分解质因数”，大学教材与竞赛资料常说“素因子”“素因数”。本书四个词混用，含义相同。类似地，“最大公约数”与“最大公因数”相同。看到新词先怀疑它是不是老词的别名，这是读数学资料省时间的诀窍。

为什么要关心“平方自由”

因为很多计数公式里会出现一个开关： $\mu(n)$ （莫比乌斯函数）。它的定义只有一行—— $n = 1$ 时为 1； n 平方自由且有偶数个素因子时为 1、奇数个时为 -1 ；一旦有平方因子就是 0。所以“平方自由”不是趣味知识，而是“哪些项会消失”的判断条件。第 7 章与第 8 章大量使用它。

0.5 同余：按余数把整数分堆

$a \equiv b \pmod{m}$ 读作“ a 与 b 模 m 同余”，定义就是 $m \mid a - b$ 。它只关心“除以 m 的余数是否相同”，所以两个数同余，等价于它们落在同一堆里。把整数按余数分成 m 堆，这 m 堆合起来称为**完全剩余系**；每一堆里挑一个代表写出来的数叫**代表元**，本书默认用 $0, 1, \dots, m-1$ （最小非负代表元）。

为什么同余式能加减乘？因为 $(a \pm b) \bmod m$ 只由 a, b 各自的余数决定； $m \mid a - b$ 且 $m \mid c - d$ 时 $m \mid (a + c) - (b + d)$ 、 $m \mid ac - bd$ 。所以“每步取模”不改变结果。**除法不行**：除法是同余式里唯一的陷阱，它要求先检查可除性，且往往要把模数一起缩小，见第 2 章的约分定理与易错框。

两个必记词：

- **模运算的“负数”**： $-1 \bmod 7 = 6$ ，不是 -1 。规范做法是先取余再加模数。C++ 里写成 $((a \% m) + m) \% m$ ；本书模板中的 `norm(a, m)` 干的就是这件事。
- **单位（可逆元）**：若存在 x 使 $ax \equiv 1 \pmod{m}$ ，称 a 是模 m 的**单位**， x 叫 a 的**逆元**。 $m = 7$ 时 1 到 6 都是单位； $m = 8$ 时只有 1, 3, 5, 7 是单位（偶数不行，因为 gcd 不为 1）。这些单位凑在一起，就是下面 0.8 节说的“单位群”。

“剩余系”这三个字到底在说什么

把 0 到 $m-1$ 排成一圈，只保留余数信息： $m = 5$ 时的剩余系是 $\{0, 1, 2, 3, 4\}$ 。其中与 5 互质的有 $\{1, 2, 3, 4\}$ ，这就是“简化剩余系”，它的个数记 $\varphi(5) = 4$ 。 $m = 8$ 时全部 8 个余数里，与 8 互质的是 $\{1, 3, 5, 7\}$ ，所以 $\varphi(8) = 4$ 。“乘以 2”在 $\{0, 1, \dots, 7\}$ 上不是一一对应，因为 $1 \cdot 2$ 与 $5 \cdot 2$ 都是 2；但“乘以 3”在四个单位 $\{1, 3, 5, 7\}$ 上是： $1 \rightarrow 3, 3 \rightarrow 1, 5 \rightarrow 7, 7 \rightarrow 5$ ，每个余数恰好出现一次——这就是一次“双射”的手算。这个“在互质的那几格里可以一一对应”的事实，就是第 2 章证明欧拉定理时最关键的一步。

0.6 幂、阶乘与组合数

a^n 表示 n 个 a 相乘, $a^0 = 1$ (包括 $0^0 = 1$, 本书按组合意义约定)。指数律 $a^m a^n = a^{m+n}$ 、 $(a^m)^n = a^{mn}$ 在整数指数下要额外注意 $a = 0$ 与负指数。

$n! = 1 \cdot 2 \cdots n$, 并约定 $0! = 1$ 。 $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ 是“从 n 个里选 k 个”的方法数, 高中课本里的写法也是 C_n^k ; 本书用 $\binom{n}{k}$, 因为分式形式在“约掉公共因子”时更好用。二项式定理: $(x+y)^n = \sum_k \binom{n}{k} x^k y^{n-k}$ 。

幂塔 (“高德纳箭号”的最小情形): a^{b^c} 按**右结合**读, 等于 $a^{(b^c)}$, 而不是 $(a^b)^c = a^{bc}$ 。 $3^{3^3} = 3^{27}$ 已经约 7.6×10^{12} , 所以处理幂塔的核心矛盾是: 指数巨大, 而余数只有 m 种可能。第 2 章的“扩展欧拉定理”与“截断塔接口”就是为此准备的。

“降幂”是什么意思

“降幂”不是初中因式分解里的“降次”。在数论里, 它指: 要算 $a^E \bmod m$, 但 E 大到写不下来, 于是把 E 换成一个更小、余数不变的指数 (例如 $E \bmod \varphi(m)$)。这个替换**只在特定条件下合法**, 条件不满足时必须保留“ E 是否已经很大”这一位信息。第 2 章用 $2^4 \bmod 8$ 这个反例演示了乱降幂的错误。

$\nu_p(n)$: 一个只有“数因子”动作的记号

$\nu_p(n)$ 读作“ n 的 p -进赋值”, 做法是: 拿 n 不断除以 p , 能除几次就是几。例如 $\nu_2(24) = 3$ ($24 \rightarrow 12 \rightarrow 6 \rightarrow 3$, 停了), $\nu_3(24) = 0$, $\nu_5(100) = 2$ 。它满足两条: $\nu_p(xy) = \nu_p(x) + \nu_p(y)$ (乘法变加法), 以及 $\nu_p(x+y) \geq \min(\nu_p(x), \nu_p(y))$ (加法只能“更好”, 除非两项赋值相同才可能大幅抵消)。第 5 章的升幂引理 (LTE) 本质上就是“什么时候恰好等于最小值”的判据。看到“赋值”这个词, 先在心里换成“这个素因子出现了几次”。

0.7 数论函数、前缀和与卷积

数论函数只是“定义在正整数上的函数” $f: \mathbb{N}_{>0} \rightarrow \mathbb{Z}$ (或 $\mathbb{R}$): $\varphi(n)$ 数互质个数、 $\tau(n)$ 数约数个数、 $\mu(n)$ 是上面那个开关。别把它想得太高深, 它其实就是一张表: $f(1), f(2), f(3), \dots$

- **积性与完全积性**: 若 $\gcd(a, b) = 1$ 时总有 $f(ab) = f(a)f(b)$, 叫 f 是**积性**的; 若不要求互质也成立, 叫**完全积性**的。例: $\text{id}_t(n) = n^t$ 完全积性; φ 、 τ 、 μ 只是积性 (拿 $f = \tau$, $a = b = 2$ 试: $\tau(4) = 3 \neq \tau(2)\tau(2) = 4$, 一眼看出“互质”这条不能丢)。有了积性, 只要知道 f 在每个素数幂上的值, 整张表就全知道了。
- **狄利克雷卷积**: 给两个数论函数定义一种新乘法 $(f * g)(n) = \sum_{d|n} f(d)g(n/d)$ 。它不是多项式乘法, 也不是逐点相乘。手算一次就懂: $f = \mathbf{1}$ (常一函数, $\mathbf{1}(n) \equiv 1$)、 $g = \text{id}_1(n) = n$, 则 $(\mathbf{1} * \text{id}_1)(6) = 1 \cdot 6 + 1 \cdot 3 + 1 \cdot 2 + 1 \cdot 1$ 按 $d = 1, 2, 3, 6$ 取 $\text{id}_1(6/d)$, 等于 $6 + 3 + 2 + 1 = 12$, 正好是 6 的约数和。本书用 $\sum_{d|n}$ 表示“对 n 的每个约数 d 求和”。
- **单位函数** ε : $\varepsilon(1) = 1$ 、 $\varepsilon(n) = 0$ ($n > 1$)。它扮演“数字 1”的角色: $f * \varepsilon = f$ 。于是“卷积逆”就是满足 $f * f^{-1} = \varepsilon$ 的那个 f^{-1} ——可以把它完全类比为“倒数”, 只是这里的乘法换成了上面的卷积。
- **前缀和**: $S_f(n) = \sum_{i=1}^n f(i)$, 即“表的前 n 项之和”, 约定 $S_f(0) = 0$ 。第 9 章“杜教筛”要解决的问题只有一个: n 高达 10^{10} , 表建不出来, 怎么仍算出 $S_f(n)$ 。

为什么要费劲定义一种“新乘法”

因为很多数论恒等式会变成“乘法等式”。经典例子： $\sum_{d|n} \varphi(d) = n$ ，用卷积语言就是 $\mathbf{1} * \varphi = \text{id}_1$ 。一旦写成这样，两边同时“乘 μ ”（ μ 是 $\mathbf{1}$ 的卷积逆）就自动得到 $\varphi = \mu * \text{id}_1$ ，反演公式不再需要灵光一现。第 7 章与第 8 章的整套机械就是把这个类比推到底。初学时你要掌握的只有三件事：怎么按定义展开、 $*$ 满足交换律与结合律、 $\mathbf{1}$ 的逆是 μ 。

0.8 “群、环、域、同态”：这一页只需听懂，不必学会

本书不要求你会群论，但正文会用这些词压缩语言。下面是够用的最小版本：

- **群**：一个集合 G 加一种运算（写成乘法），满足：任取两元素运算结果还在 G 里（封闭）； $(ab)c = a(bc)$ （结合律）；有“什么都不做”的元素 e （单位元）；每个元素都有能把结果变回 e 的逆元。例： $\mathbb{Z}$ 与加法、非零有理数与乘法。
- **子群**： G 的一部分，自己也是群。例： $\mathbb{Z}$ 里的偶数在加法下是子群。
- **阶**：有限群的元素个数叫群的阶；元素 a 的阶是使 $a^r = e$ 的最小正整数 r 。本书第 4 章的“乘法阶”就是它在模 m 乘法下的具体版本。
- **循环群**：整个群由一个元素不断自乘得到（如模 7 的单位 $\{1, 2, 3, 4, 5, 6\}$ ，取 3：3, 2, 6, 4, 5, 1 恰好遍历）。那个能遍历全体的元素就叫**原根**。
- **单位群**： $(\mathbb{Z}/m\mathbb{Z})^\times$ 表示“模 m 的可逆余数全体，配上乘法”。它的阶是 $\varphi(m)$ 。第 2 章说“乘以 a 是单位群上的双射”，意思就是“在这张余数表上乘同一个可逆数，元素只是换了位置，一个不多一个不少”。
- **环**：可以做加、减、乘（不必能除）的体系，如 $\mathbb{Z}$ 、模 m 的余数集合。
- **域**：环上还额外保证“非零元素都能除”，如 $\mathbb{Q}$ 、 $\mathbb{R}$ ，以及素数模下的余数 $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ 。这就是为什么“ d 次多项式至多 d 个根”在 $\mathbb{F}_p$ 成立、在 $\mathbb{Z}/8\mathbb{Z}$ 不成立（ $x^2 = 1$ 模 8 有 4 个根）。
- **同态/同构**：“两个体系结构相同”的精确说法。同态是保持运算的映射 $f(xy) = f(x)f(y)$ ；同构还是一一对应。本书只在“指数把乘法变成加法”这类场合用一次，见第 4 章。

遇到“群”字时的实用策略

先把它换成“一组余数 + 一种运算”，再问：这些余数是谁（是不是都得跟模数互质）？运算是取模后的普通乘法吗？90% 的场合这么一换，句子就懂了。真需要群论工具时（第 12、14 章的少数段落），正文会把要用的性质单独陈述成引理，不会拿“由群论知”糊弄你。

0.9 算法行话：复杂度、预处理、在线、对拍、溢出

- **大 O** ： $T(n) = O(f(n))$ 表示存在常数 C 与 n_0 ，使 $n \geq n_0$ 时 $T(n) \leq Cf(n)$ 。它只关心增长快慢，不承诺绝对时间，也不管常数。本书还写 O, Ω, Θ （下界、同阶）与“期望 $O(\cdot)$ ”（随机算法在平均意义下的界）。
- **字长模型**：竞赛里默认 64 位整数加减乘比较都是 $O(1)$ 。因此“ n 位数”的高精度乘法是 $O(n^2)$ ，而不是 $O(1)$ 。本书说某算法“多项式时间”时，变量是位数 $\log n$ ； $O(n^{1/4})$ 关于位数**不是**多项式。
- **预处理 / 查询**：预处理是开始前先算好一张表，查询是每次问一行。“ $O(N)$ 预处理、 $O(1)$ 查询”与“ $O(1)$ 预处理、 $O(\log N)$ 查询”是两种不同取舍，总时间相同也可能一个内存爆一个不爆。本书凡给复杂度都分开报这两项。
- **离线 / 在线**：在线是“来一问答一问，不许先看后面”；离线是“所有问题一次读完，可以重排、分块、按模数分组”。同一道题，两种口径的最优做法可能完全不同。

- **记忆化**：递归时把算过的结果存进哈希表或数组，下次直接查。它把“指数级重复计算”压成“状态数个”，是第 9 章杜教筛不可缺的一步。
- **摊还**：单步可能很贵，但 m 步的总代价除以 m 很小。典型：并查集、单调栈。别把“摊还 $O(1)$ ”读成“每步 $O(1)$ ”。
- **对拍**：写一个笨但显然正确的程序（暴力枚举），与要提交的程序在同一批随机数据上比答案。它是“找反例”的工程化版本，本书 `validation/` 与 `code/selftest.cpp` 干的就是这件事。
- **阈值**：某个“从此以后行为不同”的数值。例如扩展欧拉定理里的 $\varphi(m)$ ：指数不到它不能乱换，超过它才能换。阈值不是“经验值”，它由证明里的不等式给出。
- **溢出**： $10^9 \cdot 10^9 = 10^{18}$ 还能塞进 `long long`（上限约 9.22×10^{18} ）， $(2 \cdot 10^9)^2$ 就超了。本书模板用 `__int128` 做中间量、用 `mul_mod` 收口；`unsigned` 溢出是回绕而非未定义行为，别把两者混为一谈。
- **取模的正确姿势**：减法后可能为负，要 `norm`；除法不是“分子分母各自取模”，而是“乘分母的逆元”，且分母必须先确认与模数互质。

0.10 高中叫法 $\leftrightarrow$ 本书叫法

高中课本 / 常见叫法	本书叫法	差别与注意
因数	约数	同义；本书的“约数”含 1 与自身，且只在正整数范围讨论。
辗转相除法	欧几里得算法 / gcd	同一种算法；本书强调复杂度来自“每两轮规模至少减半”。
$(a, b) = 1$ 分解质因数	$\gcd(a, b) = 1$ 、互质 素因数分解	本书偶尔也用 (a, b) 表 gcd，出现时会说明。同义；本书还会写“拆素因子”“按素数幂拆开”。
“ a 除以 m 余 r ”	$a \bmod m = r$ 、 $a \equiv r \pmod{m}$	前者是运算、后者是关系；负数时本书规定余数落在 $[0, m)$ 。
排列、组合 C_n^k	$\binom{n}{k}$	同义；本书用分式形式，便于讨论“分母能否取逆”。
二项式定理	$(1 + X)^n$ 的系数	同一件事；第 6 章用“模 p 后只剩两项”推出 Lucas 定理。
数学归纳法 等差数列	归纳 / 最小反例 环上的固定步长	“取最小反例”与归纳等价，本书两种都说。在 $0, 1, \dots, n-1$ 上“每次加 k ”就是绕圈走的等差数列。
等比数列	几何数列 / 幂序列	第 12 章的“指数向量”研究它的周期。
充分必要条件	当且仅当、 $\iff$	双向都要证，见 0.2 节的 tip。
集合、 $\in$ 、 $\subseteq$ 、 $\emptyset$	同高中	本书默认 $\mathbb{N}$ 含 0。
对数 $\log$	$\log$ 、 $\log \log$	无特别说明时本书大 O 里的 $\log$ 底数不影响结论； $\log \log$ 增长极慢。
绝对值 $ x $	同	赋值符号 ν_p 与绝对值无关，别混淆。
函数	数论函数（一张表）	见 0.6 节。
“显然”	偶尔用于省略一层代数	你若验算不出来，就当它是作业。

0.11 证明的三种句式，以及“不跳步”长什么样

0.11.1 双向证明

要证“ P 当且仅当 Q ”，写两个方向、各起一段，并给每段一个开头词（“必要性.” “充分性.”）。若只证了一边，就把结论降格为“ P 推出 Q ”。

0.11.2 反证与最小反例

要证“所有 n 都满足 $P(n)$ ”，可设存在反例，取**最小**的那个 n_0 ，然后由 n_0 造出更小的反例 $n' < n_0$ ，矛盾。这就是“最小反例法”，它与数学归纳法等价。本书第 1 章证明“欧几里得算法正确”、第 15 章的韦达跳跃都用它，因为它们都需要“ n_0 的最小性”这一件武器。

0.11.3 归纳

写清三件事：命题 $P(n)$ 是什么； $P(1)$ （或 $P(0)$ ）成立；“若对所有 $k < n$ 成立则 $P(n)$ 成立”。需要“所有更小的”而不是只有 $n-1$ 时，叫第二归纳法，本书会说“强归纳”。

0.11.4 示范：把一段跳步的证明补全

正文第 1 章的裴蜀定理原文只写了两句“必要性来自……充分性来自……”。下面是它应有的样子——请把它当模板，而不是当额外负担：

示范命题. 设 a, b 为整数，不全为零， $d = \gcd(a, b) > 0$ ， c 为整数。则 $ax + by = c$ 有整数解 $\iff d \mid c$ 。

证明. 必要性（有解 $\Rightarrow d \mid c$ ）。假设存在整数 x, y 使 $ax + by = c$ 。由 d 是公约数，可写 $a = da'$ 、 $b = db'$ ，其中 a', b' 是整数。代入得

$$c = ax + by = da'x + db'y = d(a'x + b'y).$$

括号里是整数（整数对加、乘封闭），所以 d 整除 c 。这一步没有用到任何“显然”的性质，只是把 d 提出来。

充分性（ $d \mid c \Rightarrow$ 有解）。分两小步。

第一步：先证 d 本身能被写成 a, b 的整数线性组合，即存在 u, v 使 $au + bv = d$ 。对较小的数归纳：若 $b = 0$ ，则 $d = a$ ，取 $u = 1, v = 0$ 即可；若 $b > 0$ ，写 $a = qb + r$ ， $0 \leq r < b$ 。由归纳假设用于更小的对 (b, r) （因为 $\gcd(b, r) = \gcd(a, b) = d$ ，见第 1 章定理），存在 u_1, v_1 使 $bu_1 + rv_1 = d$ 。把 $r = a - qb$ 代回去：

$$d = bu_1 + (a - qb)v_1 = av_1 + b(u_1 - qv_1),$$

于是可取 $u = v_1$ 、 $v = u_1 - qv_1$ ，它们都是整数。归纳完成。

第二步：把右边的 d 放大成 c 。因为 $d \mid c$ ，记 $c = kt$ （ t 为整数）。把第一步的等式两边同乘 t ：

$$a(ut) + b(vt) = dt = c.$$

取 $x = ut$ 、 $y = vt$ 即为所求。注意方向：是“乘 c/d ”，不是“乘 d/c ”——原稿在这里写过反，第 1 章的解析会再强调一次。□

怎么判断自己“真的读懂了一个证明”

拿一张白纸，只看定理陈述，自己把证明重建一遍。卡住的地方就是你不理解的地方。第二遍再看书，只读你卡住的那两行。这样读十页，比顺着看五十页有效。

0.12 自查小测

每题答案不超过两行。做错的题，回到对应小节。

1. $\lfloor -7/2 \rfloor$ 与 C++ 里 $-7/2$ 的值各是多少？
2. 写出 “ $\forall x \exists y P(x, y)$ ” 的否定。
3. $\tau(72)$ 、 $\mu(72)$ 、 $\nu_2(72)$ 分别是多少？($72 = 2^3 \cdot 3^2$)
4. $-1 \bmod 7$ 是多少？2 在模 8 下有逆元吗？在模 7 下呢？
5. 17^{3^2} 与 $(17^3)^2$ 相等吗？
6. 为什么说 “ $0! = 1$ ” 不任性，而是为了递推 $\binom{n}{0} = 1$ 与空积约定一致？
7. 验证 “2 在模 8 下没有逆元”，并说出用的是哪一条判别条件。
8. “ f 是积性函数” 与 “ f 是完全积性函数”，哪个更强？用 τ 说明。
9. 一句话说清 “预处理” 和 “查询” 的区别，并给一个本书的例子。
10. “取最小反例” 与哪种证明方法等价？

参考答案

1) 都是 -4 与 -3 。2) $\exists x \forall y \neg P(x, y)$ 。3) $12, 0, 3$ 。4) 6 ；没有 ($\gcd(2, 8) = 2$)；有， $2^{-1} \equiv 4 \pmod{7}$ 。5) 不相等， $17^9 \neq 17^6$ ——幂塔右结合。6) 若 $0! = x$ ，由 $1! = 1 \cdot 0!$ 得 $x = 1$ 。7) 逐个检查 $2x \bmod 8$ ($x = 0, \dots, 7$) 得 $0, 2, 4, 6, 0, 2, 4, 6$ ，从不等于 1，故无逆元；依据是 “可逆 $\iff$ 与模数互质”，而 $\gcd(2, 8) = 2$ 。8) 完全积性更强； $\tau(2 \cdot 2) = 3 \neq 4$ 。9) 预处理是开始前建表、查询是每次读表，如第 2 章 $O(m^{2/3})$ 预处理 + $O(1)$ 查询。10) 数学归纳法。

第 1 章

整除、最大公约数与线性不定方程

章节导读

上一章我们先把本书会反复出现的数学语言统一了；这一章开始真正进入数论。我们先从“一个数能不能整除另一个数”出发，用最大公约数整理这种关系，再进一步把它变成可以求解的线性不定方程。

1.1 从质因数分解理解整除

对正整数 $n = \prod_p p^{\alpha_p}$ ，只有有限个 α_p 非零。若 $a = \prod p^{u_p}, b = \prod p^{v_p}$ ，则

$$a \mid b \iff \forall p, u_p \leq v_p, \quad \gcd(a, b) = \prod_p p^{\min(u_p, v_p)}, \quad \text{lcm}(a, b) = \prod_p p^{\max(u_p, v_p)}.$$

把整数看成指数向量

唯一分解让你把 n 记成一串指数： $72 \leftrightarrow (3, 2, 0, 0, \dots)$ 对应素数 $2, 3, 5, 7, \dots$ 。于是 $a \mid b$ 变成**每一位都不超过**， $\gcd$ 变成逐位取小， lcm 变成逐位取大。整章的许多结论都能这样**逐位**证明——看到 $\prod p^{u_p}$ 就先想这个画面。

因此 $\gcd(a, b) \text{lcm}(a, b) = ab$ 。实际计算最小公倍数时先除后乘： $a / \gcd(a, b) \cdot b$ ，仍要检查乘法是否溢出。

导入：把“共同部分”分离出来

在黑板上写 $12 = 2^2 \cdot 3, 18 = 2 \cdot 3^2$ ，请学生分别用“每种素数取较小指数”和“取较大指数”得到 $6, 36$ 。接着引出最常用换元

$$d = \gcd(a, b), \quad a = du, \quad b = dv, \quad \gcd(u, v) = 1.$$

换元最后一句不能省略。许多数论题不是缺少定理，而是没有把公共因子从互质部分中分离出来。

为什么：先除后乘

先除后乘是为了不溢出： $a / \gcd(a, b) \cdot b$ 与 $a \cdot b / \gcd(a, b)$ 数学上相同，但后者的中间值 ab 可能超过 9.22×10^{18} 。写成代码时，两个 10^9 级整数相乘就已经贴着上限。

1.2 欧几里得算法及其复杂度

定理 1.1. 若 $b > 0$ ，则 $\gcd(a, b) = \gcd(b, a \bmod b)$ ；终止条件为 $\gcd(a, 0) = a$ ($a \geq 0$)。

证明. 先固定一次带余除法: $a = qb + r$, 其中 $q = \lfloor a/b \rfloor$, $0 \leq r < b$ ($b > 0$ 时这样的 q, r 存在且唯一)。

第一步 (公约数不会变少)。设整数 c 同时整除 a 与 b , 写 $a = ca'$, $b = cb'$ 。由 $r = a - qb$ 得

$$r = ca' - q(cb') = c(a' - qb'),$$

括号中是整数, 所以 $c \mid r$ 。于是 c 也同时整除 b 与 r 。

第二步 (公约数不会变多)。设 $c \mid b$ 且 $c \mid r$, 写 $b = cb''$, $r = cr''$ 。由 $a = qb + r$ 得

$$a = q(cb'') + cr'' = c(qb'' + r''),$$

故 $c \mid a$, 即 c 同时整除 a 与 b 。

第三步 (取最大)。前两步说明 a 与 b 的公约数集合和 b 与 r 的公约数集合互为子集, 因此完全相同。两个集合都非空 (1 在内) 且有限 (公约数不超过 $\min(a, b)$), 故最大元素相同, 这正是 $\gcd(a, b) = \gcd(b, r)$ 。

终止条件: $r = 0$ 时 $\gcd(a, 0) = a$, 因为整除 a 与 0 的数就是 a 的约数 (0 被任何非零整数整除), 其中最大的为 a 。□

若 $a \geq b > 0$, 则 $a \bmod b < a/2$: 当 $b \leq a/2$ 时余数小于 b ; 否则商为 1, 余数为 $a - b < a/2$ 。所以每两轮规模至少减半, 复杂度为 $O(1 + \log \min(a, b))$ 。更细地, 令 $d = \gcd(a, b)$, 可写成

$$O\left(1 + \log \min\left(\frac{a}{d}, \frac{b}{d}\right)\right).$$

口径: 每两轮规模至少减半

每两轮规模至少减半是把两行不等式合起来说的: 一轮里 a 变成 b (b 可能只比 a 小一点点, 比如 $a = b + 1$), 但下一轮的余数一定小于 $b/2$ 。所以**规模**指两个数中的较大者, 它每两轮才保证减半。

手算例. 对 $(252, 198)$ 依次得到

$$252 = 198 + 54, \quad 198 = 3 \cdot 54 + 36, \quad 54 = 36 + 18, \quad 36 = 2 \cdot 18.$$

故 $\gcd$ 为 18。这里不是每一轮两个数都减半, 而是隔两轮观察同一位置上的数; 此区别是讲解复杂度时的常见含糊点。

追问. 连续斐波那契数为什么让欧几里得算法递归较深? 因为每一步商多为 1, 规模缩减相对缓慢。可以让学生实际记录 $(89, 55)$ 的调用链。

1.3 裴蜀定理与扩展欧几里得

定理 1.2 (裴蜀定理). 设整数 a, b 不全为零, $d = \gcd(a, b) > 0$ 。方程 $ax + by = c$ 有整数解, 当且仅当 $d \mid c$ 。更一般地, $\sum_i a_i x_i = c$ 有解当且仅当 $\gcd(a_1, \dots, a_n) \mid c$ 。

证明. 先证必要性。假设存在整数 x, y 使 $ax + by = c$ 。由于 $d = \gcd(a, b)$, 所以 $d \mid a$ 且 $d \mid b$, 可写成 $a = da'$, $b = db'$ 。于是

$$c = ax + by = d(a'x + b'y),$$

括号里的数是整数, 因此 $d \mid c$ 。

再证充分性。首先证明更强的结论: 存在整数 u, v 使

$$au + bv = d.$$

对欧几里得算法的递归深度归纳。当 $b = 0$ 时, $d = \gcd(a, 0) = a$, 取 $u = 1, v = 0$ 即可。当 $b > 0$ 时写 $a = qb + r$, 其中 $0 \leq r < b$ 。由 $\gcd(a, b) = \gcd(b, r) = d$, 对更小的二元组 (b, r) 应用归纳假设, 存在整数 u_1, v_1 使

$$bu_1 + rv_1 = d.$$

又 $r = a - qb$, 代入得到

$$d = bu_1 + (a - qb)v_1 = av_1 + b(u_1 - qv_1),$$

因此取 $u = v_1, v = u_1 - qv_1$ 即得 $au + bv = d$ 。

最后, 由 $d \mid c$, 设 $t = c/d \in \mathbb{Z}$ 。将 $au + bv = d$ 两边乘以 t , 得

$$a(ut) + b(vt) = dt = c.$$

所以 $x = ut, y = vt$ 即为整数解。

多元情形对 n 归纳。 $n = 2$ 已证。 $n > 2$ 时令 $g = \gcd(a_1, \dots, a_{n-1})$ 。由归纳假设, 可将前 $n-1$ 项表示为 g ; 于是原方程先化为二元方程 $gt + a_n x_n = c$ 。又 $\gcd(g, a_n) = \gcd(a_1, \dots, a_n) = d$, 故由二元情形可解。将得到的 t 乘回前 $n-1$ 个系数, 即得到原方程的整数解。□

思考: 这份证明为什么要这样走?

已知 $d = \gcd(a, b)$ 并且 $d \mid c$ 。我们真正要证明的是存在整数 x, y , 使 $ax + by = c$ 。这里不要一上来猜 x, y ; 先把目标拆开来看。

为什么先证明 d 能写成 $au + bv$? 如果能找到整数 u, v 使 $au + bv = d$, 那么因为 $d \mid c$, 就可以写成 $c = dt$, 其中 $t = c/d$ 是整数。把 $au + bv = d$ 整体乘以 t , 马上得到 $a(ut) + b(vt) = c$ 。所以充分性的真正关键只有一个: 为什么 d 一定能写成 a, b 的整数线性组合?

为什么 d 能写成 $au + bv$? 这里其实就是把欧几里得算法反着走。先看最简单的情况 $b = 0$ 。这时 $d = \gcd(a, 0) = a$, 所以直接有 $a \times 1 + 0 \times 0 = a = d$ 。也就是说, 取 $u = 1, v = 0$ 就可以了, 这就是递归的起点。

当 $b > 0$ 时怎么办? 对 a, b 做一次带余除法, 写成 $a = qb + r$, 其中 $0 \leq r < b$ 。例如 $a = 17, b = 5$ 时, 就是 $17 = 3 \times 5 + 2$ 。欧几里得算法最重要的性质之一是 $\gcd(a, b) = \gcd(b, r)$, 因此原来的 d 同时也是 $\gcd(b, r)$ 。

注意这里真正发生了什么: 原来我们面对的是 a, b , 现在变成了 b, r ; 而且 $r < b$, 所以问题规模变小了。这就是为什么可以使用归纳假设。

对更小的 b, r , 归纳假设告诉我们什么? 因为 $d = \gcd(b, r)$, 根据归纳假设, 存在整数 u_1, v_1 , 使 $bu_1 + rv_1 = d$ 。

这里特别容易看错。我们现在得到的不是 $au_1 + bv_1 = d$, 因为这一层归纳处理的是 b, r , 所以得到的自然是 b, r 的线性组合。现在的问题只是: 组合中还出现了 r , 而我们的最终目标需要重新只出现 a, b 。

怎么把 r 换回 a, b ? 刚才有 $a = qb + r$, 所以 $r = a - qb$ 。把它代入 $bu_1 + rv_1 = d$, 得到

$$d = bu_1 + (a - qb)v_1 = av_1 + b(u_1 - qv_1).$$

现在右边终于重新变成了 a, b 的整数线性组合。因此令 $u = v_1, v = u_1 - qv_1$, 就有 $au + bv = d$ 。这一步就是整段证明最核心的“回去”：欧几里得算法先把问题变小，归纳假设解决更小的问题，然后利用 $r = a - qb$ 把得到的结果重新换回原来的 a, b 。

到这里，裴蜀定理的充分性其实已经基本结束了。 我们已经证明了：只要 $d = \gcd(a, b)$, 就存在整数 u, v 使 $au + bv = d$ 。接下来只需利用题目的条件 $d \mid c$ 。因为 $c = dt$, 把前面的等式整体乘以 t , 就是

$$a(ut) + b(vt) = dt = c.$$

所以令 $x = ut, y = vt$, 原方程就有整数解。

整份二元证明其实可以压缩成这样一条逻辑链：先用欧几里得算法得到 $d = \gcd(a, b)$, 再利用 $d \mid c$ 把这个等式整体放大 c/d 倍。以后看到 $ax + by = c$, 不要一上来猜 x, y ; 先算 $d = \gcd(a, b)$, 检查 $d \mid c$, 如果成立, 就求出 $au + bv = d$, 最后乘上 c/d 。

最后，多元情形又是在做什么? 对于 $a_1x_1 + \cdots + a_nx_n = c$, 技巧其实非常直接：把前 $n-1$ 项先压缩成一个数。令 $g = \gcd(a_1, \dots, a_{n-1})$ 。根据已经证明的低元情形，存在整数系数使

$$a_1x_1 + \cdots + a_{n-1}x_{n-1} = g.$$

于是原来的 n 元方程可以先看成二元方程 $gt + a_nx_n = c$ 。

而 $\gcd(g, a_n) = \gcd(a_1, \dots, a_n) = d$, 所以这又回到了我们刚才已经解决的二元问题。只要 $d \mid c$, 就能得到 t, x_n 。最后把前面的表示式整体乘以 t , 得到

$$a_1(tx_1) + \cdots + a_{n-1}(tx_{n-1}) = gt,$$

再加上 a_nx_n , 原来的 n 元方程就被恢复出来了。

所以多元情形没有增加新的核心思想：它只是把前 $n-1$ 项压缩成一个 $\gcd$, 先解决一个二元问题，再把答案展开回来。

为什么：相邻斐波那契数最慢

直觉：商越大，减得越多。斐波那契数相邻两项的商永远是 1，等于每步只做了一次减法，所以最慢。这也解释了为什么竞赛里说 $\gcd$ 的最坏输入是相邻斐波那契数。

这里只用到一个初等事实：把 $au + bv$ 全体 (u, v 取遍整数) 排成一列，其中最小的正数就是 $\gcd(a, b)$ 。

令 $a = qb + r$ 。若递归已求得 $bu + rv = d$, 则

$$d = av + b(u - qv), \quad x = v, \quad y = u - qv.$$

因此 exgcd 只需在 $\gcd$ 的递归返回时维护两个系数。

对上例回代：

$$18 = 54 - 36 = 4 \cdot 54 - 198 = 4 \cdot 252 - 5 \cdot 198.$$

所以 $252x + 198y = 36$ 的一个解是 $(8, -10)$ 。“求 $ax + by = d$ ”与“求 $ax + by = c$ ”应分开：前者是算法接口，后者是检查 $d \mid c$ 后进行缩放。原稿把这两步中的比值写反了。

```
using i128 = __int128_t;
i128 exgcd(i128 a, i128 b, i128 &x, i128 &y) {
    // Contract: a,b >= 0, not both zero.
    if (b == 0) { x = 1; y = 0; return a; }
    i128 u, v;
    i128 d = exgcd(b, a % b, u, v);
    x = v;
    y = u - (a / b) * v;
    return d;
}
```

允许负系数时，先对 $|a|, |b|$ 求解，再恢复对应系数的符号。不要对最小负整数直接在原类型里取绝对值，应先转更宽类型。

1.4 从一个解得到全部解

当 $a, b > 0$ 且 $d \mid c$ ，已知特解 (x_0, y_0) ，则全部整数解为

$$x = x_0 + \frac{b}{d}t, \quad y = y_0 - \frac{a}{d}t, \quad t \in \mathbb{Z}.$$

小心：对 INT64_MIN 取相反数

因为 $\text{INT64_MIN} = -2^{63}$ 的绝对值不在 `long long` 里 ($2^{63} > 9.22 \times 10^{18}$)，直接取相反数是未定义行为。正确顺序：先转成 `__int128` 或无符号类型，再取绝对值。

证明。先看清要证的是**恰好**：既要说这个族里的每个 t 都给解，也要说任何解都在这个族里。

反向（族中皆解）。把 $x = x_0 + \frac{b}{d}t$ 、 $y = y_0 - \frac{a}{d}t$ 代入左边：

$$a\left(x_0 + \frac{b}{d}t\right) + b\left(y_0 - \frac{a}{d}t\right) = ax_0 + by_0 + \frac{ab}{d}t - \frac{ab}{d}t = c.$$

两个含 t 的项恰好抵消，所以任意整数 t 都给出解。

正向（任何解都在族中）。设 (x, y) 也是解，记 $\Delta x = x - x_0$ 、 $\Delta y = y - y_0$ 。两式相减得 $a\Delta x + b\Delta y = 0$ ，两边除以 d ：

$$\frac{a}{d}\Delta x = -\frac{b}{d}\Delta y.$$

因为 $d = \gcd(a, b)$ ， $\frac{a}{d}$ 与 $\frac{b}{d}$ 互质。上式表明 $\frac{b}{d} \mid \frac{a}{d}\Delta x$ ，由欧几里得引理 ($m \mid nk$ 且 $\gcd(m, n) = 1$ 时 $m \mid k$) 得 $\frac{b}{d} \mid \Delta x$ 。取整数 t 使 $\Delta x = \frac{b}{d}t$ ，代回 $\frac{a}{d}\Delta x = -\frac{b}{d}\Delta y$ 并约去非零因子 $\frac{b}{d}$ （这是整数等式，可以直接约）得 $\Delta y = -\frac{a}{d}t$ 。

两个方向合起来即**全部解恰为该单参数族**。这也解释了为什么参数只有一个：方程只有一个独立约束。□

若限制 $L_x \leq x \leq R_x$ 、 $L_y \leq y \leq R_y$ ，就将四个不等式转为 t 的上下界，取交集。解的计数问题变成一维整数区间计数。

P5656：二元一次不定方程 (exgcd)

给定正整数 a, b, c ，求 $ax + by = c$ 的正整数解个数以及 x, y 的最小值、最大值。若没有正整数解但有整数解，原题还要求分别报告能在整数解中取得的最小正 x 、最小正 y ；这两个值不必来自同一组解。

解题方向。先判定 $d \mid c$ ，再用通解把正整数约束转为参数区间。

讲解：先看这题到底在做什么

这题真正的工作分成两层：先用 `exgcd` 找到一组整数特解，再处理“正整数”这个额外限制。第一层只关心 $d \mid c$ 是否成立；第二层把所有解写成一个参数 t 的形式，于是正整数条件自然变成 t 的一个区间。讲解时不要把这两层混在一起，否则学生容易以为“求 `gcd`”本身就在解决正整数约束。

解析。 令 $A = a/d, B = b/d$ 。正整数要求给出

$$t \geq t_L = \left\lceil \frac{1 - x_0}{B} \right\rceil, \quad t \leq t_R = \left\lfloor \frac{y_0 - 1}{A} \right\rfloor.$$

把不等式变成一个参数的区间

以 $x = x_0 + bt \leq R_x$ 为例：移项得 $t \leq (R_x - x_0)/b$ ，取整为 $t \leq \lfloor (R_x - x_0)/b \rfloor$ ； $x \geq L_x$ 给 $t \geq \lceil (L_x - x_0)/b \rceil$ 。四个不等式给 t 一个区间 $[t_{\min}, t_{\max}]$ ，解的个数就是 $\max(0, t_{\max} - t_{\min} + 1)$ 。空区间表示**无正整数解**，但不表示**无整数解**——这两件事在 P5656 里是不同的输出分支。

若 $t_L \leq t_R$ ，则解数为 $t_R - t_L + 1$ ，并且

$$\begin{aligned} x_{\min} &= x_0 + Bt_L, & x_{\max} &= x_0 + Bt_R, \\ y_{\min} &= y_0 - At_R, & y_{\max} &= y_0 - At_L. \end{aligned}$$

若区间为空，只说明不能同时为正，并不说明无整数解。单独的最小正 x 是 $x_0 \bmod B$ 的正代表元（余数 0 改为 B ）； y 同理取模 A 。

完整算例。 $6x + 9y = 60$ 化为 $2x + 3y = 20$ ，取 $(x_0, y_0) = (10, 0)$ ，通解 $x = 10 + 3t, y = -2t$ 。正整数约束为 $-3 \leq t \leq -1$ ，三组解是

$$(1, 6), (4, 4), (7, 2).$$

词义：正代表元

正代表元 = 把同余类里的那个类写成 $1, \dots, B$ 之间的数。余数为 0 时不能留 0（题目要正整数），所以换成 B 。这类 0 **换成模数** 的小心，在取模计数题里非常常见。

故答案依次为解数 3, $x_{\min} = 1, y_{\min} = 2, x_{\max} = 7, y_{\max} = 6$ 。

负数整除不等于向下取整

C++ 的整数除法向零截断。对 $b > 0$ ，设 q_0 为 C++ 中 a/b 的结果、 r_0 为 $a \% b$ 的结果；注意 r_0 不一定是本书数学定义中的非负模余数。可以实现

$$\text{floorDiv}(a, b) = q_0 - [r_0 < 0], \quad \text{ceilDiv}(a, b) = q_0 + [r_0 > 0].$$

这样还避免了对最小负整数直接求 $-a$ 。测试 $a = -5, b = 3$ 时，应得到 `floor` 为 -2 、`ceil` 为 -1 。

复杂度与实现。 每组 $O(\log \min(a, b))$ ，额外空间为递归深度。 c/d 的缩放、区间端点的乘法应使用足够宽的整数。

题目 / 延伸资料：<https://www.luogu.com.cn/problem/P5656>

P8255: 数学游戏

给定 $x, z > 0$, 求满足 $z = xy \gcd(x, y)$ 的最小正整数 y , 无解则报告无解。原数据规模: $t \leq 5 \times 10^5, x \leq 10^9, z < 2^{63}$ 。

解题方向. 若 $x \nmid z$ 则无解。令 $w = z/x$, 考察 $\gcd(w, x^2)$ 是否为完全平方数。

讲解: 先看这题到底在做什么

先不要急着猜 y 。题目中的 $\gcd$ 可以拆成一个公共因子和两个互质部分; 设 $d = \gcd(x, y)$ 后, 原式会自然出现一个平方 d^2 。因此判定的核心变成: 从 z/x 里能不能找到一个平方因子, 而它恰好就是 $\gcd(z/x, x^2)$ 。

解析. 令 $d = \gcd(x, y)$, $x = da, y = db$, 其中 $\gcd(a, b) = 1$ 。则

$$w = \frac{z}{x} = bd^2, \quad x^2 = a^2 d^2,$$

所以

$$\gcd(w, x^2) = d^2 \gcd(b, a^2) = d^2.$$

注意这里必须是 x^2 , 不是原稿写的 a^2 。

反过来, 若 $g = \gcd(w, x^2) = d^2$ 是完全平方数, 则 $d \mid x, d^2 \mid w$ 。令 $a = x/d, b = w/d^2$, 有 $\gcd(a^2, b) = 1$, 故 $\gcd(a, b) = 1$ 。构造

$$y = \frac{w}{d}$$

即满足全部条件。这还证明了解若存在就是唯一的, “最小” 不需要额外搜索。

算例与反例. $x = 12, z = 864$ 时 $w = 72, g = \gcd(72, 144) = 72$ 不是平方, 无解。 $x = 12, z = 1296$ 时 $w = 108, g = 36, d = 6, y = 18$, 验证 $12 \cdot 18 \cdot 6 = 1296$ 。

复杂度与实现. 每组一次 $\gcd$ 与整数平方根。可用浮点平方根给出候选, 再用整数乘法向两侧校正; 不要直接判断浮点数是否为整数。用 128 位验证 $d^2 = g$ 和最终乘积。

题目 / 延伸资料: <https://www.luogu.com.cn/problem/P8255>

1.5 环上的固定步长

定理 1.3. 编号 $0, \dots, n-1$ 的环, 每步加 k 并取模 n 。设 $d = \gcd(n, k)$, 则形成 d 个循环, 每个循环长度为 n/d , 恰对应一个模 d 的同余类。

图景: 围成一圈走 k 步

把 n 个编号想象成围成一圈的小朋友, 每次往右走 k 步。形成 d 个循环就是分成 d 个小圈子, 圈子之间走不到。 $\gcd(n, k)$ 决定了圈子数, 这是互质才能走遍全部的几何版。

证明. 把问题写成带余除法语言: n 个点编号为 $0, 1, \dots, n-1$, 从 i 出发走 t 步后位于 $i + tk \pmod{n}$ 。

第一步 (周期长度)。回到出发点当且仅当 $n \mid tk$ 。设 $d = \gcd(n, k)$, 写 $n = dn', k = dk'$, 则 $\gcd(n', k') = 1$, 条件成为 $dn' \mid tdk'$, 即 $n' \mid tk'$; 再由欧几里得引理得 $n' \mid t$ 。所以使回到起点成立的正整数 t 恰为 n' 的倍数, 最小的是

$$t = n' = \frac{n}{\gcd(n, k)}.$$

这就是每个循环的长度。

第二步（谁和谁在同一个循环）。每步加 k ，而 $d \mid k$ ，故编号模 d 的余数始终不变：同一循环中的点全在同一个剩余类里。反过来，同一个剩余类 $\{j : 0 \leq j < n, j \equiv i \pmod{d}\}$ 恰有 n/d 个元素，而由第一步，从 i 出发的循环也恰有 n/d 个点（周期内不重复，否则会得到更小的周期）。前者包含后者且个数相同，所以两者相等：每个剩余类正好是一个循环。

结论：循环数为 $d = \gcd(n, k)$ ，每个长度 n/d 。 $k = 0$ 时 $d = n$ ，公式给出 n 个长度为 1 的循环，与**每点自成一环**一致，不必单独讨论。 $\square$

课堂小练习. $n = 12, k = 8$ 时 $d = 4$ ，四个环分别为

$$(0, 8, 4), (1, 9, 5), (2, 10, 6), (3, 11, 7).$$

请学生说明：这里的“子环”是置换的循环，不一定是抽象代数里的子环。后文 CF516E 要用这一分组，但“在同一环里”不意味着“距离起点相同”。

第 2 章

同余、逆元与指数降幂

章节导读

上一章，我们用 gcd 和 exgcd 处理整数之间的线性关系；但很多竞赛问题真正关心的不是整数等式，而是除以某个模数以后留下什么。这一章把这种“只看余数”的语言建立起来，并继续讨论逆元、快速幂以及如何把过大的指数压缩下来。

2.1 同余是整数等式的压缩

对 $m > 0$ ， $a \bmod m$ 是 $[0, m)$ 中与 a 同余的唯一整数；

$$a \equiv b \pmod{m} \iff m \mid a - b.$$

“最小非负代表元”这一句在管什么

同一堆余数里有无穷多个整数：-1、6、13 在模 7 下都算同一类。写 $a \bmod m$ 是 $[0, m)$ 中唯一的整数就是为了选定一个写法，此后所有比较都只比这个写法。C++ 的 % 对负数给出负余数，所以本书模板一律用 `norm(a, m)`。

$\{0, 1, \dots, m-1\}$ 把可能的余数排成一张表，称为模 m 的**完全剩余系**；表中与 m 互质的那些元素组成**简化剩余系**，它的个数记作 $\varphi(m)$ ，读 m 的**欧拉函数值**。加、减、乘可以在每一步后取模，交换律、结合律、分配律均保持成立。例如

$$(a + b)c \equiv ac + bc \pmod{m}.$$

剩余系 = 一张余数表

完全剩余系 = 每个余数出现一次的表；简化剩余系 = 表里只留能与模数互质的那些行。 $\varphi(m)$ 就是留下的行数。之后凡是在**剩余系**上乘一个数重排这张表，都在这里理解。

2.2 乘法逆元与正确约分

定理 2.1. $m \geq 2$ 时， a 的模 m 逆元存在当且仅当 $\gcd(a, m) = 1$ ；若存在，则在模 m 意义下唯一。

把除法翻译成解方程

所谓 a^{-1} ，就是找 x 使 $ax \equiv 1 \pmod{m}$ ，也就是找整数对 (x, y) 使 $ax + my = 1$ 。于是**逆元不存在立刻变成这个二元一次方程有没有整数解**，答案是有当且仅当 $\gcd(a, m) = 1$ （第 1 章裴蜀定理）。模 $m = 1$ 时同余式恒成立，本书统一约定结果为 0。

证明. 先把**逆元**翻译成方程。按同余的定义,

$$ax \equiv 1 \pmod{m} \iff m \mid ax - 1 \iff \exists y \in \mathbb{Z}, ax + my = 1$$

(取 $y = -(ax-1)/m$ 即得后一式的 y ; 反之把 my 移到右边也一样)。所以 a **有逆元**与 $ax+my=1$ **有整数解**是同一句话。

充分性: 若 $\gcd(a, m) = 1$, 由裴蜀定理这样的 (x, y) 存在, 于是 $ax \equiv 1 \pmod{m}$, 逆元存在。
必要性: 若有 x 使 $ax \equiv 1 \pmod{m}$, 设 $g = \gcd(a, m)$ 。则 $g \mid a$ 与 $g \mid m$ 给出 $g \mid (ax+my) = 1$, 故 $g = 1$ 。(用到的唯一事实是: $\gcd$ 整除任何整数线性组合。) 唯一性: 设 $ax \equiv ax' \equiv 1 \pmod{m}$, 两式相减得 $a(x-x') \equiv 0 \pmod{m}$, 即 $m \mid a(x-x')$ 。因 $\gcd(a, m) = 1$, 欧几里得引理给 $m \mid x-x'$, 即 $x \equiv x' \pmod{m}$ 。

所以**唯一**只是模 m 意义下的唯一; 一旦限定 $0 \leq x < m$, 代表元就真正唯一了。 $m=1$ 时同余式恒成立, 本书把结果统一约定为 0, 避免与**不存在**混淆。□

定理 2.2 (约分后模数的变化)。

$$ax \equiv bx \pmod{m} \iff a \equiv b \pmod{m/\gcd(x, m)}.$$

证明. 从定义出发逐步变形。 $ax \equiv bx \pmod{m}$ 意为 $m \mid x(a-b)$, 即存在整数 k 使

$$x(a-b) = km. \quad (*)$$

令 $d = \gcd(x, m)$, 写 $x = dx'$ 、 $m = dm'$, 则 $\gcd(x', m') = 1$ 。代入 $(*)$ 并约去 d (这是普通的整数等式, 可以约) 得

$$x'(a-b) = km',$$

即 $m' \mid x'(a-b)$ 。由欧几里得引理进一步得 $m' \mid a-b$, 也就是 $a \equiv b \pmod{m'} = a \equiv b \pmod{m/d}$ 。

反向: 若 $m \mid a-b$ 显然成立前一式; 具体地, $a-b = m's$ 时 $x(a-b) = dx'm's = m(dx's)$, 故 $m \mid x(a-b)$ 。

两个方向都用完了, 注意 $(*)$ 中的 k 在约分后仍是同一个整数: 不要求 $d \mid k$ 。这正是**同除一个公因子时, 被约的是模数, 不是那些商**的原因。□

把整数除法直接搬到同余式中会出错

$2 \cdot 1 \equiv 2 \cdot 4 \pmod{6}$, 但 $1 \not\equiv 4 \pmod{6}$ 。正确结论是 $1 \equiv 4 \pmod{3}$ 。一般若 $d \mid a, b, m$, 则 $a \equiv b \pmod{m}$ 等价于 $a/d \equiv b/d \pmod{m/d}$ 。对等式 $a = km + b$ 同除 d 得到的是 $a/d = k(m/d) + b/d$, 不需要 $d \mid k$ 。

算例. $12x \equiv 18 \pmod{30}$, 先除 $\gcd 6$ 得 $2x \equiv 3 \pmod{5}$, 于是 $x \equiv 4 \pmod{5}$ 。若在模 30 下列出全部解, 则是 4, 9, 14, 19, 24, 29; 新模数为 5 并不意味着在模 30 下只有一个解。

2.3 费马小定理与欧拉定理

定义 2.3. $\varphi(n) = \sum_{1 \leq i \leq n} [\gcd(i, n) = 1]$, 其中 $\varphi(1) = 1$ 。当 p 为素数时 $\varphi(p) = p - 1$ 。

定理 2.4 (欧拉定理与费马小定理). 若 $m \geq 2$ 且 $\gcd(a, m) = 1$, 则 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 。特别地, 素数 $p \nmid a$ 时 $a^{p-1} \equiv 1 \pmod{p}$ 。

证明. 取定 $R = \{r : 0 \leq r < m, \gcd(r, m) = 1\}$, 它有 $\varphi(m)$ 个元素. 分四步.

第一步 (乘完还在 R 里). 设 $\gcd(a, m) = 1$ 且 $\gcd(r, m) = 1$. 若有素数 $p \mid \gcd(ar, m)$, 则 $p \mid m$ 且 $p \mid ar$; $p \nmid a$ (否则 $p \mid \gcd(a, m)$), 故 $p \mid r$, 与 $\gcd(r, m) = 1$ 矛盾. 所以 $\gcd(ar, m) = 1$.

第二步 (乘 a 是一一对应). 若 $ar \equiv ar' \pmod{m}$, 则 $m \mid a(r - r')$, 由欧几里得引理得 $m \mid r - r'$; 而 $r, r' \in [0, m)$, 故 $r = r'$. 有限集合上的单射自动是双射 (抽屉原理: 像的元素数与原集合相同, 又不能漏, 只能恰好覆盖). 于是

$$\{ar \bmod m : r \in R\} = R$$

作为集合相等, 只是顺序被打乱.

第三步 (顺序无关, 可以整表相乘). 两个集合相同意味着把它们各自的元素全部乘起来结果相同:

$$\prod_{r \in R} (ar) \equiv \prod_{r \in R} r \pmod{m}.$$

左边重新整理成

$$\prod_{r \in R} (ar) = a^{|R|} \prod_{r \in R} r = a^{\varphi(m)} P, \quad P := \prod_{r \in R} r.$$

第四步 (约去 P). $m \mid P(a^{\varphi(m)} - 1)$. 因为 P 的每个因子都与 m 互质, P 也与 m 互质 (按素因子看: 任何 $p \mid m$ 都不整除 R 中元素, 故 $p \nmid P$). 用上节定理, 取 $x = P$, $a' = a^{\varphi(m)}$, $b' = 1$, 由 $\gcd(P, m) = 1$ 得

$$a^{\varphi(m)} \equiv 1 \pmod{m}.$$

费马小定理就是 $m = p$ 为素数、 $\varphi(p) = p - 1$ 的特例, 不需要另一套证明. $\square$

词义: 双射 (这里为何只验一半)

双射 = 一一对应: 既不碰撞 (单射) 也不遗漏 (满射). 有限集合上 **不碰撞** 自动推出 **不遗漏** (抽屉原理), 所以下面的证明只验证了不碰撞这一半.

小心: 约去必须互质

约去 是全文最容易被误用的一步: 只有当被约掉的数与模数互质时才合法. 反例: 模 8 下 $2 \cdot 1 \equiv 2 \cdot 5$ (都等于 2), 但 $1 \not\equiv 5 \pmod{8}$.

证明中的两个关卡

先问 “ $ar \bmod m$ 是否仍在简化剩余系?” 再问 “是否可能重复?” 只有两问都答对, 才能说乘法置换了整个集合. 最后约掉的不是一个普通整数, 而是一个可逆元素. 以 $m = 10, a = 3$ 演示: $\{1, 3, 7, 9\}$ 乘 3 得到 $\{3, 9, 1, 7\}$. 若取 $a = 2$, 第一步就失败.

2.4 扩展欧拉定理: 为什么要保留 “大于阈值” 的信息

设 $a \geq 1, m \geq 2, k \geq 0$, 则

$$a^k \equiv \begin{cases} a^{k \bmod \varphi(m)}, & \gcd(a, m) = 1, \\ a^k, & k < \varphi(m), \\ a^{k \bmod \varphi(m) + \varphi(m)}, & k \geq \varphi(m) \end{cases} \pmod{m}.$$

后两行也可不区分互质性使用。核心是：只有在原指数已经达到阈值时，才可以采用“取余再加 $\varphi(m)$ ”。

按素数幂拆开的证明。写 $m = \prod p_i^{e_i}$ 。对每个素数幂分别验证，再用 CRT 合并。若 $p_i \nmid a$ ，则 $\varphi(p_i^{e_i}) \mid \varphi(m)$ ，指数相差 $\varphi(m)$ 的倍数不改变余数。若 $p_i \mid a$ ，则 $k\nu_{p_i}(a) \geq e_i$ 时 $a^k \equiv 0 \pmod{p_i^{e_i}}$ 。而

$$\varphi(m) \geq \varphi(p_i^{e_i}) = p_i^{e_i-1}(p_i - 1) \geq e_i.$$

当 $k \geq \varphi(m)$ 时，原指数及替换指数都至少为 e_i ，所以两边均为 0。□

阈值这个词

阈值 = 从此以后行为改变的分界数。这里分界是 $\varphi(m)$ ：指数 k 小于它时不能改，大于等于它时改成 $k \bmod \varphi(m) + \varphi(m)$ 。之所以要加回 $\varphi(m)$ ，是为了保住 k 已经够大这一信息——非互质底数时，这个信息决定结果是不是 0。

反例应先于模板。 取 $a = 2, m = 8, k = 4$ 。只取 $k \bmod \varphi(8) = 0$ 会算成 1，正确值是 0；加回 4 后正确。反过来， $k = 1 < 4$ 时不能强行改为 5： $2^1 \equiv 2, 2^5 \equiv 0 \pmod{8}$ 。

指数塔的接口。 对巨大指数 E ，递归函数不能只返回 $E \bmod q$ ，还要返回真假值 $E \geq q$ 。两个指数 0 与 q 具有相同余数，但在非互质底数下作用不同。可以维护二元组

$$(\text{rem}, \text{large}) = (E \bmod q, [E \geq q]).$$

怎么做：接口口诀余数 + 一位真假

记住这个接口口诀：**余数 + 一位真假**。返回 $(E \bmod q, [E \geq q])$ ，上层用 $\text{rem} + q \cdot \text{large}$ 还原一个行为等价的指数。判断 $E \geq q$ 用截断计算 $\min(E, q)$ ：一旦超过就停下，绝不构造大数。

计算上层幂时使用 $\text{rem} + q \cdot \text{large}$ 。判定大小使用截断计算 $\min(E, q)$ ，而不是通过取模后的数值猜测。

截断幂与截断塔。 定义 $\text{powcap}(a, e, C) = \min(a^e, C)$ ，每次乘法超过 C 就截断。对 $a \geq 2$ 的塔 $T_0 = 1, T_h = a^{T_{h-1}}$ ，若 $C > 1$ ，先求最小 r 使 $a^r \geq C$ ，再计算 $\min(T_{h-1}, r)$ 。若结果达到 r ，直接返回 C ，否则算出小幂。阈值迅速下降，避免构造天文数字。 $a = 1$ 与 $C = 1$ 单独处理。

2.5 五类逆元问题

统一练习：如何选择求逆元的方法？

- (a) 素数模数，单次求 a^{-1} ， $m \leq 10^{18}$ 。
- (b) 一般模数，单次求 a^{-1} ，或报告不存在。
- (c) 素数模数，求 $1, 2, \dots, n$ 的逆元， $n < m$ 。
- (d) 一般模数，离线求很多与 m 互质的数的逆元。
- (e) 固定素数模数 $m \leq 10^9$ ，强制在线处理多达 10^8 次询问。

在线/离线

在线：来一问答一问，不许偷看后面的问题。离线：先把所有问题读完，可以排序、分组、分块。第 (d)(e) 两种做法只在离线/强制在线意义下有区别——题目一旦说**强制在线**，分块预处

理就不许用了。

解题方向. 单次用快速幂或 `exgcd`；连续区间用线性递推；批量互质数用前缀积。固定模数的大量在线询问可以进一步预处理。

2.5.1 前四类：应掌握的基础方法

(a) 由 $a^{m-1} \equiv 1$ 得 $a^{-1} \equiv a^{m-2} \pmod{m}$ ，快速幂 $O(\log m)$ 。(b) 求 $ax + my = 1$ ；gcd 不为 1 则不存在，反之将 x 规范到 $[0, m)$ 。(c) 令 $m = qi + r$ ，对 $2 \leq i < m$ ，有 $r \neq 0$ 且

$$\text{inv}[1] = 1, \quad \text{inv}[i] \equiv -q \text{inv}[r] \pmod{m}.$$

为什么要分五类

差别只在三个开关上：模数是素数吗（决定能不能用费马小定理）、是单次还是批量（决定要不要前缀积）、是否强制在线（决定能不能分块预处理）。做逆元题时先把这三个开关写下来，方法基本就唯一了。

前提： m 是素数且 $a \neq 0$

这一步用了 m 是素数且 $a \neq 0$ ：此时 $a^{m-1} \equiv 1$ （费马小定理），于是 $a \cdot a^{m-2} \equiv 1$ 。 m 是合数时 $a^{\varphi(m)} \equiv 1$ 才成立，且要先确认互质——直接套 $m-2$ 会算错。

$r < i$ 保证可以从小到大递推，时间 $O(n)$ 。(d) 令 $P_0 = 1, P_i = P_{i-1}a_i \pmod{m}$ 。求一次 P_n^{-1} ，逆序处理

$$a_i^{-1} = P_{i-1}P_i^{-1}, \quad P_{i-1}^{-1} = a_iP_i^{-1}.$$

线性递推逆元公式是怎么来的

写 $m = qi + r$ 、 $0 < r < i$ ，则 $qi + r \equiv 0 \pmod{m}$ ，即 $r \equiv -qi$ 。两边同乘 $i^{-1}r^{-1}$ （两者都可逆）得 $i^{-1} \equiv -qr^{-1} \pmod{m}$ 。因为 $r < i$ ， r^{-1} 已经算过，所以能一遍扫过去。注意它要求模数是素数（保证每个 $r < i < m$ 可逆），合数时会断，见下面的易错框。

时间 $O(n + \log m)$ ，空间 $O(n)$ 。所有 a_i 可逆才保证总乘积可逆；并不要求模数是素数。

小例. 模 17 求 3, 5, 7 的逆元。前缀积为 1, 3, 15, 3； $3^{-1} = 6$ ，逆推得到 $7^{-1} = 5$ 、 $5^{-1} = 7$ 、 $3^{-1} = 6$ 。

易错点

线性公式在合数模数下不能照抄，即使当前 i 可逆也未必有 r 可逆。例如模 8、 $i = 3$ 时余数 $r = 2$ 不可逆，但 $3^{-1} = 3$ 确实存在。

2.5.2 在线方案一：每步至少减半

设 $m = ua + v$ ， $0 < v < a$ 。除了 $a^{-1} \equiv -uv^{-1}$ ，还有

$$(u+1)a \equiv a-v \pmod{m}, \quad a^{-1} \equiv (u+1)(a-v)^{-1} \pmod{m}.$$

前缀积批量求逆元的想法

只花一次昂贵的求逆，其余全是乘法和查表：先算前缀积 $P_i = a_1 \cdots a_i$ ，求 P_n^{-1} 一次；然后 $a_i^{-1} = P_{i-1} \cdot P_i^{-1}$ 从后往前推，因为 $P_i^{-1} = a_i^{-1}P_{i-1}^{-1}$ 。这就是用一次 $O(\log m)$ 换 n 次 $O(1)$ 。

这个恒等式从哪来

$m = ua + v$ 意味着 $ua \equiv -v$, 两边再加 a 得 $(u+1)a \equiv a - v$ 。为什么规模减半: 新的递归对象是 $\min(v, a-v)$, 它不超过 $a/2$ 。所谓**每步至少减半**就是把**取余数**与**取补数**中较小者递下去, 至多 $\log_2 m$ 步进入预处理表。

在 v 与 $a-v$ 中选较小者, 递归规模至少减半。预处理 $[1, L]$ 的逆元后, 单次询问最多进行 $O(1 + \log(m/L))$ 次递归。取 $L = 2 \times 10^7, m \leq 10^9$, 规模在至多 6 次减半后落入表内; 若 $L \geq m$, 只预处理到 $m-1$ 。这不是一般意义上的渐近 $O(1)$, 而是数据范围内步数很小。

2.5.3 在线方案二: $O(m^{2/3})$ 预处理与常数次查询运算

这是选修内容。令 $B = U = \lceil m^{1/3} \rceil, L = \lceil m/U \rceil$ 。将 a 写为 $qB + r, 0 \leq r < B$ 。对每个块 q 寻找

$$1 \leq u_q \leq U, \quad c_q = qBu_q - k_qm, \quad |c_q| \leq L.$$

为什么存在? 把 $0, qB, \dots, UqB$ 的模 m 余数放入 U 个长度不超过 L 的区间, 有两者落在同一区间, 相减即可。于是

$$au_q \equiv c_q + ru_q = t, \quad |t| \leq L + BU = O(m^{2/3}),$$

抽屉原理一句话

$U+1$ 个数放进 U 个盒子, 必有两个同盒。这里把 $0, qB, \dots, UqB$ 这 $U+1$ 个余数放进 $[-L, L]$ 划出的 U 段, 取同段两者相减, 就造出一个**小的** $qBu - km$ 。看到**存在性证明里出现** $+1$, 基本都是抽屉原理。

并且 $t \not\equiv 0 \pmod{m}$ 。预处理这一小范围的正负逆元, 即得

$$a^{-1} \equiv u_q t^{-1} \pmod{m}.$$

查询只需除法、查表、乘法。对很小的 m 直接预处理全部非零余数。

如何在正确复杂度内找到全部块的系数? 枚举 $u = 1, \dots, U$, 再枚举 $k = 0, \dots, u+1$ 。满足条件的 q 组成整数区间

$$\left\lceil \frac{km - L}{uB} \right\rceil \leq q \leq \left\lfloor \frac{km + L}{uB} \right\rfloor,$$

口径: 这里的常数次数指哪几步

这一步的查表范围是 $[-L, L]$, 所以正负都要有逆元: $(-t)^{-1} \equiv -(t^{-1})$, 不必另算。”常数次数”指的是: 一次整数除法、两次乘法、两次查表——不是 $O(\log m)$ 。

与合法块编号范围相交。每个 u 的区间个数为 $O(u)$, 包含的候选总数为 $O(u + L/B) = O(U)$; 总预处理 $O(U^2) = O(m^{2/3})$ 。已找到系数的块不必覆盖。

算例. 模 101, $B = U = 5, L = 21$ 。对 $a = 73 = 14 \cdot 5 + 3$, 选 $u = 3, c = 14 \cdot 5 \cdot 3 - 2 \cdot 101 = 8$, 于是 $t = 8 + 3 \cdot 3 = 17$ 。查表 $17^{-1} = 6$, 得到 $73^{-1} = 3 \cdot 6 = 18$ 。

复杂度与实现. 这是字长模型中的 $O(m^{2/3})$ 时间、空间预处理与 $O(1)$ 查询; 必须把内存规模纳入选择, 而不是一看到“大量询问”就使用最复杂方案。

第 3 章

中国剩余定理与同余约束的合并

章节导读

上一章我们已经能够在单个模数下处理同余、逆元和幂；当问题同时给出多个模数的限制时，新的困难就变成“这些限制能不能合在一起”。这一章利用中国剩余定理，把多个局部条件重新拼成一个整体，并处理模数不互质时需要额外检查的情况。

3.1 互质模数：构造局部为一、其余为零的系数

考虑 $x \equiv a_i \pmod{m_i}$ ，其中 $m_i \geq 2$ 两两互质。令

$$M = \prod_i m_i, \quad M_i = M/m_i, \quad t_i \equiv M_i^{-1} \pmod{m_i}.$$

为什么：把内存规模一起算进选择

$O(m^{2/3})$ 在 $m = 10^9$ 时约 10^6 ，表很小；但 $m = 10^{18}$ 时约 10^{12} ，直接内存爆炸。所以更高级的做法可能反而不可用，选方法要同时看时间与内存。

定理 3.1 (中国剩余定理). 上述方程组在模 M 意义下有唯一解：

$$x \equiv \sum_i a_i M_i t_i \pmod{M}.$$

证明.

思考：CRT 的构造到底在做什么？

先不要急着记公式。我们真正需要的是：对每一个模数 m_i ，造出一个“开关” E_i ，使它模 m_i 等于 1，而模其他 m_j 都等于 0。这样，把 E_i 乘上目标余数 a_i ，最后把所有这些项相加，就能一次得到所有同余条件。

以三个模数为例，想让一个数模 m_1 为 1、模 m_2, m_3 为 0，最自然的起点就是 $M_1 = m_2 m_3$ 。因为它本身已经在模 m_2, m_3 下为 0；又因为三个模数两两互质， M_1 与 m_1 互质，所以只需要乘上一个系数 t_1 ，把 M_1 在模 m_1 下调成 1。这就是 $t_1 \equiv M_1^{-1} \pmod{m_1}$ 。

于是 $M_1 t_1$ 就是我们要的第一个“开关”。其他模数同理。最后令 $x = \sum_i a_i M_i t_i$ ，每一项在自己对应的模数下留下 a_i ，在其他模数下全部消失，所以所有同余条件同时成立。下面的正式推导只是把这个想法逐个模数写清楚。

存在性与唯一性分开证，并把逆元为什么存在这一步补上。

先补一句： $M_i = \prod_{j \neq i} m_j$ 与 m_i 互质（模数两两互质， M_i 的每个因子都与 m_i 互质），所以 $t_i \equiv M_i^{-1} \pmod{m_i}$ 确实存在，依据是第 2 章的可逆 $\iff$ 互质。

存在性。把 $x = \sum_j a_j M_j t_j$ 代入第 i 个同余式。当 $j \neq i$ 时 M_j 含因子 m_i , 故 $M_j t_j \equiv 0 \pmod{m_i}$, 这些项全部消失; 当 $j = i$ 时 $M_i t_i \equiv 1 \pmod{m_i}$, 该项留下 a_i 。于是 $x \equiv a_i \pmod{m_i}$, 对每个 i 都成立。

唯一性。设 x, y 都满足方程组, 则每个 i 都有 $m_i \mid x - y$ 。两两互质时反复使用 $p \mid n, q \mid n, \gcd(p, q) = 1 \Rightarrow pq \mid n$ (把 $n = pu$ 代入 $q \mid pu$, 由互质得 $q \mid u$, 故 $pq \mid n$), 得到 $M = \prod_i m_i \mid x - y$, 即 $x \equiv y \pmod{M}$ 。

把结论说准确: 解集是模 M 的一个剩余类, 所以唯一只在 $[0, M)$ 内成立, 不是只有一个整数。□

板书例. 解

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}, \quad x \equiv 2 \pmod{7}.$$

$M = 105$, $M_i = 35, 21, 15$, 相应逆元为 $2, 1, 1$, 所以

$$x \equiv 2 \cdot 35 \cdot 2 + 3 \cdot 21 + 2 \cdot 15 = 233 \equiv 23 \pmod{105}.$$

讲解时应让学生先验算 70 模三个模数的余数为 $(1, 0, 0)$, 再展示整式。

易错点

M_i 不是 m_i 。 t_i 的逆元模数必须写明为 m_i 。所谓“唯一解”是唯一的同余类, 不是唯一整数。

3.2 不互质模数：合并两个等差数列

现在考虑

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}.$$

写 $x = a + mt$, 得到 $mt \equiv b - a \pmod{n}$ 。设 $d = \gcd(m, n)$:

- $d \nmid b - a$ 时无解;
- 否则解 $(m/d)t \equiv (b - a)/d \pmod{n/d}$, 得到 t_0 ;
- 合并结果为 $x \equiv a + mt_0 \pmod{\text{lcm}(m, n)}$ 。

重复合并即可得到扩展 CRT。模数为 1 的约束不限制整数, 可跳过。

为什么没有漏解? 关于 t 的所有解相差 n/d , 所以对应 x 相差 mn/d 。反过来, 每个满足两式的 x 都有 $t = (x - a)/m$, 因此每一个合法 x 都被表示到了。

可解与不可解的对照.

$$x \equiv 2 \pmod{6}, \quad x \equiv 5 \pmod{9} \implies 6t \equiv 3 \pmod{9} \implies t \equiv 2 \pmod{3},$$

词义：扩展 CRT 的扩展

扩展只是指模数不必两两互质。做法：每次只合并两个式子, 把结果当成一个新式子再和下一个合并。合并一次可能无解 (相容性 $d \mid b - a$), 也可能把模数变成 lcm。顺序不影响最终解集, 但影响中间数值大小。

故 $x \equiv 14 \pmod{18}$ 。把第二个余数改为 4, 则 $3 \nmid (4 - 2)$, 无解。

```
// Contracts: m,n > 0; all intermediate products fit i128.
i128 norm(i128 a, i128 m) { a %= m; return a < 0 ? a + m : a; }
bool merge_crt(i128 a, i128 m, i128 b, i128 n,
               i128 &r, i128 &mod) {
    a = norm(a,m); b = norm(b,n);
    i128 u,v, d = exgcd(m,n,u,v);
    if ((b-a) % d != 0) return false;
    i128 q = n/d;
    i128 t = norm(((b-a)/d) * u, q);
    mod = m*q;
    r = norm(a+m*t, mod);
    return true;
}
```

若输入规模使乘积超出 128 位，此段代码不再适用，应改用大整数或安全模乘。exCRT 并不会自动解决整数溢出。

附带一个下界。 若最终解为 $x \equiv r \pmod{M}$, $0 \leq r < M$, 还要求 $x \geq L$, 则最小解是

$$r + M \max\left(0, \left\lceil \frac{L - r}{M} \right\rceil\right).$$

词义：大整数（要换掉的不止乘法）

大整数 = 用数组或 `boost::multiprecision::cpp_int` 存任意长度。竞赛里能不用就不用（慢一个量级）；实在躲不掉时，要连带把 gcd、取模、快速幂都换成高精度版本，不能只换乘法。

下界公式是怎么推出来的

解集是 $r, r + M, r + 2M, \dots$ 。要挑第一个不小于 L 的，就是求最小 $t \geq 0$ 使 $r + tM \geq L$, 即 $t \geq (L - r)/M$, 故 $t = \max(0, \lceil (L - r)/M \rceil)$ 。写成代码时注意 $L \leq r$ 时不能让「负数」把 t 压成负数，所以外面套 $\max(0, \cdot)$ 。

同余约束解决的是周期，大小约束决定选周期中的哪一项。

P4774：屠龙勇士

依次攻击每条龙。面对生命值 a_i , 从当前剑的多重集合中选攻击力不高于 a_i 的最大剑；若不存在，则选最小剑。设选出的攻击力为 c_i 。攻击固定的 x 次后，龙可以不断恢复 p_i 生命值；要求恰好恢复到 0。击败后所用剑被消耗，并得到奖励剑。求可通关的最小 x 。

解题方向。 有序多重集合确定每次武器；每条龙给出 $c_i x \equiv a_i \pmod{p_i}$ 和 $c_i x \geq a_i$, 不能只保留同余式。

讲解：先看这题到底在做什么

先把“选什么剑”和“攻击多少次”拆开。武器选择只由当前生命值和武器集合决定，因此可以先独立模拟；确定每条龙使用的攻击力 c_i 后，剩下的问题才是同时满足同余式和攻击次数下界，再用 exCRT 合并。

解析。 分四步完成。

第一步：武器序列独立于攻击次数。 选剑只与当前集合、 a_i 和奖励剑有关。用 `multiset` 的 `upper_bound(a[i])` 寻找第一个大于 a_i 的元素，若它不是 `begin`, 则退一步；否则选择 `begin`。删

除时只删除这一个迭代器，不能删除所有相等攻击力的剑。

第二步：准确翻译死亡条件。 攻击后生命值 $a_i - c_i x$ ，只能增加 p_i ，不能减少。因此存在整数 $k_i \geq 0$ 满足

$$a_i - c_i x + k_i p_i = 0$$

两个容器术语

`upper_bound(x)` 返回第一个严格大于 x 的元素的位置（迭代器），不是位置编号；所以不高于 a_i 的最大值 = 先找第一个大于的，再往回退一步，退不了（已是 `begin`）就取 `begin`。`erase`（迭代器）只删这一个，`erase(值)` 会把所有相等的都删掉——本题恰好在有重复剑时出错。

等价于

$$c_i x \equiv a_i \pmod{p_i}, \quad x \geq \lceil a_i / c_i \rceil.$$

若没有第二条，可能输出一个攻击伤害还不够的“同余解”。

第三步：把带系数同余化为标准式。 令 $d_i = \gcd(c_i, p_i)$ 。若 $d_i \nmid a_i$ ，立即无解；否则除去 d_i 后求逆元，得到

$$x \equiv \frac{a_i}{d_i} \left(\frac{c_i}{d_i} \right)^{-1} \pmod{p_i / d_i}.$$

将这些式子逐个 exCRT 合并，并记录 $L = \max_i \lceil a_i / c_i \rceil$ 。

第四步：选择不小于下界的第一项。 用前面的等差数列公式即可。原题保证所有 p_i 的最小公倍数不超过 10^{12} ，但中间乘法仍可能溢出 64 位。

把官方说明中的 59 推出来。 初始剑 $\{1, 9, 1000\}$ ，生命值依次为 3, 5, 7，恢复量为 4, 6, 10，前两次奖励剑为 7, 3。选出的剑是 1, 7, 3，得到

$$x \equiv 3 \pmod{4}, \quad 7x \equiv 5 \pmod{6}, \quad 3x \equiv 7 \pmod{10}.$$

即 $x \equiv 3 \pmod{4}, x \equiv 5 \pmod{6}, x \equiv 9 \pmod{10}$ ，先合并前两式得 $x \equiv 11 \pmod{12}$ ，再合并第三式得 $x \equiv 59 \pmod{60}$ 。下界为 3，故答案 59。

复杂度与实现。 选剑 $O(n \log m)$ ，合并 $O(n \log V)$ ；集合大小始终为 m 。当 $p_i = 1$ 时同余式没有约束，但伤害下界仍不能丢。

学生容易漏掉的三件事

选剑条件是“不高于”，不是“严格小于”；恢复只能向上，不是任意取模操作； $a_i \leq p_i$ 并非所有测试点都具有的统一条件，应使用一般下界处理。

题目 / 延伸资料：<https://www.luogu.com.cn/problem/P4774>

第 4 章

乘法阶、原根与离散对数

章节导读

上一章解决的是多个同余约束如何合并；这一章换一个方向，研究一个数不断乘下去会形成怎样的周期。乘法阶告诉我们周期有多长，原根刻画整个单位群，而离散对数则反过来要求我们从结果找回指数。

4.1 周期与乘法阶

定义 4.1. 若 $m \geq 2$ 且 $\gcd(a, m) = 1$ ，满足 $a^r \equiv 1 \pmod{m}$ 的最小正整数 r 称为阶，记作 $\text{ord}_m(a)$ 。

定理 4.2. $\text{ord}_m(a)$ 存在当且仅当 $\gcd(a, m) = 1$ 。设其为 r ，则

$$a^k \equiv 1 \pmod{m} \iff r \mid k, \quad r \mid \varphi(m).$$

若 $a^x \equiv b \pmod{m}$ 有解 x_0 ，则其非负整数解为 $x \equiv x_0 \pmod{r}$ 中非负的那些整数。

证明. 逐条展开。

阶的存在性等价于互质。若某个 $r \geq 1$ 使 $a^r \equiv 1 \pmod{m}$ ，则 $a \cdot a^{r-1} \equiv 1$ ，说明 a 有逆元 a^{r-1} ，由上节可逆 $\iff$ 互质得 $\gcd(a, m) = 1$ 。反之若 $\gcd(a, m) = 1$ ，欧拉定理给 $a^{\varphi(m)} \equiv 1 \pmod{m}$ ，所以集合 $\{t \geq 1 : a^t \equiv 1 \pmod{m}\}$ 非空；正整数集的非空子集有最小元（最小数原理），记作 $r = \text{ord}_m(a)$ 。

整除刻画。设 $a^k \equiv 1$ 。用带余除法写 $k = qr + s$ ， $0 \leq s < r$ 。则

$$1 \equiv a^k = (a^r)^q a^s \equiv 1^q a^s = a^s \pmod{m}.$$

若 $s > 0$ ， s 就是比 r 更小、又能使幂为一的正指数，与 r 的最小性矛盾。故 $s = 0$ ，即 $r \mid k$ 。反向也写出来：若 $r \mid k$ ， $k = rt$ ，则 $a^k = (a^r)^t \equiv 1$ 。

$r \mid \varphi(m)$ 。取 $k = \varphi(m)$ （此时 $a^k \equiv 1$ 由欧拉定理保证，前提 $\gcd(a, m) = 1$ 已由阶的存在给出），套上一条即得。

离散对数的解结构。设 $a^{x_0} \equiv b \pmod{m}$ 有解 x_0 。若 $a^x \equiv b$ ，两边同乘 a^{x_0} 的逆元 a^{r-1} （注意 $x - x_0$ 可能为负，此时用 $a^{x-x_0} \equiv a^{x-x_0+r}$ 化归非负指数）得 $a^{x-x_0} \equiv 1$ ，由整除刻画得 $r \mid x - x_0$ 。反向： $x \equiv x_0 \pmod{r}$ 时 $x = x_0 + rt$ ， $a^x \equiv a^{x_0}(a^r)^t \equiv b$ 。所以全部非负整数解正是同余类 $x_0 \bmod r$ 中的非负元——这也说明最小非负解唯一存在。 $\square$

为什么：求阶前先查 $\gcd(a, m) = 1$

$a^r \equiv 1$ 与 a **可逆**是一件事的两面：能回到 1 就自动有逆元 a^{r-1} ；可逆才有欧拉定理给出周期。所以任何**求阶**的代码，第一行都该是检查 $\gcd(a, m) = 1$ ，检查不过就报**不存在**，而不是返回一个错误的最小值。

具体周期. 模 7 的 2^x 依次为 $1, 2, 4, 1, \dots$ ，阶为 3；不是每个非零余数都能表示成 2 的幂，例如 3 不行。模 8 的 2^x 是 $1, 2, 4, 0, 0, \dots$ ，只有最终周期，不存在乘法阶。

求阶. 已知 $\varphi(m)$ 的质因数分解，令 $r = \varphi(m)$ 。对每个素因子 q ，只要 $q \mid r$ 且 $a^{r/q} \equiv 1$ ，就令 $r \leftarrow r/q$ ，直至不能继续。循环结束时 r 正好是阶：若仍大于真阶，商中必有某个素因子可以再除。预处理含求 $\varphi(m)$ 及其分解；每次求阶通常用 $O(\log^2 m)$ 次字长运算作粗上界。不能把分解的成本藏在一次 $O(\log^2 m)$ 查询里。

4.2 有限域上的根数与原根

定理 4.3 (多项式根数界，亦称此处的拉格朗日定理). 在 $\mathbb{F}_p$ 上，首项系数非零的 d 次多项式至多有 d 个不同根。

证明. 对次数 d 归纳，并把**可逆**用在哪一行标出来。

$d = 0$ ：非零常数多项式没有根，至多 0 个根，成立。设 $d \geq 1$ 且结论对 $d - 1$ 次成立。若 f 无根，结论已成立；否则取一根 r ，即 $f(r) = 0$ 。

第一步（提取因式）。用带余除法（首项系数非零保证可除）： $f(X) = (X - r)g(X) + c$ ， $\deg g = d - 1$ 、 c 为常数。代入 $X = r$ 得 $0 = f(r) = 0 \cdot g(r) + c$ ，故 $c = 0$ 。

第二步（其余根落到 g 上）。设 $s \neq r$ 也是 f 的根，则

$$0 = f(s) = (s - r)g(s).$$

$s - r \neq 0$ ，而在 $\mathbb{F}_p$ 中每个非零元素都有逆元；两边乘 $(s - r)^{-1}$ 得 $g(s) = 0$ 。

第三步（收尾）。于是 f **除 r 以外的根**全是 g 的根，由归纳假设至多 $d - 1$ 个，故 f 至多 $1 + (d - 1) = d$ 个。

注意第二步是唯一用到**域**的地方。在 $\mathbb{Z}/8\mathbb{Z}$ 中 $2 \neq 0$ 却不可逆，且 $2 \cdot 4 = 0$ ，同样的推理立刻断掉——这就是易错框里四个根的来历。□

图景：0 是黑洞

0 是黑洞：一旦乘出 0 就再也回不到 1。所以模 8 的 2 只有**最终周期**（尾巴进 0），没有乘法阶——阶要求 $a^r \equiv 1$ 。这正是求阶之前必须查互质的原因。

提醒：两个拉格朗日定理不是一回事

这条定理的名字容易撞车：群论里还有一条**子群的阶整除群的阶**也叫拉格朗日定理，与这里无关。本书尽量用**多项式根数界**这种描述性名字，避免靠人名记忆。

怎么做：从 $\varphi(m)$ 往下除出最小周期

把做法翻译成成人话就是：先猜一个周期 $r = \varphi(m)$ ，再逐个素因子试着往下除，**除了之后还是一、且还能继续除**，就除掉。最后剩下的一定是最小正周期。前提是你已经知道 $\varphi(m)$ 和它的分解——这两项成本要单独计入预处理。

易错点

这不是任意合数模数下都成立的性质。例如 $X^2 - 1 \equiv 0 \pmod{8}$ 有 1, 3, 5, 7 四个根。“次数为二”只有在相应域或整环条件下才能限制根数。本节名称也不要与群论中“子群阶整除群阶”的拉格朗日定理混淆。

定义 4.4. 满足 $\text{ord}_m(g) = \varphi(m)$ 的元素 g 称为模 m 的原根。

原根一句话

原根就是一个数不断自乘，能走遍所有与 m 互质的余数。模 7 下 3 的幂给出 3, 2, 6, 4, 5, 1——六个互质余数一个不落，所以 3 是原根。这样的数存在与否完全由 m 的形状决定 ($2, 4, p^\alpha, 2p^\alpha$)。

定理 4.5. 对 $m \geq 2$ ，原根存在当且仅当

$$m = 2, 4, p^\alpha, 2p^\alpha \quad (p \text{ 为奇素数}, \alpha \geq 1).$$

若 $\gcd(g, m) = 1$ ，则 g 是原根当且仅当对每个不同素因子 $q \mid \varphi(m)$ 都有 $g^{\varphi(m)/q} \not\equiv 1 \pmod{m}$ 。(注意枚举的对象是 $\varphi(m)$ 的素因子，不是它的约数； $\gcd(g, m) = 1$ 这条前提也不能省，否则逆元不存在，测试无从谈起。)

原根存在时，简化剩余系为 $1, g, \dots, g^{\varphi(m)-1}$ ，原根个数为 $\varphi(\varphi(m))$ ；且

$$\text{ord}_m(g^t) = \frac{\varphi(m)}{\gcd(t, \varphi(m))}.$$

判定式为什么只查素因子？ 若真阶 $r < \varphi(m)$ ，则整数 $\varphi(m)/r > 1$ 有某个素因子 q ，于是 $r \mid \varphi(m)/q$ ，记 $\varphi(m)/q = rt$ ，得 $g^{\varphi(m)/q} = (g^r)^t \equiv 1$ ，会被检测到。反向：若存在素因子 q 使 $g^{\varphi(m)/q} \equiv 1$ ，则阶整除 $\varphi(m)/q < \varphi(m)$ ， g 不是原根。原根存在性分类的完整证明需要初等数论之外的工具（对 2^α 与奇素数幂分别构造，再由中国剩余定理合并），篇幅超过一节；基础课把它当作已知定理使用，判定式、原根计数与阶公式则应当课堂证明。

模 7 的例子. $\varphi(7) = 6$ ，只需测指数 3, 2。 $3^3 \equiv 6$ ， $3^2 \equiv 2$ ，故 3 为原根。与 6 互质的指数为 1, 5，所以原根是 $3^1 = 3$ 与 $3^5 = 5$ 。

随机找原根的边界. 若原根存在，已知 $\varphi(t)/t = \Omega(1/\log \log t)$ ，可得到在 $[1, m)$ 随机抽样找到原根的期望次数为 $O((\log \log m)^2)$ 。每次还要做若干快速幂，不能把抽样次数当成总时间；小模数单独处理。

4.3 BSGS：用平方根空间换时间

离散对数

求 $a^x \equiv b \pmod{m}$ 的最小非负整数解，或报告无解。

当 $\gcd(a, m) = 1$ 时，若有解，则存在 $0 \leq x < \varphi(m) \leq m$ 的解。这由欧拉定理得到，不需要误用“合数模数下的费马小定理”。设 $B = \lceil \sqrt{m} \rceil$ ，写 $x = iB + j$ ， $0 \leq j < B$ ，则

$$a^j \equiv b(a^{-B})^i \pmod{m}.$$

口径：抽几次不等于跑多久

期望抽几次和总共跑多久是两件事：每次抽样后要做 $\omega(\varphi(m))$ 次快速幂，还要有 $\varphi(m)$ 的分解。小模数直接枚举反而快；把渐近式当成实现建议是常见的坑。

预处理 baby steps a^j ，再枚举 giant steps 的右边并查表，时间、空间均为 $O(\sqrt{m})$ （哈希期望）。

保证最小解的细节。 哈希表的同一个余数只保留最小 j 。按照 $i = 0, 1, \dots$ 的顺序查询，那么各块的指数区间互不重叠且递增，首次命中的 $iB + j$ 即为最小解。即使枚举区间略超过 m ，首次命中仍然不会错；无解需完整检查足够覆盖 $[0, m)$ 的块数。若 b 与 m 不互质而 a 与 m 互质，则立即无解，因为任何 a^x 都是单位。

手算例。 解 $2^x \equiv 5 \pmod{13}$ 。取 $B = 4$ ，baby 表为

$$1 \mapsto 0, \quad 2 \mapsto 1, \quad 4 \mapsto 2, \quad 8 \mapsto 3.$$

BSGS 的核心一次代换

把未知的指数按块拆开： $x = iB + j$, $0 \leq j < B$ 。等式 $a^{iB+j} \equiv b$ 两边同乘 a^{-iB} ，变成**只有 j 在左、只有 i 在右**： $a^j \equiv b(a^{-B})^i$ 。左边全部预先算好放进表，右边一个个枚举去查表。这就是用空间换时间的标准形状， $\sqrt{m}$ 来自 $B \cdot B \approx m$ 。

为什么：表里只留最小的 j

查表法求**最小解**时，插入策略和枚举顺序共同决定正确性：表里保留最小的 j ，外层 i 从小到大，于是第一次命中就是字典序最小的 (i, j) ，也就是最小 x 。若表被后来的大 j 覆盖，答案可能偏大。

$2^4 \equiv 3$ ，逆元为 9；giant 依次为 5, 6, 2，在 $i = 2$ 时命中 $j = 1$ ，得 $x = 9$ 。

多次询问如何分块？ 若底数 a 与模数 m 都固定，可以预处理 a^{iB} 的表，每问枚举 ba^j ，将 $x = iB - j$ 还原。预处理 $O(m/B)$ ， Q 次查询总计 $O(QB)$ ，平衡时 $B \approx \sqrt{m/Q}$ ；若该值小于 1，取 $B = 1$ 。总时间应写成 $O(\sqrt{Qm} + Q)$ 的相应界，并说明取整与内存限制。如果只有模数固定、底数改变，原表一般不能复用。求最小解时，对同一 j 找到最小可行 giant 指数，再对所有 j 的候选选取最小值，不能随意用哈希表覆盖重复值。

4.4 exBSGS：剥离非互质部分

先处理 $m = 1$ 、 $x = 0$ 、 $a = 0$ 等边界。令 $d = \gcd(a, m) > 1$ 。对 $x \geq 1$ ，若 $d \nmid b$ 则无解；否则

$$\frac{a}{d}a^{x-1} \equiv \frac{b}{d} \pmod{m/d}.$$

exBSGS 在剥什么

a 与 m 不互质时 a 不可逆， a^{-1} 无从谈起。技巧是**约掉一个公因子再降一层**： $a^x \equiv b \pmod{m}$ 且 $d = \gcd(a, m) \mid b$ 时，可化为 $\frac{a}{d}a^{x-1} \equiv \frac{b}{d} \pmod{m/d}$ 。每剥一层模数变小、指数减一；剥到互质为止，再交给 BSGS。

平衡两个成本的做法

总时间形如 $T(B) = m/B + QB$ (预处理一项、查询一项)。两项一个随 B 减、一个随 B 增, 令相等 $m/B = QB$ 得 $B = \sqrt{m/Q}$, 此时 $T = O(\sqrt{Qm})$ 。也可以用求导验证; 再检查取整、内存上限、以及 $B < 1$ 时要夹回 1。

由于 $\gcd(a/d, m/d) = 1$, 该系数可逆, 可以递归到更小的模数。递归前每层都要检查零指数解。最终底数与新模数互质时使用 BSGS。

更适合实现的形式是维护已剥离次数 c 与系数 k , 循环不变量为

$$ka^{x-c} \equiv b \pmod{m}, \quad \gcd(k, m) = 1.$$

什么叫循环不变量

每次循环开头都必须成立的一句话, 例如剥掉 c 层后, 余下的式子是 $ka^{x-c} \equiv b$, 且 k 可逆。写代码前先写不变量, 循环结束时的不变量加退出条件就自动是答案; 调试时也照这句话打表检查。

初始 $c = 0, k = 1$ 。每次先判断 $b = k$, 若成立则 $x = c$ 就是最小解; 再令 $d = \gcd(a, m)$ 。若 $d = 1$, 解 $a^{x-c} \equiv bk^{-1} \pmod{m}$; 若 $d > 1$ 且 $d \nmid b$ 则无解。否则

$$m \leftarrow m/d, \quad b \leftarrow b/d, \quad k \leftarrow k(a/d) \pmod{m}, \quad c \leftarrow c + 1.$$

这里 a/d 使用本轮的 d ; 底数 a 保持不变。 m 至少减半, 剥离最多 $O(\log m)$ 轮。

算例. $4^x \equiv 8 \pmod{14}$ 。先检查 $x = 0$ 不满足。剥离 $d = 2$ 后 $m = 7, b = 4, k = 2, c = 1$, 于是需解 $4^{x-1} \equiv 2 \pmod{7}$, 最小 $x - 1 = 2$, 答案 3。验证 $4^3 = 64 \equiv 8 \pmod{14}$ 。

易错点

离散对数的零指数是合法候选; $a \equiv 0$ 时, $x = 0$ 给 1, $x \geq 1$ 给 0。非互质情形的解集可能带有限前缀, 不能一开始就写成 $x_0 + k \operatorname{ord}_m(a)$ 。

P5605: 小 A 与两位神仙

$m = p^q$, p 为奇素数。多次询问单位 x, y , 判断是否存在 $a \geq 0$ 使 $x^a \equiv y \pmod{m}$ 。 $m \leq 10^{18}$, 询问数不超过 2×10^4 , 允许使用快速因数分解工具。

快速因数分解工具指什么

指 Miller-Rabin 判素 + Pollard-Rho 分解 (附录模板里的 `is_prime` 与 `factor`)。它们在 64 位范围内实践上很快, 但 Rho 的速度依赖随机函数的表现, 没有严格多项式界; 而分解只是把 m 拆成素数幂的工具, 拆完之后每一步都是确定的整数运算。

解题方向. 单位群是循环群, 条件恰为 $\operatorname{ord}_m(y) \mid \operatorname{ord}_m(x)$; 无需真正求离散对数。

讲解: 先看这题到底在做什么

题目的关键转化是把单位群里的元素写成某个原根的幂。这样 $x^a = y$ 就从乘法问题变成指数上的线性同余: 若 $x = g^u, y = g^v$, 就只需研究 $au \equiv v \pmod{\varphi(m)}$ 。因此真正要检查的是这个线性同余是否可解, 而不是去真的求出离散对数。

解析. 取原根 g , 令 $x = g^u, y = g^v$, 则需解

$$au \equiv v \pmod{\varphi(m)}.$$

这条提示里的每个词

单位 = 与 m 互质的余数 (可逆); **单位群是循环群** = 存在一个 g , 使得每个单位都写成 g^t ; **阶** $\text{ord}_m(x)$ = 使 $x^r \equiv 1$ 的最小正指数。把 $x = g^s$ 、 $y = g^t$ 代入 $x^a \equiv y$, 得 $sa \equiv t$, 这类方程有解的条件正是 $\text{ord}_m(y) \mid \text{ord}_m(x)$ ——所以根本不必真的求出离散对数。

裴蜀定理给出 $\gcd(u, \varphi(m)) \mid v$ 。等价地, 它整除 $\gcd(v, \varphi(m))$; 使用阶公式便得到所述整除关系。也可以从循环群子群唯一性理解: $\langle x \rangle$ 是阶为 $\text{ord}_m(x)$ 的唯一子群。

为什么条件不能推广到任意模数? 模 8, 3 与 5 的阶都为 2, 但 3 的幂只能取 1, 3, 不能得到 5。单位群不是循环群, “阶整除” 仅必要而不充分。

复杂度与实现. 分解 m 、计算并分解 $\varphi(m)$ 后, 每问两次求阶。因数分解的 $O(m^{1/4})$ 应理解为后文 Pollard-Rho 的常用启发式估计, 不是无条件最坏界。

题目 / 延伸资料: <https://www.luogu.com.cn/problem/P5605>

P6736: 白泽教育——高阶运算取模

定义 $a \uparrow^1 b = a^b$, 且 $a \uparrow^n 0 = 1$; $n > 1, b > 0$ 时

$$a \uparrow^n b = a \uparrow^{n-1} (a \uparrow^n (b-1)).$$

给定 $1 \leq a \leq 10^9, 1 \leq n \leq 3, 0 \leq b < m \leq 10^9$, 求最小非负 x 使 $a \uparrow^n x \equiv b \pmod{m}$ 。

解题方向. $n = 1$ 用 exBSGS; $n = 2$ 利用欧拉链计算有限高度的塔并证明稳定; $n = 3$ 利用高度增长极快而不是构造大整数。先特判 $x = 0, m = 1, a = 1$ 。

讲解: 先看这题到底在做什么

幂塔题最容易犯的错是直接对指数取模。正确顺序是先判断底数、模数和指数是否处于“可以降低幂”的条件, 再沿欧拉链逐层处理; 当底数和模数不互质时, 还要保留“指数是否已经足够大”这一位信息。

解析. 给出一条以正确性易于讲清为优先的算法路线。令 $T_0 = 1, T_h = a^{T_{h-1}}$, 它是 $a \uparrow^2 h$ 。例如底数 2 的前几项为

$$1, 2, 4, 16, 65536, 2^{65536}, \dots$$

为什么这三个特判不能省

$x = 0$: 任何 $a^0 = 1$, 若 $b \equiv 1$ 答案就是 0, 而按**指数为正**的循环会漏掉; $m = 1$: 所有余数都是 0, 答案恒为 0, 但 $\varphi(1)$ 、逆元之类的记号在此都退化; $a = 1$: 1 的幂永远是 1, 要么立刻命中要么永远不命中, 递归降幂会打转。边界情形不特判, 正确的算法也会给出错误答案, 这是竞赛里最常见的失分方式之一。

有限枚举的依据: 稳定上界. 设欧拉链 $m_0 = m, m_{i+1} = \varphi(m_i)$, 到 $m_L = 1$ 。取

$$R = \lceil \log_2 m \rceil, \quad H = R + L + 1.$$

对 $a \geq 2, T_h \geq 2^h$ 。当 $h \geq R$, 在链上所有模数处, “是否达到阈值” 的标记都为真。底层模 1 的余数恒定, 再向上逐层归纳: 每向上一层多需要一层塔高; 因此 $T_h \bmod m$ 在 $h \geq H$ 时稳定。于是得到一个安全的 $O(\log m)$ 高度上界, 不追求最紧的常数。

$n = 2$ 时从 $h = 0$ 到 H 枚举, 使用扩展欧拉递归与截断塔计算 $T_h \bmod m$, 第一次命中就是答案; 都不命中则永远不命中。**不能仅凭两次顶层余数相同就提前宣布稳定**, 应使用上述证明或完

整状态的稳定判据。

$n = 3$ 为什么只需极少候选？记 $P_x = a \uparrow^3 x$ ，则 $P_0 = 1, P_{x+1} = T_{P_x}$ 。当 $a \geq 3$,

$$P_1 = a, \quad P_2 = T_a \geq 3^{27} > H.$$

故 $P_3 = T_{P_2}$ 已使用稳定的塔高，此后结果相同，只需检查 $x = 0, 1, 2, 3$ 。当 $a = 2$, $P_1 = 2, P_2 = 4, P_3 = T_4 = 65536 > H$ ，故还需检查 $x = 4$ 。在本题 $m \leq 10^9$ 时，上述 H 小于 100，比较非常安全。

计算 P_2 或 P_3 的“高度”时，只保留截断到 H 的结果；高度达到 H 即可使用稳定值。原稿中 $3 \uparrow^3 3 = 3^{3^{27}}$ 的等式不正确：前者是高度 3^{27} 的 3 的幂塔，而右边仅是高度 4 的塔。

复杂度与实现. 每问预处理欧拉链，缓存重复模数的 φ ，再做有限次快速幂。上述教学实现应按“枚举高度数 $\times$ 欧拉链深度 $\times$ 快速幂成本”计时；不是一句“答案很小”就能省略全部复杂度。大量测试下可复用链、预筛试除质数，并改为自底向上批量计算。 $n = 1$ 的瓶颈仍可能是平方根级 BSGS。

题目 / 延伸资料：<https://www.luogu.com.cn/problem/P6736>

第 5 章

升幂引理：把巨大幂变成指数的加法

章节导读

上一章我们研究了模意义下的周期和指数；现在继续追问一个更细的问题：两个非常大的幂相减，究竟含有多少个素因子 p ？ p 进赋值把“乘法中的指数”变成可加的量，升幂引理则把这个观念直接用于幂的差值。

5.1 p -进赋值与使用前的检查

对非零整数 x ， $\nu_p(x) = \max\{e \geq 0 : p^e \mid x\}$ 。基本性质为

$$\nu_p(xy) = \nu_p(x) + \nu_p(y), \quad \nu_p(x + y) \geq \min(\nu_p(x), \nu_p(y)),$$

幂塔与高度

$a \uparrow\uparrow h$ 表示 h 层的塔 $a^{a^{\cdot^{\cdot^a}}}$ ，**高度**就是层数。塔的计算顺序是从最上层往下（右结合），所以**高度 5 的 2 的塔**是 $2^{2^{2^{2^2}}} = 2^{65536}$ ，而不是 $((2^2)^2)^2$ 。

箭号记号的两条约定

$a \uparrow^1 b = a^b$ ， $a \uparrow^k 1 = a$ ， $a \uparrow^k b = a \uparrow^{k-1} (a \uparrow^k b)$ 的塔式展开： $3 \uparrow^2 3 = 3^{3^3}$ 高度为 3；再升一级 $\uparrow^3$ 就是**高度的高度**。凡是这种记号，先代最小的数值（高度 1, 2, 3）确认自己算的和定义一致，再往下看结论。

若 $\nu_p(x) \neq \nu_p(y)$ ，后一式取等。下文统一设 p 为素数， $p \nmid xy$ ， n 为正整数。涉及差为 0 的情况时可用 $\nu_p(0) = +\infty$ ，实际算题通常先排除零。

直观解释。 ν_p 是“能连续除以 p 多少次”。乘法变加法；加法却可能发生抵消，所以不能写 $\nu_p(x + y) = \nu_p(x) + \nu_p(y)$ 。例如 $\nu_2(6) = 1, \nu_2(10) = 1$ ，但 $\nu_2(16) = 4$ 。

LTE 的使用检查表

1. 要计算的是幂的差还是幂的和？指数是否为正？
2. 素数是否整除 $x - y$ ，或整除 $x + y$ ？
3. $p \nmid x$ 、 $p \nmid y$ 是否都满足？若不满足，先提出共同素数幂。
4. $p = 2$ 还是奇素数？指数是奇数还是偶数？

只背“加一个 $\nu_p(n)$ ”容易误用到二进制的偶指数情况。

5.2 与素数互质的指数

定理 5.1. 若 $p \mid x - y$ 且 $p \nmid n$, 则

$$\nu_p(x^n - y^n) = \nu_p(x - y).$$

若 $p \mid x + y$ 、 n 为奇数且 $p \nmid n$, 则

$$\nu_p(x^n + y^n) = \nu_p(x + y).$$

口径：为什么规定 $\nu_p(0) = +\infty$

规定 $\nu_p(0) = +\infty$ 是为了让 $p^e \mid x$ 对任意 e 成立这句话有地方安放, 从而 $\nu_p(xy) = \nu_p(x) + \nu_p(y)$ 在 $y = 0$ 时不用另写情形。做题时代入具体数字前先检查 $x = y$, 否则会得到 $\nu_p(0)$ 等于某个有限值的错误等式。

前提： $p \nmid xy$ 为什么不能少

$p \nmid xy$ 是三条结论共同的前提, 它等价于 $p \nmid x$ 且 $p \nmid y$ 。少了它, x, y 都能被 p 整除时左边右边同时虚增, 公式就不成立了: 例如 $p = 3, x = 3, y = 6, n = 2$, 左边 $\nu_3(9 - 36) = 2$, 右边 $\nu_3(-3) + \nu_3(2) = 1$ 。

证明. 先写出恒等式

$$x^n - y^n = (x - y)S, \quad S = \sum_{i=0}^{n-1} x^{n-1-i} y^i.$$

它只是多项式恒等式 (把右边展开、逐项相消即得), 对任何整数都成立, 不依赖整除条件。

由赋值的可加性 (第 0 章 0.6 节, 或本章开头那条 $\nu_p(xy) = \nu_p(x) + \nu_p(y)$):

$$\nu_p(x^n - y^n) = \nu_p(x - y) + \nu_p(S).$$

所以只剩一件事要证: $\nu_p(S) = 0$, 即 $p \nmid S$ 。

模 p 计算 S 。条件 $p \mid x - y$ 给 $x \equiv y \pmod{p}$, 于是 S 的每一项都满足 $x^{n-1-i} y^i \equiv x^{n-1-i} x^i = x^{n-1} \pmod{p}$, 共 n 项, 故

$$S \equiv nx^{n-1} \pmod{p}.$$

题设有 $p \nmid n$; 又 $p \nmid x$ (若 $p \mid x$, 由 $p \mid x - y$ 得 $p \mid y$, 与 $p \nmid xy$ 矛盾)。两者相乘仍不被 p 整除, 故 $p \nmid S$, 即 $\nu_p(S) = 0$ 。结论得证。

和的情形: n 为奇数时 $x^n + y^n = x^n - (-y)^n$, 且 $p \mid x - (-y) = x + y$ 。把已证的差情形用于 $-y$ 即得 $\nu_p(x^n + y^n) = \nu_p(x + y) + \nu_p(n)$ 。 n 必须为奇数, 否则 $(-y)^n = y^n$, 第一步就不成立——这就是分类的原因。□

5.3 奇素数的完整形式

定理 5.2 (LTE, 奇素数). 若 p 为奇素数且 $p \mid x - y$, 则

$$\nu_p(x^n - y^n) = \nu_p(x - y) + \nu_p(n).$$

若 $p \mid x + y$ 且 n 为奇数, 则

$$\nu_p(x^n + y^n) = \nu_p(x + y) + \nu_p(n).$$

先把指数乘一次 p ，再反复提升. 令 $s = \nu_p(x - y) \geq 1$ ，写 $x = y + h$ ， $\nu_p(h) = s$. 二项式展开

$$x^p - y^p = py^{p-1}h + \sum_{j=2}^{p-1} \binom{p}{j} y^{p-j} h^j + h^p.$$

首项的赋值恰为 $s + 1$. 中间各项赋值至少为 $1 + 2s \geq s + 2$ ；末项赋值为 $ps \geq s + 2$ ，这里用了 $p \geq 3$. 因此最低赋值只有首项取得，不会抵消，故

$$\nu_p(x^p - y^p) = s + 1.$$

写 $n = p^e u$ ， $p \nmid u$. 先用上一节处理 u ，再连续 e 次把指数乘 p ，每次增加一，得到 $s + e$. 幂的和仍以 $y \mapsto -y$ 转为差. $\square$

教学重点. 原稿的二重求和证明可以成立，但容易让学生看不见真正原因。用“唯一最低赋值项”说明为什么不抵消，之后只需归纳，结构更清楚。

5.4 素数二的特殊形式

定理 5.3 (LTE, $p = 2$). 设 x, y 为奇数. 若 n 为奇数，则

$$\nu_2(x^n - y^n) = \nu_2(x - y), \quad \nu_2(x^n + y^n) = \nu_2(x + y).$$

若 n 为偶数，则

$$\nu_2(x^n - y^n) = \nu_2(x - y) + \nu_2(x + y) + \nu_2(n) - 1.$$

特别地，若 $4 \mid x - y$ ，则对所有正整数 n 有

$$\nu_2(x^n - y^n) = \nu_2(x - y) + \nu_2(n).$$

方法：唯一最低赋值项

唯一最低赋值项是一种很常用的论证格式：把一项写成若干项之和，证明其中某一项所含 p 比其它任何一项都少，那么这个少无法被抵消，整个和的赋值就等于它。用这个格式，二重求和可以省掉。

证明. 奇指数已由第一种形式处理. 偶数写为 $n = 2^e u$ ， u 为奇数、 $e \geq 1$. 先去掉奇数部分：

$$\nu_2(x^n - y^n) = \nu_2(x^{2^e} - y^{2^e}).$$

继续因式分解

$$x^{2^e} - y^{2^e} = (x - y)(x + y) \prod_{i=1}^{e-1} (x^{2^i} + y^{2^i}).$$

奇数的平方模 8 为 1，因此每个 $i \geq 1$ 对应的括号模 4 为 2，恰贡献一个因子 2，一共 $e - 1$ 个。相加得到公式. 若 $4 \mid x - y$ ，则 $x + y \equiv 2 \pmod{4}$ ，所以 $\nu_2(x + y) = 1$ ，偶数公式简化；奇数时 $\nu_2(n) = 0$ ，也一致. $\square$

另一个边界. 若 x, y 为奇数且 n 偶，则 $x^n + y^n \equiv 2 \pmod{4}$ ，因此赋值为 1. 不要把“幂的和的 LTE”用于偶数指数。

5.5 分层例题与解析

例 1：原稿中的两个练习

求 $\nu_7(10^7 - 3^7)$ 与 $\nu_2(7^{10} - 3^{10})$ 。

解题方向. 第一个是奇素数差的形式；第二个必须用 $p = 2$ 的偶指数形式。

讲解：先看这题到底在做什么

第一题直接套奇素数版本的 LTE，但必须先把条件逐条检查清楚；第二题是 $p = 2$ 的特殊情形，不能照搬奇素数公式。这里最值得讲的是“先识别定理，再核对条件”，而不是只看最后的数字。

解析. 因为 $7 \mid 10 - 3$ 且 $7 \nmid 30$,

$$\nu_7(10^7 - 3^7) = \nu_7(7) + \nu_7(7) = 2.$$

怎么用一节里的例题

每题先只读题面，写下用**哪条定理、条件是否满足**，再看解析的第一句是否与你猜的一致。对不上时不要往下读，先找出是自己漏了条件还是解析换了方法。

第二题中 7, 3 均为奇数，指数 10 为偶数，故

$$\nu_2(7^{10} - 3^{10}) = \nu_2(4) + \nu_2(10) + \nu_2(10) - 1 = 2 + 1 + 1 - 1 = 3.$$

注意两个 $\nu_2(10)$ 来自不同对象：一个是底数之和，一个是指数。

例 2：先分组再使用 LTE

求 $\nu_3(2^{2025} + 1)$ 与 $\nu_5(3^{100} - 1)$ 。

解题方向. 第一题为奇指数幂的和；第二题先把 3^{100} 看成 81^{25} 。

讲解：先看这题到底在做什么

第一题先看成幂的和；第二题则先把指数拆成一个让底数之幂与 1 发生整除关系的部分，再使用 LTE。学生容易记住公式却忘记“先找到合适的 $a^d - 1$ ”，所以这里应强调结构而不是算术。

解析. $2025 = 3^4 \cdot 25$ ，所以

$$\nu_3(2^{2025} + 1) = \nu_3(2 + 1) + \nu_3(2025) = 1 + 4 = 5.$$

$5 \nmid 3 - 1$ ，不能直接对底数 3, 1 使用差的 LTE。但 $3^4 - 1 = 80$ 可被 5 整除，因此

$$\nu_5(3^{100} - 1) = \nu_5(81^{25} - 1) = \nu_5(80) + \nu_5(25) = 1 + 2 = 3.$$

此例展示“先用阶找到能被 p 整除的基本差，再用 LTE 提升”。

例 3：最小指数与素数幂模数下的阶

求最小正整数 n 使 $5^4 \mid 3^n - 1$ 。

解题方向. 先由模 5 的阶得到 $4 \mid n$ ，再比较赋值。

讲解：先看这题到底在做什么

先解决“ n 至少应该是什么倍数”这个粗条件，再处理“还需要多少个 p 因子”这个精条件。前者来自模 p 的阶，后者来自 LTE；两个条件合起来，最小指数就不再需要试枚举。

解析. 模 5 下 3 的阶为 4, 设 $n = 4t$ 。LTE 给出

$$\nu_5(3^{4t} - 1) = \nu_5(3^4 - 1) + \nu_5(t) = 1 + \nu_5(t).$$

要至少为 4, 最小 $t = 5^3 = 125$, 故答案 $n = 500$ 。一般对奇素数 p 、 $d = \text{ord}_p(a)$, 若 $s = \nu_p(a^d - 1)$, 则

$$\text{ord}_{p^k}(a) = dp^{\max(0, k-s)}.$$

证明正是上面的两步。它把阶和升幂引理连接起来； $p = 2$ 需另行分类。

第 6 章

阶乘、素数幂与组合数取模

章节导读

上一章的赋值方法能告诉我们一个数里含多少个 p ；这一章把它应用到阶乘。因为阶乘本身会迅速变得巨大，我们要把“ p 的指数”和“去掉 p 后留下的单位部分”拆开，最终得到素数幂模数下组合数的计算方法。

6.1 Wilson 定理及单位群乘积

定理 6.1 (Wilson 定理). 整数 $n > 1$ 为素数，当且仅当 $(n-1)! \equiv -1 \pmod{n}$ 。

证明. 分素数时成立与合数时不成立两个方向。

正向 (p 为素数)。 $p = 2$: $(2-1)! = 1 \equiv -1 \pmod{2}$ (此时 1 与 -1 是同一余数，需单独看一眼)。设 p 为奇素数，考虑集合 $U = \{1, 2, \dots, p-1\}$ ，其中每个元素都可逆。把 U 按 x 与 x^{-1} 相同吗分成两类：若 $x \neq x^{-1}$ ，把 x 与 x^{-1} 配成一对，每对的乘积 $\equiv 1$ 。这样的配对不会重叠，因为 $(x^{-1})^{-1} = x$ 。若 $x = x^{-1}$ ，即 $x^2 \equiv 1 \pmod{p}$ ，则 $p \mid (x-1)(x+1)$ 。 p 是素数，故 $p \mid x-1$ 或 $p \mid x+1$ (欧几里得引理)，得 $x \equiv 1$ 或 $x \equiv -1$ 。反过来这两者确实自逆。于是

$$(p-1)! \equiv \left(\prod_{\text{对}} 1 \right) \cdot 1 \cdot (-1) \equiv -1 \pmod{p}.$$

这里素数用在两处：每个非零元可逆（才能配对），以及 $p \mid AB$ 蕴含 $p \mid A$ 或 $p \mid B$ （才能说自逆元只有 ± 1 ）。

反向 (n 为合数则 $(n-1)! \not\equiv -1$)。取 $1 < d < n$ 且 $d \mid n$ 。情形一： $d < n-1$ ，即 $n > 2d$ 。此时 d 出现在 $1 \cdot 2 \cdots (n-1)$ 的因子中，故 $d \mid (n-1)!$ 。若同时 $(n-1)! \equiv -1 \pmod{n}$ ，则 $n \mid (n-1)! + 1$ ，从而 $d \mid (n-1)! + 1$ ；两式相减得 $d \mid 1$ ，矛盾。情形二： $n = 2d$ 且 d 是大于 2 的素数（即 n 是 2 倍奇素数）。此时取 d 本身， $d \geq 3$ 与 $d \leq n-2$ 使 d 与 $2d$... 更简单的做法是： n 有平方因子时 ($n = d^2$ 或 $d^2 \mid n$) d 与 n/d 是两个不同且都 $\leq n-1$ 的因子，都出现在 $(n-1)!$ 中，故 $d^2 \mid (n-1)!$ ，于是 $n \mid (n-1)!$ ；若又有 $(n-1)! \equiv -1 \pmod{n}$ ，则 $n \mid 1$ ，矛盾。剩下 $n = 2p$ (p 为奇素数) 时直接验算： $(n-1)! \equiv 0 \pmod{p}$ ，而 $-1 \not\equiv 0 \pmod{p}$ ，故不成立。 $n = 4$ 时 $(4-1)! = 6 \equiv 2 \not\equiv -1 \pmod{4}$ 。

综合： $(n-1)! \equiv -1 \pmod{n} \iff n$ 为素数。 $\square$

一节里其实在算什么乘积

所有把一堆余数乘起来的问题 ($(p-1)!$ 、单位群全体、约数之积) 都用同一套：把元素按互为逆元配对，配对项乘积为 1，剩下自逆元单独数。所以真正要算的总是解 $x^2 \equiv 1$ 有几个解。

单位群乘积在算什么

模 m 下所有可逆余数的乘积。做法永远是同一套：把元素按与自己互为逆元配对，乘积为 1，剩下的只有自逆元（解 $x^2 \equiv 1$ ）。所以真正要算的是自逆元有几个，这由 m 的形状决定；模素数时恰有 ± 1 。

对一般 $m > 1$ ，所有单位的乘积模 m 为 -1 或 1 。取 -1 的模数类型是 $2, 4, p^\alpha, 2p^\alpha$ (p 奇素数)，其他取 1 ；模 2 下 $1 = -1$ ，数值上重合。特别地，令 $Q = p^\alpha$ ，则

$$C_Q = \prod_{\substack{1 \leq i \leq Q \\ p \nmid i}} i \equiv \begin{cases} 1, & p = 2, \alpha \geq 3, \\ -1, & \text{其他} \end{cases} \pmod{Q}.$$

推广形式的理由。 在任意有限阿贝尔群中，将非自逆元素成对消去，剩下的是满足 $u^2 = 1$ 的元素。它们构成若干个二阶循环群的直积：若只有一个非单位自逆元素，乘积是该元素；若有至少两个独立的二阶方向，每个坐标取 1 的次数为偶数，乘积是单位元。模奇素数幂的 $x^2 = 1$ 只有 ± 1 ： p 不可能同时整除 $x - 1, x + 1$ ，故一个因子吸收全部 p^α 。模 2^α ($\alpha \geq 3$) 则有四个自逆元素。再用 CRT 组合，就得到分类。

易错点

C_Q 是删去 p 的倍数后的单位乘积，而 $(Q!)_p$ 是对每个整数除尽自身包含的 p 因子后再相乘，两者不相同。例如 $p = 2, Q = 8$ ： $C_8 = 1 \cdot 3 \cdot 5 \cdot 7 = 105$ ，而 $(8!)_2 = 315$ 。

6.2 Legendre 公式：阶乘中的素数指数

定理 6.2. 对素数 p 与非负整数 n ，

$$\nu_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor = \frac{n - s_p(n)}{p - 1},$$

其中 $s_p(n)$ 是 p 进制各位数字之和。 $\nu_p(0!) = 0$ 。

一句话版

$\nu_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor$ ：先数 p 的倍数，再数 p^2 的倍数（它们多贡献一个），一路数到超过 n 为止。记不住公式时，用 $10!$ 里有几个 2 现场推一遍只要十秒。

证明。 先把按倍数数写成等式，再化成数位和。

$n!$ 中恰好含有多少个因子 p ？逐个看 $1, 2, \dots, n$ 各贡献 ν_p ，所以

$$\nu_p(n!) = \sum_{m=1}^n \nu_p(m).$$

把每个 m 的贡献按它是 p 的几次倍数的倍数拆开： $\nu_p(m) = \sum_{j \geq 1} [p^j \mid m]$ (m 被 p^j 整除就有第 j 个 p)。代进去并交换求和：

$$\nu_p(n!) = \sum_{j \geq 1} \sum_{m=1}^n [p^j \mid m] = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor,$$

因为 1 到 n 中 p^j 的倍数恰有 $\left\lfloor \frac{n}{p^j} \right\rfloor$ 个（即 $p^j, 2p^j, \dots$ ）。 $j > \log_p n$ 时项为 0 ，和是有限的。

现在写 $n = \sum_i a_i p^i$, $0 \leq a_i < p$ (这就是 p 进制表示)。由 $\lfloor n/p^j \rfloor = \sum_{i \geq j} a_i p^{i-j}$ 得

$$\sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor = \sum_{j \geq 1} \sum_{i \geq j} a_i p^{i-j} = \sum_i a_i \sum_{j=1}^i p^{i-j} = \sum_i a_i (1 + p + \cdots + p^{i-1}) = \sum_i a_i \frac{p^i - 1}{p - 1}.$$

而 $n = \sum_i a_i p^i$ 、数位和 $s_p(n) = \sum_i a_i$, 于是

$$\nu_p(n!) = \frac{n - s_p(n)}{p - 1}.$$

最后这个式子就是 Kummer 定理要用到的形态: 分子 n 减数位和正是这些位上的借位总量。□

算例. $\nu_2(10!) = 5 + 2 + 1 = 8$; 二进制 $10 = (1010)_2$, 数位和为 2, 也得到 $10 - 2 = 8$ 。 $\nu_5(100!) = 20 + 4 = 24$, 所以 $100!$ 的十进制末尾零数为 $\min(\nu_2, \nu_5) = 24$ 。实现时反复令 $n \leftarrow \lfloor n/p \rfloor$ 并累加, 而不是不断计算 p^j ; 后者可能溢出。

6.3 Kummer 定理: 进位就是损失的数位和

定理 6.3. 对 $0 \leq k \leq n$,

$$\nu_p \binom{n}{k} = \frac{s_p(k) + s_p(n - k) - s_p(n)}{p - 1}.$$

它等于在 p 进制下把 k 与 $n - k$ 相加时的进位次数, 也等于从 n 减 k 的借位次数 (逐位、包含连续借位)。

它的用法是判零而不是算值

实战里几乎不用它算出 ν_p 的具体值, 只用它的一句话推论: $p \nmid \binom{n}{k} \iff$ 竖式 $k + (n - k)$ 在 p 进制下不进位。Lucas 定理就是这个推论的另一面。

证明. Legendre 公式 (上一条) 说 $\nu_p(m!) = \frac{m - s_p(m)}{p - 1}$, 所以

$$\nu_p \binom{n}{k} = \nu_p(n!) - \nu_p(k!) - \nu_p((n - k)!) = \frac{s_p(k) + s_p(n - k) - s_p(n)}{p - 1}.$$

只剩一句话要说清: 为什么 $s_p(k) + s_p(n - k) - s_p(n) = (p - 1) \times$ (进位次数)。

看竖式加法 $k + (n - k) = n$ 。设从低到高共有 c 次进位。若不进位, 每一位的数码相加就等于该位结果; 每发生一次第 i 位向第 $i + 1$ 位进 1, 第 i 位上留下的和就少 p , 而第 $i + 1$ 位多 1, 因此两个加数的数位之和比结果的数位之和多 $p - 1$ 。多个进位互不抵消: 可以把进位链按发生顺序逐个消除, 每消除一次总数位和减少 $p - 1$, 而进位次数也减少一次。归纳于 c 即得

$$s_p(k) + s_p(n - k) = s_p(n) + c(p - 1).$$

代回上式即 $\nu_p \binom{n}{k} = c$ 。

这也给出一个实用推论: $\binom{n}{k}$ 不被 p 整除, 当且仅当竖式加法一次进位都没有, 即每一位 $k_i \leq n_i$ ——这正是下一条 Lucas 定理的内容。□

手算例. $\binom{10}{4} = 210$ 。二进制 $4 = (0100)_2$, $6 = (0110)_2$, 相加得到 $(1010)_2$, 在 2^2 位发生一次进位, 因此 $\nu_2(210) = 1$ 。三进制 $4 = (11)_3$, $6 = (20)_3$ 相加为 $(101)_3$, 也有一次进位, 所以 $\nu_3(210) = 1$ 。

连接到 Lucas

无进位等价于每一位 $k_i \leq n_i$ ，也等价于组合数不被 p 整除。Lucas 计算余数，Kummer 计算被 p 整除的深度，两者描述的是同一个数字结构的不同侧面。

6.4 Lucas 定理：逐位计算组合数

约定 $\binom{n}{k} = 0$ ($k < 0$ 或 $k > n$)。

定理 6.4 (Lucas). p 为素数, $n = \sum_i n_i p^i, k = \sum_i k_i p^i$, 则

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

等价的递归形式为

$$\binom{n}{k} \equiv \binom{\lfloor n/p \rfloor}{\lfloor k/p \rfloor} \binom{n \bmod p}{k \bmod p} \pmod{p}.$$

Lucas 只在素数模数下成立

证明依赖 $(1+X)^p \equiv 1+X^p \pmod{p}$, 这一步要求 p 为素数 (保证 $\binom{p}{i}$ 被 p 整除)。模 4 时它直接失效: $\binom{4}{2} = 6 \equiv 2 \pmod{4}$, 按**逐位相乘**却会得 $\binom{10}{10}$ 型的 1。合数模数要走**素数幂 + CRT** 那条路。

生成函数系数法. 二项式展开中 $\binom{p}{i}$ ($0 < i < p$) 均被 p 整除, 所以

$$(1+X)^p \equiv 1+X^p \pmod{p}.$$

不断使用此式, 得到

$$(1+X)^n \equiv \prod_i (1+X^{p^i})^{n_i}.$$

要取到 X^k , 各因子应分别选 $X^{k_i p^i}$. 因为每层指数小于 p , 这个选择由 p 进制表示唯一确定; 系数相乘即为结论。若某位 $k_i > n_i$, 相应系数为零。□

生成函数在这里其实就是多项式

$\sum_k \binom{n}{k} X^k = (1+X)^n$: 把**选法条数**当成一个多项式的系数。于是**模 p 的 $\binom{n}{k}$** 变成**多项式在 $(\bmod p)$ 下的某个系数**。用多项式的好处是可以相乘、可以只看某一项, 而不用讨论组合意义。

完整算例. 模 3 求 $\binom{10}{3}$. $10 = (101)_3, 3 = (010)_3$, 中间位出现 $1 > 0$, 故余数为 0。模 7 求 $\binom{10}{3}$, $10 = (13)_7, 3 = (03)_7$, 得到 $\binom{1}{0} \binom{3}{3} = 1$, 与 $120 \bmod 7 = 1$ 一致。

二进制结论. $\binom{n}{k}$ 为奇数当且仅当 k 的所有二进制 1 位都包含于 n , 可测试 $(k \& n) = k$ 。

复杂度与实现. 当 p 足够小, 可预处理 $0, \dots, p-1$ 的阶乘与逆阶乘, 单问 $O(\log_p n)$ 。若 p 本身非常大, $O(p)$ 预处理不现实, 需利用小 k 等附加条件; Lucas 本身不是所有素数模数下的“常数时间组合数”。

6.5 去掉素因子后的阶乘递推

定义 $U_p(n) = (n!)_p$, 即

$$n! = p^{\nu_p(n!)} U_p(n), \quad \gcd(U_p(n), p) = 1.$$

$(n!)_p$ 与 $U_p(n)$ 的区别

两个都很像、但不同: $(n!)_p$ 是把 $n!$ 里的 p 因子全部除尽; 而**去因子阶乘** $U_p(n)$ 是对 1 到 n 每个数先除尽 p 因子, 再相乘。数值上 $U_p(n) \equiv (n!)_p$, 但递归推导时必须用后者的定义, 才能把 p 的倍数那部分单独提出来。

固定 $Q = p^\alpha$, 预处理

$$F_Q(t) = \prod_{\substack{1 \leq i \leq t \\ p \nmid i}} i \pmod{Q}, \quad 0 \leq t \leq Q, \quad F_Q(0) = 1.$$

则

$$U_p(n) \equiv C_Q^{\lfloor n/Q \rfloor} F_Q(n \bmod Q) U_p(\lfloor n/p \rfloor) \pmod{Q}.$$

特别地, $Q = p$ 时

$$U_p(n) \equiv (-1)^{\lfloor n/p \rfloor} (n \bmod p)! U_p(\lfloor n/p \rfloor) \pmod{p}.$$

证明. 目标是把**去因子阶乘** $U_p(n)$ (把 $1, \dots, n$ 每个数中所有 p 因子除尽后再相乘, 模 $Q = p^\alpha$) 写成递归。把 $1, \dots, n$ 分成两部分。

第一部分: 非 p 倍数。模 Q 的完全剩余系中, 非 p 倍数的集合是 Q 周期重复的 (因为**是否 p 的倍数与同余模 Q 相容**: $p \mid (x + tQ) \iff p \mid x$)。所以每连续 Q 个整块的乘积都相同, 等于 $F_Q := \prod_{1 \leq j \leq Q, p \nmid j} j \bmod Q$; 一共 $\lfloor n/Q \rfloor$ 块, 剩下不足一块的尾部直接用预处理表查。

第二部分: p 的倍数, 即 $p, 2p, 3p, \dots, \lfloor n/p \rfloor p$ 。把其中 pj 的一个 p 除掉, 得到 j ; 若 j 还含因子 p , 那部分与 p^2 的**倍数**在第一层已被处理, 递归到 $\lfloor n/p \rfloor$ 正好继续除尽。故这部分的贡献恰是 $U_p(\lfloor n/p \rfloor)$ 。

合并两部分:

$$U_p(n) \equiv F_Q^{\lfloor n/Q \rfloor} \cdot (\text{尾部 } U_p(n \bmod Q)) \cdot U_p(\lfloor n/p \rfloor) \pmod{Q}.$$

深度为 $\log_p n$, 每层 $O(1)$ 查表。最后要检查的是: F_Q 只有 ± 1 (Wilson 型论证: 单位集合在取逆下不变, 只有自逆元 ± 1 无法配对; 当 Q 为 $2, 4, p^\alpha, 2p^\alpha$ 之外的模数时可能有更多自逆元, 此时应直接按预处理表算, 而不是套用 ± 1)。□

验算. $p = 2, Q = 8, n = 10$. $C_8 = 1, F_8(2) = 1$; 递归的 n 为 $10, 5, 2, 1$, 得到

$$U_2(10) \equiv F_8(2)F_8(5)F_8(2)F_8(1) \equiv 1 \cdot 7 \cdot 1 \cdot 1 = 7 \pmod{8}.$$

实际 $10!/2^8 = 14175$, 余数也为 7。

复杂度与实现. 预处理 $O(Q)$ 时间、空间。由于 $C_Q \in \{1, -1\}$, 其幂只看奇偶; 一次 $U_p(n)$ 仅 $O(\log_p(n+1))$ 次模乘。若每层仍用普通快速幂计算整块贡献, 代码会额外产生一个对数因子, 复杂度表必须与实现一致。

6.6 扩展 Lucas：先计算素数幂，再 CRT

对 $M = \prod_i p_i^{\alpha_i}$ ，分别计算模 $Q_i = p_i^{\alpha_i}$ 的答案。令

$$e = \nu_p(n!) - \nu_p(k!) - \nu_p((n-k)!),$$

三步流程

$Q = p^\alpha$ ：第一步用 Legendre 算指数 $e = \nu_p$ ，若 $e \geq \alpha$ 答案就是 0；第二步用去因子阶乘递归算单位部分，分母用逆元（单位一定可逆）；第三步对各个 Q 的结果做 exCRT。每一步都要单独检查模数，不能合并成一次求阶乘逆元。

则

$$\binom{n}{k} \equiv \begin{cases} 0, & e \geq \alpha, \\ p^e U_p(n) U_p(k)^{-1} U_p(n-k)^{-1}, & e < \alpha \end{cases} \pmod{p^\alpha}.$$

分母的两个单位部分一定可逆，使用 exgcd 求逆即可。最后按互质素数幂进行 CRT。

从头算 $\binom{10}{4} \pmod{12}$ 。模 4： $e = 8 - 3 - 4 = 1$ ， $U_2(10) \equiv 3, U_2(4) \equiv 3, U_2(6) \equiv 1$ ，所以

$$\binom{10}{4} \equiv 2 \cdot 3 \cdot 3^{-1} \cdot 1^{-1} \equiv 2 \pmod{4}.$$

模 3： $e = 4 - 1 - 2 = 1 \geq 1$ ，余数为 0。解 $x \equiv 2 \pmod{4}, x \equiv 0 \pmod{3}$ 得 $x \equiv 6 \pmod{12}$ ，与 $210 \pmod{12} = 6$ 一致。

实现流程.

1. 若 $M = 1$ ，返回 0；若 $k < 0$ 或 $k > n$ ，按约定返回 0。
2. 分解 M ，对每个素数幂建立 F_Q 。
3. 用 Legendre 求 e ，先做零值判定，避免无用计算。
4. 求三个 U ，只对单位求逆，再乘 p^e 。
5. CRT 合并；禁止在模 M 下先求 $k!$ 的逆元。

复杂度与实现. 不计模数分解，预处理 $O(\sum_i Q_i)$ 。使用单位块符号优化后，一次询问的主要成本为

$$O\left(\sum_i (\log_{p_i}(n+1) + \log Q_i)\right)$$

以及 CRT 合并成本。原稿将查询复杂度只写成模数的函数，漏掉了 n 的递归层数。

6.7 原稿例题的完整建模

HDU 2973: YAPTCHA

对 $n \leq 10^6$ ，多次求

$$\sum_{k=1}^n \left\lfloor \frac{(3k+6)! + 1}{3k+7} - \left\lfloor \frac{(3k+6)!}{3k+7} \right\rfloor \right\rfloor.$$

怎么做：把嵌套取整化成判定式

看到**嵌套取整 + 素数判定**的组合（例如把 [某式] 当成示性函数），第一反应应该是：能否化成 Wilson 型判定，或者 gcd 型计数。这类题很少真的需要枚举。

解题方向. 单项恰为 $[3k + 7 \text{ 为素数}]$ ，筛法加前缀和。

讲解：先看这题到底在做什么

先观察题目的巨大阶乘表达式，不要真的去算阶乘。把 $(N - 1)!$ 除以 N 的余数写出来后，括号里的取整量恰好变成 Wilson 定理的判定条件，于是原题就被翻译成“统计某个区间里的素数”。

解析. 设 $N = 3k + 7$, $(N - 1)! = qN + r$, $0 \leq r < N$ 。括号内等于 $(r + 1)/N$ ，取整为 1 当且仅当 $r = N - 1$ 。由 Wilson 的充要条件，这恰等价于 N 为素数。只需筛到 $3n_{\max} + 7$ ，令 $A_k = A_{k-1} + [3k + 7 \in \mathbb{P}]$ 。例如 $n = 5$ 时考察 10, 13, 16, 19, 22，答案为 2。

复杂度与实现. 埃氏筛 $O(N \log \log N)$ 或线性筛 $O(N)$ ，每问 $O(1)$ 。本题绝不计算阶乘；超大表达式常常是在提示一个判定定理。

题目 / 延伸阅读资料: <https://acm.hdu.edu.cn/showproblem.php?pid=2973>

P2183：礼物

n 件互不相同的礼物送给 m 个有编号的人，第 i 人收到 a_i 件，允许剩余。求方案数模 M ；若不可能则按原题输出 Impossible。 $n \leq 10^9, m \leq 5$ ，模数的每个素数幂因子不超过 10^5 。

解题方向. $\sum_i a_i > n$ 时无解；否则是一个多项式系数，可写为一串组合数的乘积。

讲解：先看这题到底在做什么

把“依次给每个人分配 a_i 件”看成一连串组合选择：第 i 个人从还剩下的物品中选出 a_i 件。这样每一步独立形成一个组合数，乘起来后自然得到阶乘商。关键是先定义前缀总量 s_i ，把每一步还剩多少说清楚。

解析. 令 $s_0 = 0, s_i = \sum_{j=1}^i a_j$ ，则

$$\text{Ans} = \prod_{i=1}^m \binom{n - s_{i-1}}{a_i} = \frac{n!}{(n - s_m)! \prod_i a_i!}.$$

提醒： $O(N)$ 不等于省内存

$O(N)$ 不等于省内存。 $N = 10^8$ 时，多个 int 数组就是 400 MB 级别。只判素数用位压缩的埃氏筛；要 φ, μ, lp 再上线性筛，并先算内存再写代码。

逐人选礼物：第一人从 n 件中选，第二人从剩余中选，依此类推。每个完整分配恰计一次，不需要再除 $m!$ ，因为人有编号。可以调用 m 次扩展 Lucas，或者直接在每个素数幂上对多项式系数统计单位部分与指数。

算例. $n = 4, m = 2, (a_1, a_2) = (1, 2)$ ，答案 $\binom{4}{1} \binom{3}{2} = 12$ 。模 6 后为 0，但这是有方案而余数为零，不能输出 Impossible。只有数量约束失败才输出 Impossible。

题目 / 延伸阅读资料: <https://www.luogu.com.cn/problem/P2183>

P7488: 黑色毛衣

选 m 个整数边长的等腰三角形, 腰长 $a \in [1, n]$ 、底长 $b \in [1, 2a - 1]$, 要求所有腰长互不相同、所有底长也互不相同, 忽略三角形的排列顺序。求组数模一般模数 M 。 $m \leq n \leq 10^{18}, M \leq 10^5$ 。

解题方向. 答案是 $\binom{n}{m} n^m = \binom{n}{m}^2 m!$, 先证明计数, 再处理不可逆阶乘。

讲解: 先看这题到底在做什么

先忽略“模数可能不能除阶乘”的实现问题, 只解决计数。把腰长看成行、底长看成列, 就是一个阶梯棋盘上的不重复选点问题; 最后一行是否被使用会给出递推式, 再把递推化成组合数。

解析. 把腰长 a 看成棋盘的第 a 行, 该行可选列为 $1, \dots, 2a - 1$ 。选三角形等价于在行、列都不能重复的阶梯棋盘上放 m 个车。记方案数为 $R(n, m)$ 。不使用最后一行的方案数为 $R(n - 1, m)$ 。使用最后一行时, 前面已经放了 $m - 1$ 个车, 占用 $m - 1$ 个不同列; 这些列全部在最后一行的范围内, 所以还有

$$(2n - 1) - (m - 1) = 2n - m$$

个位置。因此

$$R(n, m) = R(n - 1, m) + (2n - m)R(n - 1, m - 1), \quad R(n, 0) = 1.$$

将 $\binom{n}{m}^2 m!$ 代入即可验证递推 (先提出公共因子, 或使用 Pascal 恒等式), 初值也相同, 所以由归纳法成立。

小规模核对. $n = 3, m = 1$ 时为 $1 + 3 + 5 = 9$; $n = 3, m = 2$ 时, 所选两行分别为 $(1, 2), (1, 3), (2, 3)$, 方案数为 $2, 4, 12$, 总计 $18 = \binom{3}{2}^2 \cdot 2!$ 。

合数模数下如何算? 使用

$$R(n, m) = \frac{(n!)^2}{m!((n - m)!)^2}.$$

对每个 $p^\alpha \mid M$, 总指数为

$$e = 2\nu_p(n!) - \nu_p(m!) - 2\nu_p((n - m)!),$$

单位部分相同地相乘求逆, 再 CRT。不能先算 $m! \bmod M$, 再据此决定整个分式是否为零。

易错点

题意不是要求全部 $2m$ 个“腰长数值与底长数值”两两不同。一个三角形可以是等边三角形, 某只的底长也可以等于另一只的腰长。若误删 $b = a$, 计数公式就变了。

题目 / 延伸资料: <https://www.luogu.com.cn/problem/P7488>

第 7 章

数论函数、狄利克雷卷积与筛法

章节导读

前面几章主要研究单个整数的结构；这一章开始把视野扩大到定义在所有正整数上的函数。先区分积性与完全积性，再用狄利克雷卷积把约数求和重新组织起来，最后把这些公式转化成可以在线性或近线性时间运行的筛法。

7.1 先分清积性与完全积性

定义 7.1. 数论函数是定义在正整数上的函数，通常取值于复数或某个模数环。若 $f(1) = 1$ ，且 $\gcd(a, b) = 1$ 时 $f(ab) = f(a)f(b)$ ，称为积性函数。若不要求互质也恒有 $f(ab) = f(a)f(b)$ ，称为完全积性函数。

积性函数由所有素数幂上的取值唯一确定： $f(\prod p_i^{e_i}) = \prod f(p_i^{e_i})$ 。

函数	定义	p^k 处的值 ($k \geq 1$)
ε	$\varepsilon(n) = [n = 1]$	0
$\mathbf{1}$	$\mathbf{1}(n) = 1$	1
μ	有平方因子时为 0；否则为 $(-1)^{\omega(n)}$	-1 ($k = 1$), 0 ($k \geq 2$)
τ	正约数个数	$k + 1$
σ_t	所有正约数的 t 次幂之和	$1 + p^t + \cdots + p^{kt}$
φ	与 n 互质的正整数个数	$(p - 1)p^{k-1}$
id_t	$\text{id}_t(n) = n^t$	p^{kt}

最小反例. φ 积性但不完全积性： $\varphi(4) = 2 \neq \varphi(2)^2 = 1$ 。 $\mu(4) = 0 \neq \mu(2)^2 = 1$ 。将积性函数当成完全积性，是后文洲阁筛中最危险的错误之一。

为什么 μ 长这样？ 对 $n = \prod_{i=1}^r p_i^{e_i} > 1$ ，只有平方自由约数贡献非零项，因此

$$\sum_{d|n} \mu(d) = \sum_{S \subseteq \{1, \dots, r\}} (-1)^{|S|} = (1 - 1)^r = 0.$$

词义：积性 / 完全积性

一句话记法：积性 = **互质才能拆**，完全积性 = **随便拆**。检查函数属于哪一种，只要代 $a = b = 2$ 试一次； $\tau(4) = 3 \neq 4$ 就说明 τ 只是积性。后面洲阁筛里把 x^t （完全积性）与积性函数混用会直接算错。

词义：把 $\mu(d)$ 当开关

只有……贡献非零项的意思是：把 $\mu(d)$ 当成开关，非平方自由的 d 一律 $\mu(d) = 0$ ，求和时自动跳过。所以 $\sum_{d|n} \mu(d)$ 只取决于 n 有几个不同素因子： 2^r 个平方自由约数，一半带 +、一半带 -，除非 $n = 1$ 。

由素数幂确定有多省

例： $72 = 2^3 \cdot 3^2$ ，只要知道 $\varphi(8) = 4$ 、 $\varphi(9) = 6$ ，就有 $\varphi(72) = 24$ ，不必枚举 72 个数。做题时的动作是：先把 n 分解，再逐素数幂查公式，最后相乘；写成代码就是 $\log n$ 的因子数次查表。

$n = 1$ 时只有约数 1，结果为 1。它本质上是对不同素因子的容斥。

7.2 欧拉函数的公式与恒等式

容斥或 CRT 可得

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right).$$

因此 φ 是积性函数，且对任意正整数 a, b 、 $d = \gcd(a, b)$,

$$\varphi(ab)\varphi(d) = \varphi(a)\varphi(b)d.$$

第二个恒等式. 按素因子比较。当 p 只整除 a, b 中一个时，两边相同；当它同时整除两者时，设赋值为 $u, v \geq 1$ ，两边关于 p 的因子分别是

$$(p-1)^2 p^{u+v+\min(u,v)-2},$$

仍相同。把各个素数的贡献乘起来即可。 $\square$

对 $n = 12$ ，公式为 $12(1 - 1/2)(1 - 1/3) = 4$ ，对应 1, 5, 7, 11。实现时用 $ans = ans/p \cdot (p-1)$ ，避免浮点误差。

7.3 狄利克雷卷积与逆元

定义 7.2.

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d) = \sum_{ab=n} f(a)g(b).$$

卷积满足交换律、结合律，并对逐点加法分配；其单位元是 ε ，不是 **1**。若 $g(1) \neq 0$ （在模数环中需为可逆元素），则 $g * h = f$ 有唯一解。递推式为

$$h(1) = \frac{f(1)}{g(1)}, \quad h(n) = \frac{f(n) - \sum_{d|n, d>1} g(d)h(n/d)}{g(1)}.$$

自己验证一次只要两行

交换律：把 $(f * g)(n) = \sum_{d|n} f(d)g(n/d)$ 里的 d 换成 n/d （这是约数集合上的对合，一一自对应），求和就变成 $\sum_{d|n} f(n/d)g(d) = (g * f)(n)$ 。结合律：两边都等于 $\sum_{abc=n} f(a)g(b)h(c)$ 。凡是**显然满足**的说法，都能这样两行落地。

两个积性函数的卷积仍积性；积性函数的卷积逆及积性函数之间的卷积商仍积性。

为什么卷积保持积性？若 $\gcd(a, b) = 1$ ，每个 ab 的约数唯一写成 uv ， $u \mid a, v \mid b$ 。于是

$$(f * g)(ab) = \sum_{u \mid a} \sum_{v \mid b} f(u)f(v)g(a/u)g(b/v) = (f * g)(a)(f * g)(b).$$

对卷积商，先在每个素数幂上递推求值，再由积性延拓得到 h' ，可验证 $g * h' = f$ ；唯一性保证 $h' = h$ 。□

局部生成函数。 固定一个素数 p ，令 $F_p(z) = \sum_{k \geq 0} f(p^k)z^k$ 。那么卷积对应形式幂级数乘法

$$H_p(z) = F_p(z)G_p(z).$$

这里说的是形式幂级数，不要求它真的收敛。 $\mathbf{1}$ 的局部级数为 $(1-z)^{-1}$ ， μ 为 $1-z$ ，所以 $\mathbf{1} * \mu = \varepsilon$ 。

易错点

写 $\varphi = \text{id} * \mu$ 是卷积恒等式，不是 $\varphi(n) = n\mu(n)$ 。同样，“两个积性函数相加”未必积性，例如 $\mathbf{1} + \mathbf{1}$ 在 1 处为 2，已不满足归一化定义。

7.4 线性筛：唯一的最小素因子分解

对合数 x ，令 p 为其最小素因子、 $i = x/p$ ，则 $p \leq \min p(i)$ 。线性筛枚举 i ，再递增枚举素数 p ，标记 ip ；一旦 $p \mid i$ 即标记后退出内层循环。每个合数恰被处理一次。计算 φ, μ 时使用

	$p \mid i$	$p \nmid i$
$\varphi(ip)$	$p\varphi(i)$	$(p-1)\varphi(i)$
$\mu(ip)$	0	$-\mu(i)$

为什么：每个合数只被最小素因子筛一次

$O(N)$ 的关键是**每个合数只被它的最小素因子筛掉一次**：循环遇到 $i \bmod p == 0$ 就 break，保证不会用更大的素因子去生成同一个数。 φ, μ 能同步线性求出，也是靠这条唯一分解。

形式两个字的含义

只看系数、不问 z 取什么值： $\sum a_k z^k$ 就是一张系数表，乘法是**系数的卷积**。所以 $(1-z)^{-1} = 1 + z + z^2 + \dots$ 在这里是**定义**而不是**收敛计算**——你要验证的是两边系数相乘后 z^k 的系数为 1，而不是级数是否收敛。

并设置 $\varphi(1) = \mu(1) = 1$ 。

为何要在整除时退出？ 若 $p \mid i$ ，那么 p 已是 i 的最小素因子；继续枚举更大的 q ， iq 的最小素因子仍是 p ，它会在唯一的分解 $p \cdot (iq/p)$ 中被处理。继续做只会重复，并破坏线性复杂度的计数依据。

```
vector<int> primes, lp(N+1), phi(N+1), mu(N+1);
phi[1]=mu[1]=1;
for (int i=2; i<=N; ++i) {
    if (!lp[i]) {
        lp[i]=i; primes.push_back(i);
        phi[i]=i-1; mu[i]=-1;
    }
}
```

```

for (int p:primes) {
    if (p>N/i) break;
    int v=i*p; lp[v]=p;
    if (i%p==0) {
        phi[v]=phi[i]*p; mu[v]=0;
        break;
    }
    phi[v]=phi[i]*(p-1); mu[v]=-mu[i];
}
}

```

扩展到一般积性函数. 当 $p \nmid i$ 时 $f(ip) = f(i)f(p)$; 当 $p \mid i$ 时应把 $i = p^e u$ 分离出来, 维护最小素因子的指数和对应幂, 算 $f(p^{e+1})f(u)$ 。不能假定任意积性函数都存在“除以 $f(p^e)$ ”的安全转移: 该值可能为零或不可逆。

复杂度与实现. 时间 $O(N)$, 常规数组空间 $O(N)$ 。 $N = 10^8$ 时四个 int 数组已约 1.6 GB, 应按需要只保留必要数据; 若只筛素数, 位图埃氏筛可能更合适。

7.5 整除分块: 相同商的最大区间

对 $1 \leq l \leq n$, 令 $q = \lfloor n/l \rfloor$, 则商保持 q 的区间右端点为

$$r = \lfloor n/q \rfloor.$$

$r = \lfloor n/q \rfloor$ 是怎么想到的

固定 l , $q = \lfloor n/l \rfloor$, 要找最大的 r 使 $\lfloor n/x \rfloor = q$ 对整段成立。条件等价于 $q \leq n/x < q+1$, 左半边给出 $x \leq n/q$, 所以 $r = \lfloor n/q \rfloor$ (右半边给出 $x \geq \lfloor n/(q+1) \rfloor + 1$, 由 l 已经满足)。代码里 $l = r + 1$ 跳到下一块, 块数 $O(\sqrt{n})$ 来自小 l 至多 $\sqrt{n}$ 个, 大的商也不超过 $\sqrt{n}$ 。

这是因为 $\lfloor n/x \rfloor \geq q \iff x \leq \lfloor n/q \rfloor$ 。所有不同商只有 $O(\sqrt{n})$ 个: $x \leq \sqrt{n}$ 的位置不超过 $\sqrt{n}$, 其余位置的商也不超过 $\sqrt{n}$ 。

例子. $n = 10$ 的商块为

$$[1, 1] \mapsto 10, [2, 2] \mapsto 5, [3, 3] \mapsto 3, [4, 5] \mapsto 2, [6, 10] \mapsto 1.$$

若同时含 $\lfloor n/i \rfloor, \lfloor m/i \rfloor$, 共同右端点应取

$$r = \min(\lfloor n/\lfloor n/l \rfloor \rfloor, \lfloor m/\lfloor m/l \rfloor \rfloor),$$

并仅枚举到 $\min(n, m)$ 。不能用其中一个商的区间跨过另一个商的变化点。

```

for (long long l=1, r; l<=n; l=r+1) {
    long long q=n/l;
    r=n/q;
    // sum over [l, r] = (prefix[r]-prefix[l-1]) * F(q)
}

```

7.6 狄利克雷前缀和、后缀和与差分

“前缀”在此按整除偏序，不是普通区间前缀。

$$Z_f(n) = \sum_{d|n} f(d) = (f * \mathbf{1})(n), \quad U_f(n) = \sum_{\substack{k \leq N \\ n|k}} f(k).$$

这三个词在说什么

把 $f \mapsto g$, $g(n) = \sum_{d|n} f(d)$ (每个位置吸收它的约数) 叫狄利克雷**前缀和**; $g(n) = \sum_{n|d} f(d)$ (每个位置吸收它的倍数) 叫**后缀和**; 它们的逆运算统称**差分**。实现就是枚举 d 或枚举倍数, 复杂度 $O(N \log N)$, 本质是调和级数。

利用素数逐层传播, 可在 $O(N \log \log N)$ 完成这些变换; 反向操作给出差分。

```
// a[1..N] initially stores f. primes contains all primes <= N.
// Divisor zeta: a[n] <- sum_{d|n} f[d]
for (int p:primes)
    for (int i=1; i<=N/p; ++i) a[i*p] += a[i];

// Its inverse (run on the transformed array):
for (auto it=primes.rbegin(); it!=primes.rend(); ++it)
    for (int i=N/(*it); i>=1; --i) a[i*(*it)] -= a[i];

// Multiple zeta: a[n] <- sum_{n|k, k<=N} f[k]
for (int p:primes)
    for (int i=N/p; i>=1; --i) a[i] += a[i*p];
```

为何前缀递增、后缀递减？ 前缀时希望 p^2 能收到已经汇总到 p 的贡献, 故从小到大; 后缀时希望 1 能收到已经汇总到 p 的贡献, 故从大到小。固定 p , 整数按“不含 p 的部分”分成链 $u, up, up^2, \dots$, 上述循环恰在每条链上做普通前缀和或后缀和。不同素数维度依次处理, 正好枚举每个约数一次。

推广：任意函数卷一个积性函数。 已知积性 g 的素数幂点值, 逐个素数在上述幂链上做一维卷积即可。长度为 $\ell + 1$ 的链即使朴素 $O(\ell^2)$, 固定 p 的非平凡链总成本仍为 $O(N/p)$; 因此总计 $O(N \log \log N)$ 。这里默认素数幂点值可常数时间读取, 且只枚举起点 $u \leq N/p, p \nmid u$, 不能每个素数都扫完整个 $[1, N]$ 。

第 8 章

莫比乌斯反演与 gcd 型求和

章节导读

上一章我们学会了用卷积组织“对所有约数求和”的关系；有了这套语言，就可以反过来把被混在一起的贡献拆开。莫比乌斯反演正是这种“消掉约数层层贡献”的工具，它会进一步把许多看似复杂的 gcd 求和变成可以交换顺序、直接计算的形式。

8.1 两种基础“拆贡献”恒等式

$$\sum_{d|n} \mu(d) = [n=1], \quad \sum_{d|n} \varphi(d) = n.$$

词义：拆贡献

拆贡献 = 不直接按 i 算，而是按谁对答案造成多少重新分组。两种写法：按约数拆 $\sum_{d|n}$ 、按倍数拆 $\sum_{d|k}$ 。选哪一种，取决于化简后哪个和式容易算——通常先各自试两行，再决定。

卷积写法为 $\mu * 1 = \varepsilon$ 、 $\varphi * 1 = \text{id}$ ，从而 $\varphi = \text{id} * \mu$ 。

定理 8.1 (莫比乌斯反演).

$$F(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) F(n/d).$$

若函数在有限范围外为零，则还有倍数版本

$$F(n) = \sum_{k \geq 1} f(kn) \iff f(n) = \sum_{k \geq 1} \mu(k) F(kn).$$

因数版本. 两个方向都写全，且把哪些和式合法说清楚。

因数版本：若 $g(n) = \sum_{d|n} f(d)$ ，则

$$\sum_{d|n} \mu(d) g(n/d) = \sum_{d|n} \mu(d) \sum_{e|n/d} f(e).$$

右边是有限和，可以自由换序。 (d, e) 跑遍 $d|n$ 且 $e|n/d$ ，等价于 $e|n$ 且 $d|n/e$ (因为 $de|n$)。于是

$$\sum_{e|n} f(e) \sum_{d|n/e} \mu(d).$$

内层和是经典判别式: $\sum_{d|m} \mu(d) = [m=1]$ (按 μ 的定义, $m = \prod p^{u_p}$ 时内层为 $\prod_p (1 + (-1) + \dots + (-1)^{u_p})$, 只要某个 $u_p \geq 1$ 就有因子 $1 - 1 = 0$; $m = 1$ 时为 1)。因此只剩 $n/e = 1$, 即 $e = n$ 的项, 得到 $f(n)$ 。

倍数版本: 若 $g(n) = \sum_{n|d, d \leq N} f(d)$, 则

$$\sum_{k \geq 1} g(kn) \mu(k) = \sum_{k \geq 1} \mu(k) \sum_{\ell \geq 1} f(k\ell n) = \sum_{m \leq N, n|m} f(m) \sum_{k|m/n} \mu(k) = f(n),$$

最后一步同样是把 $k\ell n$ 合成一个变量 m , 并用 $\sum_{d|m/n} \mu = [m/n = 1]$ 。

两个版本的区别只在于 f 是挂在约数上还是挂在倍数上, 用哪一版取决于你先把原式整理成哪一种。□

欧拉恒等式的组合证明. 把 $1, \dots, n$ 按 $\gcd(i, n) = g$ 分类。写 $i = gj$, 该类恰有 $\varphi(n/g)$ 个数; 各类总数为 n , 换元 $d = n/g$ 得到公式。

怎样想到反演?

看到 $[\gcd(i, j) = 1]$, 替换成 $\sum_{d|i, d|j} \mu(d)$; 看到 $\gcd(i, j)$, 替换成 $\sum_{d|i, d|j} \varphi(d)$; 看到任意 $C(\gcd(i, j))$, 先求 $D = C * \mu$, 再写 $C = 1 * D$ 。反演的目的不是“制造更多求和号”, 而是让 i, j 分离。

8.2 先处理单变量与互质计数

$$\sum_{i=1}^n \gcd(i, n) = \sum_{d|n} \varphi(d) \lfloor n/d \rfloor = \sum_{d|n} d \varphi(n/d).$$

拆贡献的三步套路

第一步把要求的東西写成 $\sum_i f(\gcd(\dots))$ 之类; 第二步用 $f(n) = \sum_{d|n} c(d)$ 把 f 换掉, 于是 $f(\gcd)$ 变成 d 整除各变量的 $\square$ 条件; 第三步交换求和顺序, 内层只剩倍数个数。整套动作没有灵感要求, 练十道题就会背下来。

以及

$$\#\{(i, j) : 1 \leq i \leq n, 1 \leq j \leq m, \gcd(i, j) = 1\} = \sum_{d=1}^{\min(n, m)} \mu(d) \lfloor n/d \rfloor \lfloor m/d \rfloor.$$

第一式的每一步.

$$\sum_i \gcd(i, n) = \sum_i \sum_{d|i, d|n} \varphi(d) = \sum_{d|n} \varphi(d) \sum_i [d|i].$$

原稿在插入约数求和后仍保留 $\gcd(i, n)$, 会把同一个 $\gcd$ 重复累加约数个数次; 这里必须替换为 $\varphi(d)$ 。例如 $n = 6$, 直接求和 $1 + 2 + 3 + 2 + 1 + 6 = 15$; 约数形式给出 $6 \cdot 1 + 3 \cdot 1 + 2 \cdot 2 + 1 \cdot 2 = 15$ 。

互质计数验算. $n = m = 3$ 时 $9 - 1 - 1 = 7$; 九个有序数对中只有 $(2, 2), (3, 3)$ 不互质。注意计的是有序数对, 不能再额外乘二。

P2398: GCD SUM求 $\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j)$ 。**解题方向.** 答案为 $\sum_{d=1}^n \varphi(d) \lfloor n/d \rfloor^2$ ，可筛 φ 并利用前缀和做整除分块。**讲解：先看这题到底在做什么**看到双重求和里的 gcd，先把 gcd 用欧拉展开成“所有公共约数的权重之和”。交换求和后，两个坐标只剩下“有多少个数同时被 d 整除”，这就是 $\lfloor n/d \rfloor$ ，于是二维问题变成一维整除分块。**解析.** 把 gcd 的欧拉展开代入，交换三层求和：

$$\sum_{i,j} \sum_{d|i,d|j} \varphi(d) = \sum_d \varphi(d) \left(\sum_i [d|i] \right) \left(\sum_j [d|j] \right).$$

两个括号都为 $\lfloor n/d \rfloor$ ，得到公式。 $n=3$ 时矩阵为

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 3 \end{pmatrix},$$

总和 12，公式也给 $9 + 1 + 2 = 12$ 。**复杂度与实现.** 线性筛预处理 $O(N)$ 后，每问分块 $O(\sqrt{n})$ 。若只有一个不大的 n ，直接累加 n 项也是 $O(n)$ ，不用为了形式上“高级”而特意分块。前缀和、商的平方和最终答案都要用足够宽的整数。题目 / 延伸资料：<https://www.luogu.com.cn/problem/P2398>**一般加权 gcd 求和**给定三个长度为 n 的数组 A, B, C ，求

$$\sum_{i=1}^n \sum_{j=1}^n A(i)B(j)C(\gcd(i, j)), \quad n \leq 2 \times 10^6.$$

怎么做：先做莫比乌斯差分，再数倍数凡是答案形如 $\sum f(\gcd(a_i, b_j))$ ，机械做法都一样：把 f 写成 $f = \sum_{d|n} c(d)$ （即对 f 做莫比乌斯差分），于是 gcd 变成 d 同时整除两个数的计数，后面全是倍数计数与容斥。**解题方向.** 求 $D = C * \mu$ ，以及 A, B 的倍数后缀和，答案变成一维点积。**讲解：先看这题到底在做什么**这里把上一题的核心机制抽象出来：只要一个函数能够写成约数卷积形式 $C = \mathbf{1} * D$ ，那么 $C(\gcd(i, j))$ 就可以按公共因子 d 拆贡献。之后分别预处理两个数组中“被 d 整除”的元素总和，答案就成为一维点积。**解析.** 由 $C = \mathbf{1} * D$,

$$\begin{aligned} \text{Ans} &= \sum_{i,j} A(i)B(j) \sum_{d|i,d|j} D(d) \\ &= \sum_{d=1}^n D(d) \left(\sum_{d|i} A(i) \right) \left(\sum_{d|j} B(j) \right). \end{aligned}$$

A, B, C 都不要积性。上一章的素数维度变换适用于任意数组：对 C 做因数前缀的逆变换，得到 D ；对 A, B 各做倍数后缀，最后逐点相乘求和。

复杂度与实现. 时间 $O(n \log \log n)$ ，空间 $O(n)$ 。若用朴素枚举所有倍数，也能得到 $O(n \log n)$ 的更易写版本，应根据时限选择。

追问. 取 $A = B = 1, C(n) = n$ 会恢复哪题？恢复 GCD SUM，因为 $D = \varphi$ 。取 $C(n) = [n = 1]$ 则 $D = \mu$ ，恢复互质数对计数。

8.3 多询问与不规则预处理表

P4240：毒瘤之神的考验

多次求

$$\sum_{i=1}^n \sum_{j=1}^m \varphi(ij) \pmod{998244353},$$

其中 $T \leq 10^4$ ， $n, m \leq N = 10^5$ 。

词义：不规则的预处理表

不规则指：表不能建成整齐的 $N \times N$ ，只能按 $\lfloor n/k \rfloor$ 的取值分段存储。做法是把大参数 ($> K$) 的情况列成一张稀疏表，查询时先定位区间端点，再取前缀差。

解题方向. 先用 $\varphi(ij) = \varphi(i)\varphi(j) \gcd(i, j) / \varphi(\gcd(i, j))$ 转成加权 gcd；再预处理倍数方向的部分和。不能每次询问都做 $O(N)$ 全扫。

讲解：先看这题到底在做什么

先处理函数本身：把 $k/\varphi(k)$ 写成卷积形式，从而制造出适合公共因子拆分的系数。完成这一步后，题目就退化成一般加权 gcd 求和；实现上真正需要优化的是倍数方向的预处理，而不是重新设计一套计数公式。

解析. 把函数 $A(k) = k/\varphi(k)$ 拆为 $A = \mathbf{1} * f$ ，则

$$f(k) = \sum_{d|k} \frac{d\mu(k/d)}{\varphi(d)}.$$

套用一般加权 gcd 公式，得到

$$\text{Ans}(n, m) = \sum_{k=1}^{\min(n, m)} f(k) G(k, \lfloor n/k \rfloor) G(k, \lfloor m/k \rfloor),$$

其中 $G(k, t) = \sum_{i=1}^t \varphi(ki)$ 。

第一层优化：化简原稿中未继续化简的 f . A 积性，因此 $f = A * \mu$ 积性；在素数幂处

$$f(p) = \frac{p}{p-1} - 1 = \frac{1}{p-1}, \quad f(p^e) = 0 \quad (e \geq 2).$$

所以

$$f(k) = \frac{\mu(k)^2}{\varphi(k)}.$$

在本题模数下所有 $\varphi(k) \leq N < 998244353$ 都可逆。这个形式可以线性筛或直接用 μ, φ 表得到。

第二层优化：只存实际存在的 G 项。 对于固定 k ，只需 $0 \leq t \leq \lfloor N/k \rfloor$ ，递推

$$G(k, 0) = 0, \quad G(k, t) = G(k, t-1) + \varphi(kt).$$

总项数 $\sum_k \lfloor N/k \rfloor = O(N \log N)$ 。不要开 $N \times N$ 矩阵；使用按 k 分行的变长数组或一段连续内存。

第三层优化：小 k 直接算，大 k 按商分块。 取阈值参数 $1 \leq B \leq N$ ，令 $K = \lfloor N/B \rfloor$ 。 $k \leq K$ 时每问直接枚举； $k > K$ 时两个商均不超过 B 。对 $1 \leq u, v \leq B$ 预处理

$$H_{u,v}(r) = \sum_{K < k \leq r} f(k)G(k, u)G(k, v), \quad K \leq r \leq \lfloor N/\max(u, v) \rfloor.$$

对没有合法 r 的 (u, v) 不开表。若询问的共同商区间为 $[l, r]$ 、 $u = \lfloor n/l \rfloor, v = \lfloor m/l \rfloor$ ，该块贡献就是

$$H_{u,v}(r) - H_{u,v}(l-1).$$

共同右端点为 $\min(n/u, m/v)$ ，所以索引一定处于该行合法范围。

为什么这张三维表开得下？ 若对所有 (u, v, r) 开等长数组，需要 $O(NB^2)$ ，但实际 r 的上界依赖 $\max(u, v)$ 。只开有效范围，项数不超过

$$\sum_{u=1}^B \sum_{v=1}^B \lfloor N/\max(u, v) \rfloor \leq N \sum_{w=1}^B \frac{2w-1}{w} = O(NB).$$

等价地，对每个 $k > K$ 枚举 $u, v \leq N/k$ ，成本为 $\sum_{k>K} O((N/k)^2) = O(N^2/K) = O(NB)$ 。 $H_{u,v} = H_{v,u}$ 还可以再减半存储。

复杂度与实现。 总预处理时间、空间为 $O(N \log N + NB)$ ；每次询问 $O(N/B + \sqrt{N})$ 。例如 $N = 10^5, B \approx 100$ 时，直接枚举约 1000 项，其余使用商块；内存按真实表项计算后选择阈值。这里的界对应**变长有效表**实现，不能套在未经压缩的三维数组上。

小例验算。 $n = 2, m = 3$ ，直接求和为 $\varphi(1) + \varphi(2) + \varphi(3) + \varphi(2) + \varphi(4) + \varphi(6) = 9$ 。公式中 $k = 1$ 贡献 $2 \cdot 4 = 8$ ， $k = 2$ 贡献 $1 \cdot 1 \cdot 1 = 1$ ，合计 9。

这道题的三个教学收获

把复杂二维函数转为 gcd 权重；在素数幂处继续化简卷积系数；开多维表前先写出每一维的合法范围。原稿“前面后面分一下”不足以支撑实现，应把三个函数的参数含义与存储上界完整写出。

题目 / 延伸资料：<https://www.luogu.com.cn/problem/P4240>

第 9 章

积性函数求和：杜教筛与 Powerful Number

章节导读

上一章的反演解决了如何拆贡献，但如果 n 本身已经大到不能逐项枚举，新的瓶颈就变成“这些函数值怎么快速求前缀和”。这一章从商集和倍增估计出发，再进入杜教筛与 Powerful Number，用结构化的递归和稀疏支撑代替完整枚举。

9.1 先看规模，再看倍增估计

目标是求 $S_f(n) = \sum_{i \leq n} f(i)$ 。当 n 很大时，还经常要求所有 $v = \lfloor n/k \rfloor$ 的 $S_f(v)$ ，本讲义称其为商集上的前缀和表（原稿称“块筛”）。不同 v 只有 $O(\sqrt{n})$ 个，但计算它们仍需要算法。

对非负项，若区间 $[x, 2x)$ 内各项同阶，可按 $[2^j, 2^{j+1})$ 倍增分块估计。例如

$$\sum_{i \leq n} i^{-\alpha} = \begin{cases} \Theta(n^{1-\alpha}), & 0 \leq \alpha < 1, \\ \Theta(\log(n+1)), & \alpha = 1, \\ \Theta(1), & \alpha > 1. \end{cases}$$

倍增估计是什么意思

把求和项按商相同分成块：在 $[l, r]$ 内 $\lfloor n/i \rfloor$ 都一样，于是整块可用块长 $\times$ 值估计。写成 O 或 Θ ，不要写成逐步等号——这一步只保证同阶，不保证精确。

证明应写成估计而不是等式。 $0 \leq \alpha < 1$ 时，第 j 块有 $\Theta(2^j)$ 项，每项为 $\Theta(2^{-\alpha j})$ ，故总和为 $\Theta(\sum_j 2^{(1-\alpha)j}) = \Theta(n^{1-\alpha})$ 。把一整块的所有函数值替换为端点，只是常数倍估计，不能像原稿一样连写精确等号。

算法选择问题。 知道 $f(p^k)$ 并不自动意味着能亚线性求和。还需要至少一种结构：容易求和的卷积关系、与另一函数在素数处相同、或者素数处的值能由可筛的完全积性函数组合。

9.2 用卷积把求和变成递归

若 $f = g * h$ ，则

$$S_f(n) = \sum_{ab \leq n} g(a)h(b) = \sum_{a=1}^n g(a)S_h(\lfloor n/a \rfloor).$$

选 g 的原则

要的是 $f * g = h$, 其中 h 的前缀和比 S_f 好算, 且 g 的前缀和也好算 (通常挑 $g = \mathbf{1}$ 、 $g = \mu$ 、 $g = \text{id}_t$ 之一, 或它们的组合)。选定后写出 $S_f(n) = \sum_d h(d) - \sum_{l \geq 2} (\cdots) S_f(\lfloor n/l \rfloor)$, 剩下的就是复杂度分析。

如果能常数时间读取 S_g, S_h 在商集所需位置的值, 整除分块可在 $O(\sqrt{n})$ 求出上式。反过来, 若 $f * g = h$ 且 $g(1) \neq 0$, 则

$$S_f(n) = \frac{S_h(n) - \sum_{j=2}^n g(j) S_f(\lfloor n/j \rfloor)}{g(1)}.$$

在模数环中需要 $g(1)$ 可逆; 积性函数的 $g(1) = 1$, 分母可省。

杜教筛的关键不是“套递归”。它把未知的 $S_f(n)$ 从卷积前缀里单独抽出来, 剩下的递归参数 $\lfloor n/j \rfloor$ 都小于 n , 所以可以自顶向下记忆化。每个商块 $[l, r]$ 的贡献为

$$(S_g(r) - S_g(l-1)) S_f(\lfloor n/l \rfloor).$$

记忆化为什么够用

递归的自变量只有 $\lfloor n/j \rfloor$ 这一族值, 而不同的商最多 $2\sqrt{n}$ 个 (小于 $\sqrt{n}$ 的商有 $\sqrt{n}$ 个, 大于的对应 $j \leq \sqrt{n}$ 也只有 $\sqrt{n}$ 个)。所以**状态数**不是猜的, 是可以数出来的——这就是杜教筛复杂度分析的核心一行。

其中 S_g, S_h 必须容易计算; 随便找一个卷积恒等式并不一定有用。方法本身不要求 f 积性, 积性只常用于构造关系和快速预处理。

为何状态仍在原商集中?

$$\left\lfloor \frac{\lfloor n/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor.$$

对子问题的子问题继续整除, 仍落在原商集中, 因此记忆化不会生成所有整数 $1, \dots, n$ 。

9.2.1 无预处理与带预处理的复杂度

若每个规模 v 的子问题花费 $O(\sqrt{v})$, 则总成本可估计为

$$\sum_{v \leq \sqrt{n}} O(\sqrt{v}) + \sum_{i \leq \sqrt{n}} O(\sqrt{n/i}) = O(n^{3/4}).$$

这不是 $O(\sqrt{n})$: 状态数为 $O(\sqrt{n})$ 不等于总工作量。若线性筛预处理到 $B \geq \sqrt{n}$, 大状态至多为 n/i ($i \leq n/B$), 所以

$$O\left(B + \sum_{i \leq n/B} \sqrt{n/i}\right) = O(B + n/\sqrt{B}).$$

提醒: 状态数 $\neq$ 工作量

状态数只告诉你**有多少个格子要填**, 不告诉你**填一格要多久**。这里一格要 $O(\sqrt{v})$, 于是总时间是 $\sum_v O(\sqrt{v}) = O(n^{3/4})$ 。凡是递归带记忆化的复杂度, 都按这个格式老老实实求和。

取 $B \asymp n^{2/3}$ 得 $O(n^{2/3})$ 。空间是 $O(B + n/B)$, 其中后者为记忆化表的量级。对 $v \leq B$ 直接返回已存的 $S_f(v)$, 绝不是返回 $\sqrt{v}$ 。

P4213：杜教筛

给定 $n \leq 10^9$ ，求 $S_\mu(n)$ 和 $S_\varphi(n)$ 。

利用 $\mu * 1 = \varepsilon, \varphi * 1 = \text{id}$ ，得到

$$S_\mu(n) = 1 - \sum_{j=2}^n S_\mu(\lfloor n/j \rfloor),$$

≍ 与平衡

$A \asymp B$ 表示两者同阶（差常数倍，忽略对数）。做法同上：总代价 $B + n/\sqrt{B}$ ，让两项同阶解出 $B = n^{2/3}$ ，再代回得 $O(n^{2/3})$ 。这里还要额外报空间 $O(B + n/B)$ ，因为记忆化表是真会占内存的。

$$S_\varphi(n) = \frac{n(n+1)}{2} - \sum_{j=2}^n S_\varphi(\lfloor n/j \rfloor).$$

讲解：先看这题到底在做什么

杜教筛的核心不是“记一个公式”，而是让原本需要扫到 n 的卷积关系重新排列。选择一个可以快速求前缀和的 g ，使 $f * g$ 的前缀值容易计算，然后把大的 $S_f(n)$ 表达成若干个更小的 $S_f(\lfloor n/k \rfloor)$ ，再用记忆化递归。

解析. 两式中 $g(j) = 1$ ，块系数直接为块长。先线性筛出 μ, φ 和普通前缀，再分别记忆化大状态即可。

```
// S is either the Mertens prefix or the phi prefix.
// H(n) is 1 for Mertens; n*(n+1)/2 for phi.
i128 solve(long long n) {
    if (n<=B) return prefix[n];
    auto it=memo.find(n);
    if (it!=memo.end()) return it->second;
    i128 ans=H(n);
    for (long long l=2,r;l<=n;l=r+1) {
        long long q=n/l;
        r=n/q;
        ans-=(i128)(r-l+1)*solve(q);
    }
    return memo[n]=ans;
}
```

S_μ 可能为零或负数，所以不能用 `if (memo[n])` 判断是否已计算；也不能把零当作“未访问”。

手算 $n = 10$. $\varphi(1), \dots, \varphi(10)$ 为 1, 1, 2, 2, 4, 2, 6, 4, 6, 4，前缀为 32。递归式给出

$$55 - S_\varphi(5) - S_\varphi(3) - 2S_\varphi(2) - 5S_\varphi(1) = 55 - 10 - 4 - 4 - 5 = 32.$$

$\mu(1), \dots, \mu(10) = 1, -1, -1, 0, -1, 1, -1, 0, 0, 1$ ，前缀为 -1 。

复杂度与实现. 多询问可按最大 n 预处理并复用表，但不能简单将单次复杂度当成所有任意询问的总复杂度；记忆化表的状态并集仍需分析。

题目 / 延伸资料: <https://www.luogu.com.cn/problem/P4213>

9.3 Powerful Number: 稀疏的高次素因子结构

定义 9.1. 正整数 x 是 Powerful Number (PN), 若每个整除 x 的素数 p 都满足 $p^2 \mid x$ 。1 也属于 PN。

每个 PN 可唯一写为 $x = a^2b^3$, 其中 b 为平方自由数: 对每个指数, 偶数写成 $2u$, 奇数 (至少为 3) 写成 $2u+3$ 。因此

$$\#\{x \leq n : x \in \text{PN}\} \leq \sum_{b \leq n^{1/3}} \sqrt{n/b^3} = O(\sqrt{n}).$$

词义: Powerful Number

Powerful Number = 每个素因子指数都 ≥ 2 的数, 等价于形如 a^2b^3 。 $\leq n$ 的个数约 $\zeta(3/2)$ 级、约 $O(\sqrt{n})$: 因为平方部分至多 $\sqrt{n}$ 种, 剩下每个 a 的立方部分至多 $\sqrt{n}$ 种, 相乘即 $O(\sqrt{n})$ 。
支撑稀疏就是靠这个数出来的。

若不要求 b 平方自由, 则表示不唯一, 但仍可以用于上界计数。

哪些数属于 PN? 1, 4, 8, 9, 16, 25, 27, 32, 36 属于 PN; $12 = 2^2 \cdot 3$ 、 $18 = 2 \cdot 3^2$ 不属于。定义要求**每一个**出现的素因子指数都至少为二, 不是“存在平方因子”。

枚举而不重复. 递归状态记录当前乘积与最后使用的素数下标, 后续只加更大的素数, 每个新素数指数至少为二。用 $p \leq \sqrt{n/\text{cur}}$ 判断是否能扩展, 不先计算可能溢出的 $\text{cur} \cdot p^2$ 。每个 PN 的质因数分解唯一, 因此恰访问一次; 递归同时携带函数值。

9.4 把差异留给 PN 支撑

定理 9.2 (PN 求和转化). 设 f, g 积性, 且对所有素数 p 有 $f(p) = g(p)$ 。令 $f = g * h$, 则 $h(p) = 0$, 从而 h 的非零支撑只可能落在 PN 上。于是

$$S_f(n) = \sum_{\substack{d \leq n \\ d \in \text{PN}}} h(d) S_g(\lfloor n/d \rfloor).$$

怎么做: 差值为何只落在 PN 上

套路: 构造 g 使 $\hat{g}(p) = \hat{f}(p)$ (素数处一致), 于是 $h = f * g^{-1}$ 在每个素数处为 0, 由积性, h 在含单次素因子的数上也全为 0。差值只能落在**所有素因子至少二次**的数上——那里的数很少, 枚举得起。

证明. 先把**支撑**讲明白: 说某个函数 h 的支撑是集合 A , 意思就是 $h(d)$ **只在** $d \in A$ **时可能非零**。这里要证的是 h 只在**完全平方数**上非零。

第一步 (素数处)。 $f = g * h$ 在 $n = p$ 处展开只有两项 (p 的约数是 1 与 p):

$$f(p) = g(p)h(1) + g(1)h(p) = g(p) + h(p),$$

用了 g, h 积性所必需的 $g(1) = h(1) = 1$ (非零积性函数在 1 处取值 1: $g(1) = g(1 \cdot 1) = g(1)^2$, 且 g 不恒为零)。所以 $h(p) = f(p) - g(p)$ 被两边在素数处的差唯一决定——这正是**把差异全部交给** h 的做法。

第二步（含单次素因子的数不贡献）。设 d 有某个素因子 p 的指数恰为 1，写 $d = pe$ 、 $p \nmid e$ 。积性给出

$$h(d) = h(p)h(e) = (f(p) - g(p))h(e).$$

若我们构造 g 使得 $g(p) = f(p)$ （例如让 $\hat{g}(p) = \hat{f}(p)$ 在生成函数层面一致），则 $h(p) = 0$ ，从而 $h(d) = 0$ 。也就是说： h 在一个某个素因子只出现一次的数上必为零，于是非零只可能出现在每个素因子指数都 ≥ 2 的数上，即完全平方数（更严格地说，是 Powerful Number 集合）。

第三步（换序得到公式）。 $f = g * h$ 即 $f(n) = \sum_{d|n} g(n/d)h(d)$ ，把求和限制在支撑上：

$$f(n) = \sum_{\substack{d|n \\ d \text{ 为平方数}}} g(n/d)h(d).$$

要求 $f(n)$ 就只需要枚举 n 的平方数约数 d （个数很少，见下一节的计数），并且 $h(d)$ 已经由素数处的差递推出来。□

固定素数 p ，局部系数由

$$h(p^e) = f(p^e) - \sum_{j=1}^e g(p^j)h(p^{e-j}), \quad h(1) = 1$$

递推。这里不能把 $h(p^e)$ 写成 $f(p^e) - g(p^e)$ ，因为还有交叉项。

复杂度与实现. 若 S_g 的商集前缀已备好、局部系数已预处理且可常数时间读取，则 PN 枚举阶段为 $O(\sqrt{n})$ ；总时间还必须加上 g 的前缀预处理与局部系数构造。不能不加前提就宣布任意这样的函数都有 $O(\sqrt{n})$ 总算法。

入门例：平方自由数的个数

求 $\sum_{i \leq n} \mu(i)^2$ 。

讲解：先看这题到底在做什么

先把“平方自由”翻译成“没有任何平方因子”。莫比乌斯函数正好提供了一个按平方因子做容斥的系数，因此最后只需要枚举 $d^2 \leq n$ 。这个例子非常适合先讲 PN 的支撑为什么会变稀疏。

解析. 取 $g = 1$ 。局部级数 $(1+z)/(1-z)^{-1} = 1 - z^2$ ，所以 $h(d^2) = \mu(d)$ ，其他位置为零，得到

$$\sum_{i \leq n} \mu(i)^2 = \sum_{d \leq \sqrt{n}} \mu(d) \lfloor n/d^2 \rfloor.$$

筛到 $\sqrt{n}$ 后直接求和。 $n = 10$ 时 $10 - 2 - 1 = 7$ ，排除的是 4, 8, 9。

LOJ 6053：异或定义的积性函数

积性函数满足 $f(p^k) = p \oplus k$ ， $\oplus$ 表示按位异或。求 $S_f(n)$ ， $n \leq 10^{10}$ 。

解题方向. 奇素数处 $f(p) = p - 1 = \varphi(p)$ ；素数 2 是唯一例外。拆成 $f = \varphi * h$ 后枚举奇数 PN 与二的幂。

讲解：先看这题到底在做什么

先看奇素数：因为奇数的最低位是 1，所以 $p \oplus 1 = p - 1 = \varphi(p)$ ，于是卷积商在奇素数处消失。唯一特殊的是 2，所以 PN 枚举只需要在“奇数部分 + 二的幂”这个结构上展开。

解析. 当 $p > 2$, 最低二进制位为一, 所以 $p \oplus 1 = p - 1$ 。因此 $h(p) = 0$ (奇素数)。在 $p = 2$ 处 $f(2) = 3$, $\varphi(2) = 1$, 故 $h(2) = 2$, 不能把 2 也当成普通 PN 因子。 h 有值的位置形如 $2^e u$, 其中 u 为奇数 PN。它们的总数量仍不超过

$$\sum_{e \geq 0} O\left(\sqrt{n/2^e}\right) = O(\sqrt{n}).$$

用杜教筛求 S_φ , 再按上述支撑枚举。作为局部计算示例,

$$h(3) = 0, \quad h(3^2) = f(3^2) - \varphi(3^2) - \varphi(3)h(3) = 1 - 6 = -5.$$

卷积商可能出现负数, 模运算中应及时规范。素数幂 p^k 的点值是 $p \oplus k$, 不是 $(p^k) \oplus k$ 。

题目 / 延伸资料: <https://loj.ac/p/6053>

第 10 章

洲阁筛与素数贡献的分层统计

章节导读

上一章我们处理的是一般积性函数的前缀和；进一步地，如果最重要的贡献集中在素数及其幂上，就可以把“素数部分”和“合数部分”分层处理。这一章沿着这个方向建立洲阁筛的状态，并特别说明哪些完全积性条件是这种拆分能够成立的基础。

10.1 先把适用条件说清楚

本章讨论原稿“洲阁筛”所用的两阶段思路：先在商集上筛出素数位置的函数和，再按最小素因子补入合数。它与常见 Min_25 类筛法的若干实现共享同一核心结构；不同资料的命名、状态和具体复杂度可能不同，不能光看名字就混用模板。

适用于以下条件：

1. f 积性，素数幂 $f(p^e)$ 可高效计算；
2. $f(p)$ 可表示成常数个**完全积性函数**在素数处取值的线性组合；
3. 这些完全积性函数的普通前缀和可快速计算。

最常见的是 $f(p)$ 为关于 p 的低次多项式，基函数为 $1, p, p^2, \dots$ 。

P5325: Min_25 筛模板所对应的积性函数

$f(1) = 1$, $f(p^e) = p^e(p^e - 1)$, 求 $\sum_{i \leq n} f(i)$, $n \leq 10^{10}$, 答案按题目要求取模。

解题方向. 素数处 $f(p) = p^2 - p$, 先分别求素数的一次幂和与平方和。不要误认为所有 n 上都有 $f(n) = n(n-1)$ 。

10.2 第一阶段：把合数从简单前缀和中筛掉

令 $p_1, p_2, \dots, p_s$ 为 $\sqrt{n}$ 以内的素数。对 $t = 0, 1, 2$ 等所需次数，令

$$G_t(v, j) = \sum_{2 \leq x \leq v} [x \text{ 为素数或 } \min p(x) > p_j] x^t.$$

前提：洲阁筛的要求与它的输出

洲阁筛 (Min_25) 要求 f 是积性函数、 $f(p)$ 是关于 p 的多项式（且能在素数幂上递推）。它的输出是 $\sum_{i \leq n} f(i)$ ，不是 $\sum_{p \leq n} f(p)$ ；后者只是它的中间量 G 。

两阶段各自在做什么

第一阶段：从所有整数的 $\sum p^t$ 出发，一点点扣掉合数贡献，得到 $G_t(v, j) = \text{最小素因子} > p_j \text{ 的数 (含素数与 1) 的 } p^t \text{ 和}$ 。第二阶段：按最小素因子枚举，把合数一块一块补回来，此时才用到 f 的真实值。第一阶段只依赖完全积性，所以能提前算。

初始 $j = 0$ 时没有筛掉任何 $x \geq 2$ ，所以

$$G_0(v, 0) = v - 1, \quad G_1(v, 0) = \frac{v(v+1)}{2} - 1,$$

$$G_2(v, 0) = \frac{v(v+1)(2v+1)}{6} - 1.$$

记 $P_t(j) = \sum_{i=1}^j p_i^t$ 。当 $p_j^2 \leq v$ 时，

$$G_t(v, j) = G_t(v, j-1) - p_j^t (G_t(\lfloor v/p_j \rfloor, j-1) - P_t(j-1)).$$

若 $p_j^2 > v$ ，不需要转移。最后 $G_t(v, s)$ 只剩素数贡献。

为什么恰好扣掉最小素因子为 p_j 的合数？。这样的合数唯一写成 $x = p_j y$ ，其中 $y \geq p_j$ 且 y 没有小于 p_j 的素因子。 $G_t(\lfloor v/p_j \rfloor, j-1)$ 已保留这些 y ，但还含小于 p_j 的素数，因此减去 $P_t(j-1)$ 。函数 x^t 完全积性，故 $(p_j y)^t = p_j^t y^t$ ，可提出系数。该步允许 $p_j \mid y$ ，所以普通积性不够用。□

例子。 $v = 10, t = 1$ ，初始 $G_1 = 54$ 。用 2 筛时扣掉 $2(2+3+4+5) = 28$ ，剩 26，对应 2, 3, 5, 7, 9；用 3 筛时再扣 9，剩素数和 17。

10.2.1 只保留商集与滚动数组

需要的 v 只有 $\mathcal{V} = \{\lfloor n/k \rfloor : k \geq 1\}$ ，因为每次下标都变为 $\lfloor v/p \rfloor$ ，仍在原商集中。可对小值用编号 v 、大值用编号 n/v 映射，或先离散化；不要对 10^{10} 开数组。外层按素数递增，内层 v 必须**递减**，这样源状态 $\lfloor v/p \rfloor < v$ 仍是处理本素数之前的值。若递增原地更新，会读到已经扣过同一素数的状态。

```
for (int j=0; j<(int)primes.size(); ++j) {
    long long p=primes[j];
    for (long long v: values_descending) {
        if (p>v/p) break;
        G[id(v)] -= weight(p) * (G[id(v/p)] - prime_prefix[j]);
        // prime_prefix[j] contains primes strictly before p.
    }
}
```

初始和式除以 2, 6 时，如果工作模数下分母不可逆，应先在整数域精确约分，再取模；本题常见素数模数下可使用逆元，但仍要写清条件。

10.3 第二阶段：用素数幂重建一般积性函数

记 $P_f(x) = \sum_{p \leq x} f(p)$ ，它已由第一阶段的线性组合得到。定义

$$H(v, j) = \sum_{2 \leq x \leq v} [x \text{ 为素数或 } \min p(x) > p_j] f(x).$$

起点为 $H(v, s) = P_f(v)$, 按 $j = s, s-1, \dots, 1$ 反向加入最小素因子为 p_j 的合数。对 $p_j^2 \leq v$,

$$H(v, j-1) = H(v, j) + \sum_{\substack{e \geq 1 \\ p_j^e \leq v}} f(p_j^e) \left([e \geq 2] + H(\lfloor v/p_j^e \rfloor, j) - P_f(\min(p_j, \lfloor v/p_j^e \rfloor)) \right).$$

最终答案为 $1 + H(n, 0)$, 其中 $1 = f(1)$ 。

把公式拆成三类对象。 每个待补合数唯一写作 $p_j^e u$, $p_j \nmid u$:

- $u = 1$ 时必须 $e \geq 2$, 否则是已经存在的素数 p_j ; 这给出 $[e \geq 2]$ 。
- $u > 1$ 时要求 $\min p(u) > p_j$; H 中仍包含所有素数, 所以要减去不超过 p_j 的素数贡献。
- p_j^e 与 u 互质, 故普通积性足以提出 $f(p_j^e)$ 。这里和第一阶段使用完全积性的理由不同。

大于 v 的素数幂不能进入求和, 否则常数项 $[e \geq 2]$ 会产生不存在的数。

原地滚动时, 素数要按递减处理, 而 v 仍按**递减**处理, 以保证读到的是本轮之前的 $H(\lfloor v/p^e \rfloor, j)$ 。两次筛的素数顺序相反, 商值顺序却都相同。

一个覆盖所有类型的核对。 原题 $n = 10$ 时,

$$(f(1), \dots, f(10)) = (1, 2, 6, 12, 20, 12, 42, 56, 72, 40),$$

总和为 263。例如 $f(6) = f(2)f(3) = 12$, 而不是 $6 \cdot 5 = 30$; $f(4) = 12$, 而不是 $f(2)^2 = 4$ 。前者提醒“公式只给素数幂”, 后者提醒“不是完全积性”。

10.3.1 复杂度说明与备课时的取舍

每个商值 v 的素数转移数大致为 $\pi(\sqrt{v})$ 。把所有商值相加, 可先得到保守的 $O(n^{3/4})$ 数量级; 再利用素数计数上界并精细求和, 可得常见的

$$O(n^{3/4}/\log n)$$

量级 (默认固定次数的基函数、素数幂点值可常数时间读取、滚动表常数时间访问)。第二阶段的额外幂次枚举可按

$$\sum_{p \leq \sqrt{v}} \lfloor \log_p v \rfloor = 2\pi(\sqrt{v}) + \sum_{e \geq 3} \pi(v^{1/e})$$

计数, 不会改变主导量级。小 n 的对数表达应理解为渐近说明。空间为常数张 $O(\sqrt{n})$ 的商集表和素数前缀表。若改成递归搜索、使用平衡树映射、每次转移重新求 $f(p^e)$, 实际复杂度需要重新计入这些成本。

不要在第一次课就背完整转移

先用 $t = 0$ 实现素数个数筛; 再用 $t = 1$ 算素数和; 最后才加入一般积性函数的第二阶段。要求学生分别解释“减去较小素数前缀”和“补一个纯素数幂”两个容易漏掉的项。

题目 / 延伸资料: <https://www.luogu.com.cn/problem/P5325>

第 11 章

二维积性函数：消去主要贡献以得到稀疏支撑

章节导读

前面的筛法大多只处理一个整数；这一章把对象提升到两个坐标。二维卷积看起来更复杂，但核心仍然是“先匹配主要贡献，再让剩余部分变得稀疏”，因此我们会先消掉坐标轴，再处理对角线，最终把庞大的状态空间压缩成可枚举的支撑。

11.1 问题、定义与二维卷积

UOJ 885：红场阅兵

已知积性函数 S ，满足 $S(1) = 1$ ，对素数 p 与 $k \geq 1$ ，

$$S(p^k) = w_0 + w_1 p^k + w_2 p^{2k}.$$

求 $\sum_{i=1}^n \sum_{j=1}^n S(ij) \bmod 998244353$ ， $n \leq 10^9$ 。

词义：二维积性

二维 = 自变量是数对 (x, y) ，且固定一个变量后对另一个是积性的。处理方式与一维一样：先把**对角线/坐标轴**上的一维积性部分消掉，剩下的非零项只出现在稀疏的坐标对上。

定义 11.1. 二维函数 f 满足 $f(1, 1) = 1$ ，且 $\gcd(ab, cd) = 1$ 时有

$$f(ac, bd) = f(a, b)f(c, d),$$

则称 f 为二维积性函数。等价地，若 $x = \prod p^{\alpha_p}, y = \prod p^{\beta_p}$ ，则

$$f(x, y) = \prod_p f(p^{\alpha_p}, p^{\beta_p}).$$

二维卷积定义为

$$(f * g)(x, y) = \sum_{u|x} \sum_{v|y} f(u, v)g(x/u, y/v).$$

常数项为可逆元素时可以逐项求卷积逆。二维积性函数的卷积、卷积商仍为二维积性函数；固定素数后对应二元形式幂级数乘除。

从一维过渡时必须补上的条件

两个坐标不是分别要求 $\gcd(a, c) = \gcd(b, d) = 1$ ，而是要求第一对坐标所含的所有素因子与第二对坐标所含的所有素因子完全分离，即 $\gcd(ab, cd) = 1$ 。否则可能把同一个素数拆成两个独立局部因子，积性分解就不成立。此外，素数幂公式只用于 $k \geq 1$ 。 $S(p^0) = S(1) = 1$ ，不是 $w_0 + w_1 + w_2$ 。

11.2 算法一：先消去坐标轴

令 $f(x, y) = S(xy)$ 、 $g(x, y) = S(x)S(y)$ ，定义 $f = g * h$ 。 g 的二维前缀易算：

$$S_g(A, B) = S_S(A)S_S(B), \quad S_S(t) = \sum_{i \leq t} S(i).$$

由于 $f(p^k, 1) = g(p^k, 1)$ 以及另一坐标轴上的对应等式，

$$h(p^k, 1) = h(1, p^k) = 0 \quad (k \geq 1).$$

因此只有 x, y 含相同素因子集合时 $h(x, y)$ 才可能非零，答案为

$$\sum_{x \leq n} \sum_{y \leq n} h(x, y) S_S(\lfloor n/x \rfloor) S_S(\lfloor n/y \rfloor).$$

直接按共同素因子递归枚举这些位置，不必遍历 n^2 个点。

11.2.1 局部系数怎样实际求？

固定素数 p ，令 $s_0 = 1, s_k = S(p^k)$ ($k \geq 1$)， $h_{a,b} = h(p^a, p^b)$ ，则

$$h_{0,0} = 1, \quad h_{a,b} = s_{a+b} - \sum_{\substack{0 \leq i \leq a, 0 \leq j \leq b \\ i+j > 0}} s_i s_j h_{a-i, b-j}.$$

按 $a+b$ 递增计算。例：

$$h_{1,0} = h_{0,1} = 0, \quad h_{1,1} = S(p^2) - S(p)^2.$$

局部二维多项式除法的成本应单独预处理或缓存，不要在每个递归节点从头做一次。

11.2.2 为何只有 $O(n)$ 个位置？一个更直接的证明

将 $x, y \leq n$ 放宽为 $xy \leq n^2$ ，不是原稿误写的 $xy \leq n$ 。令 $a(t)$ 为满足 $xy = t$ 、 x, y 具有相同素因子集合的有序数对个数。它是积性的，且

$$a(p^k) = \begin{cases} 0, & k = 1, \\ k - 1, & k \geq 2. \end{cases}$$

$k - 1$ 来自正指数拆分 $k = 1 + (k - 1) = \dots = (k - 1) + 1$ 。

令 T_2 为完全平方数的示性函数，写 $a = T_2 * b$ 。局部生成函数相乘可得

$$b(p) = b(p^2) = 0, \quad b(p^k) = 2 \quad (k \geq 3).$$

于是

$$\sum_{t \leq X} a(t) = \sum_{d \leq X} b(d) \left\lfloor \sqrt{X/d} \right\rfloor \leq \sqrt{X} \sum_{d \geq 1} \frac{b(d)}{\sqrt{d}}.$$

右边的级数可分解成

$$\prod_p \left(1 + 2 \sum_{k \geq 3} p^{-k/2} \right).$$

因为每个局部因子为 $1 + O(p^{-3/2})$ ，且 $\sum_p p^{-3/2}$ 收敛，整个乘积有限。因此 $\sum_{t \leq X} a(t) = O(\sqrt{X})$ 。代入 $X = n^2$ ，便得到 $O(n)$ 的支撑上界。这个证明不需要“随便写出一个高次卷积支配函数后省略检查”。

复杂度边界。 算法一的枚举阶段为 $O(n)$ ，另加 S_S 的商集前缀与局部系数的预处理。适合中等规模或作为正确性基准， $n = 10^9$ 时需要进一步优化。

11.3 算法二：再消去局部对角线

定义一维积性函数 $D(t) = h(t, t)$ ，以及只在对角线上有值的二维函数

$$g_3(x, y) = [x = y]D(x).$$

令 $h = g_3 * h'$ 。逐素数在对角线上作一维除法可得

$$h'(p^a, p^b) = 0 \quad \text{当 } a = 0 < b, \ b = 0 < a, \text{ 或 } a = b > 0.$$

所以对 $h'(x, y) \neq 0$ 的位置，每个素数要么在两边都不出现，要么两边指数都为正且不相等。答案转为

$$\sum_{x, y \leq n} h'(x, y) L(\lfloor n/x \rfloor, \lfloor n/y \rfloor),$$

其中

$$L(A, B) = \sum_{d \leq \min(A, B)} D(d) S_S(\lfloor A/d \rfloor) S_S(\lfloor B/d \rfloor).$$

对两个商共同分块，在已知 S_D 前缀时，单次计算需要

$$O(\min(A, B, \sqrt{A} + \sqrt{B}))$$

次操作。

为什么前缀预处理仍可做？

$$D(p) = S(p^2) - S(p)^2 = w_0 + w_1 p^2 + w_2 p^4 - (w_0 + w_1 p + w_2 p^2)^2,$$

是关于 p 的四次多项式。因此 S, D 的素数值均可拆成常数个幂函数，能用上一章方法求商集前缀。 $D(p^k) = h(p^k, p^k)$ 由局部表计算。由于 $A = \lfloor n/x \rfloor$ ， $\lfloor A/d \rfloor = \lfloor n/(xd) \rfloor$ ，所有需要的 S_S 参数仍在原商集中；共同块的端点来自 A 或 B 的整除商， S_D 也只需原商集所对应的值。

11.3.1 稀疏支撑数为 $O((AB)^{1/3})$

令 $v(x, y)$ 表示上述“每个出现的素数指数均正且不同”的支撑，且 $v(1, 1) = 1$ 。定义积性 v_1 ：除 $(1, 1)$ 的局部常数项外，只在局部 $(p, p^2), (p^2, p)$ 处为 1；定义 v_2 在其他合法非零局部位置为 1，两处被删除的位置为 0，并令 $v_2(1, 1) = 1$ 。在非负系数意义下有 $v \leq v_1 * v_2$ 。

v_1 的位置可表示成 (a^2b, ab^2) 。实际还对 a, b 有平方自由及互质限制，去掉这些限制只会增加数量，所以

$$S_{v_1}(A, B) \leq \sum_a \min\left(\frac{A}{a^2}, \sqrt{\frac{B}{a}}\right).$$

两项在 $a_0 = A^{2/3}/B^{1/3}$ 附近相等；在此处分段求和，得到 $O((AB)^{1/3})$ 。若分点落在合法范围外，使用 $O(A)$ 或 $O(B)$ 的更粗界也满足结论。

v_2 的非零局部项满足两坐标指数之和至少为 4，故

$$\sum_{x, y \geq 1} \frac{v_2(x, y)}{(xy)^{1/3}} = \prod_p \left(1 + O(p^{-4/3})\right) < \infty.$$

展开卷积前缀并使用上界，得到

$$S_v(A, B) \leq \sum_{x, y} v_2(x, y) O\left((AB/(xy))^{1/3}\right) = O((AB)^{1/3}).$$

这说明剥掉最低总次数为二的对角项后，主要的规模指数从 $1/2$ 变成了 $1/3$ 。

11.3.2 将单次商块成本求和

使用不等式

$$\min(A, B, \sqrt{A} + \sqrt{B}) \leq \min(A, \sqrt{B}) + \min(B, \sqrt{A}).$$

两部分对称，只分析第一部分。把最小值写成层数计数，有

$$\begin{aligned} \sum_{x, y \leq n} v(x, y) \min\left(\frac{n}{x}, \sqrt{\frac{n}{y}}\right) &\ll \sum_{t \leq \sqrt{n}} S_v(\lfloor n/t \rfloor, \lfloor n/t^2 \rfloor) \\ &\ll \sum_{t \leq \sqrt{n}} \left(\frac{n^2}{t^3}\right)^{1/3} = O(n^{2/3} \log n). \end{aligned}$$

因此除前缀表及局部系数预处理以外，算法二耗时 $O(n^{2/3} \log n)$ 。如果前缀用洲阁筛计算，总复杂度还要加对应的预处理项，不能一笔抹去。

11.4 推广：第一个非零局部系数决定主导规模

设非负整数序列 a_k 为常数次多项式增长， $a_0 = 1$ ，令 r 是第一个满足 $a_r > 0$ 的正下标。积性函数 F 若满足 $F(p^k) = a_k$ ，则常见的上界为

$$S_F(n) = O\left(n^{1/r} (\log(n+1))^{a_r-1}\right),$$

怎么做：先代一个小例子

r 是第一个 $a_r > 0$ 的下标的意思是：局部生成函数 $a_0 + a_1 z + \cdots$ 前面连续有 r 个零。这些零把每一项能出现的素因子次数抬高，于是和式的主导规模从 n 变成 $n^{1/r}$ 。先代 $a_0 = 1, a_1 = 0, a_2 = 1$ ($r = 2$) 算一个具体例子，再读一般式子。

其中 r, a_r 视为常数。这里给出的是在上述归一化、非负整数系数与多项式增长条件下的结论，不能无条件推广到任意系数列。

证明思路。 定义 $T_r(n) = [n \text{ 是完全 } r \text{ 次方}]$ 。从局部级数中提出 $(1 - z^r)^{-a_r}$ ，相当于 $F = T_r^{*a_r} * H$ 。剩余局部级数从 z^{r+1} 才开始，因此

$$\sum_{d \geq 1} |H(d)| d^{-1/r} < \infty.$$

而 $T_r^{*a_r}$ 的前缀和等于 a_r 重约数函数在 $n^{1/r}$ 的前缀和，为 $O(n^{1/r} \log^{a_r-1}(n+1))$ ；卷积上界便给出结论。例如 $\tau(n)^2$ 的局部值为 $(k+1)^2$ ， $r = 1, a_1 = 4$ ，得到 $O(n \log^3 n)$ 。若 a_r 非整数或取负值，需要另行使用分析数论工具，不能直接用“做 a_r 次卷积”的证明。

11.5 算法三：去掉枚举阶段的对数（研究拓展）

原稿最后提到可继续优化。本讲义保留这一方向，但把所需的新结论单独列出来，不将它冒充前两种算法的自然常数优化。如果能在合适预处理后，把 $L(A, B)$ 的一次询问做到 $O(\min(A, B)^\rho)$ ，其中固定 $\rho < 2/3$ ，那么稀疏枚举阶段就能降到 $O(n^{2/3})$ 。

为什么是阈值 $2/3$ ？ 令 $w(t) = t^\rho - (t-1)^\rho = O(t^{\rho-1})$ 。总询问成本不超过常数倍

$$\sum_{t \geq 1} w(t) S_v(\lfloor n/t \rfloor, \lfloor n/t \rfloor) \ll n^{2/3} \sum_{t \geq 1} t^{\rho-1-2/3}.$$

只有 $\rho < 2/3$ 时该级数收敛；例如取 $\rho = 3/5$ 即可。

新的难点是什么？ 需要批量维护所有商值 A 对应的加权前缀 $\sum_{d \leq t} D(d) S_S(\lfloor A/d \rfloor)$ ，而不是每个 (A, B) 都独立重做分块。对大 A 可单独预处理商块；对小 A 需要利用相邻商值的 $\lfloor A/i \rfloor$ 数组变化总量，维护小下标项及按商聚合项的两套前缀。这涉及额外的数据结构与摊还证明。本节给出目标和总复杂度归约，完整高性能实现及变化量分析参见下列延伸资料；课堂上可将证明 $\sum w(t) S_v(n/t, n/t)$ 作为专题作业。采用哪种前缀预处理也影响完整算法的时间界。

题目 / 延伸资料：<https://uoj.ac/problem/885>

题目 / 延伸资料：<https://www.cnblogs.com/zkyJuruo/p/18204106>

第 12 章

综合专题：循环、指数向量、图与期望

章节导读

前面几章已经把同余、阶、函数与筛法中的工具分别建立起来；现在把这些工具放回真正复杂的综合题里。这里不再只有一个公式，而是会同时出现循环结构、指数向量、图上的状态传播以及概率计数，需要把前面学到的结构识别能力组合起来。

12.1 CF516E：快乐传播不是只保留最小起点

Drazil and His Happy Friends

n 个男生、 m 个女生分别编号从 0 开始。第 t 天男生 $t \bmod n$ 与女生 $t \bmod m$ 见面；只要有一人快乐，另一人也会变快乐，快乐不可逆。给定初始快乐的男生、女生集合，求所有人都快乐的最早日期，或报告不可能。 $n, m \leq 10^9$ ，初始快乐人数总计不超过 2×10^5 ，保证至少有一人初始不快乐。

解题方向. 用 $d = \gcd(n, m)$ 分成独立传播分量；在每一侧转成步长环上的带不同初始时间的多源最短路，利用环坐标排序与前后缀最小值求最晚到达时间。

12.1.1 传播时间的精确形式

将两侧初始快乐的编号合并为多重集合 Z 。能发生有效传播的时刻恰是

$$t = z + vn + wm, \quad z \in Z, \quad v, w \geq 0.$$

必要性沿传播链归纳：男生再次见面时间增加 n 的倍数，女生增加 m 的倍数。充分性：若 z 来自男生，在 $z + vn$ 时该男生使对应女生快乐，再在 $z + vn + wm$ 时由该女生传播；若 z 来自女生，交换顺序即可。

对某个初始不快乐的男生， vn 不改变其编号且只会推迟时间，因此最早时间可写为

$$F(i) = \min_{\substack{z \in Z, w \geq 0 \\ z + wm \equiv i \pmod{n}}} (z + wm).$$

女生同理交换 n, m 。初始快乐的人无需等到第一次见面，应从“最晚变快乐”统计中排除。

12.1.2 修正原稿的关键反例

取 $n = 5, m = 3$ ，初始快乐男生为 $\{4\}$ ，女生为 $\{0\}$ 。同一个环中最小源为 $z = 0$ ，它单独向男生 2 传播需要时间 12；而源 $z = 4$ 在时间 7 就能到达男生 2。实际全员快乐的日期为 7。因此“同一子环上所有初始不快乐的点都由最小 z 最先到达”是错误的。小编号源可能离目标更远，必须保留多源竞争。

12.1.3 把步长变成加一

对男生侧，令 $d = \gcd(n, m)$, $L = n/d$ 。每个余数 $r \pmod{d}$ 是独立环。 $L > 1$ 时取

$$u = (m/d)^{-1} \pmod{L}.$$

编号为 $i \equiv r \pmod{d}$ 的点映射到环坐标

$$t_i = ((i - r)/d)u \bmod L.$$

每走一步 $+m$ ，新坐标恰加一。 $L = 1$ 时只有坐标零，无需求逆。源 z 的坐标用 $z \bmod n$ 或直接用上述同余式计算，初始代价仍是**整数** z ，而不是 $z \bmod n$ 。若某个余数类没有源，两侧对应的人都永远无法快乐。无需开 d 大小数组：哈希分组后，检查出现的余数类个数是否为 d 。

12.1.4 环上的带权多源扫线

一个环上把不同源坐标排序为 $t_1 < \dots < t_s$ ，同坐标只保留最小初始代价 z_i 。设

$$c_i = z_i - mt_i.$$

对于目标坐标 t ，最早时间为

$$F(t) = mt + \min \left(\min_{t_i \leq t} c_i, \quad mL + \min_{t_i > t} c_i \right).$$

第一项表示从左侧源不绕环到达，第二项表示从右侧源先绕一整环。预处理 c_i 的前缀最小值和后缀最小值，每个源之间的区间都能 $O(1)$ 确定常数项；区间内斜率为正数 m ，最晚时间在其最右端。

按区间 $[0, t_1 - 1]$ 、 $[t_i, t_{i+1} - 1]$ 、 $[t_s, L - 1]$ 检查右端点，并排除初始快乐男生。由于每个初始快乐男生本身就是源，区间内部没有需要额外跳过的初始快乐点；只有长度为一、右端点就是该初始快乐源的区间需要跳过。女生侧对称处理，取两侧初始不快乐者时间的最大值。

复杂度与实现. 令初始快乐总人数为 K 。使用排序，确定性排序部分为 $O(K \log K)$ ，gcd 与逆元为 $O(\log \max(n, m))$ ，额外空间 $O(K)$ 。分组可用哈希，或者按余数排序获得不依赖哈希的实现。不沿用原稿未经成立支配结论支撑的 $O(K)$ 宣称。

备课验证

先让学生手工模拟 $n = 5, m = 3$ 的反例，再画出步长坐标；最后才给前缀、后缀公式。随附的小规模对拍脚本将该公式与逐日模拟比较，覆盖 $1 \leq n, m \leq 12$ 的随机初始集合。

题目 / 延伸资料: <https://codeforces.com/problemset/problem/516/E>

12.2 CF571E: 把等比数列交集转到指数空间

Geometric Progressions

给定 n 个等比数列 $a_i, a_i b_i, a_i b_i^2, \dots$ ，求最小公共正整数，答案模 $10^9 + 7$ ；无公共项则报告无解。 $n \leq 100, a_i, b_i \leq 10^9$ 。原稿还提出 $n \leq 10^5, a_i, b_i \leq 10^{18}$ 的 Bonus，作为研究拓展保留。

解题方向. 分解质因数后，一个等比数列变为非负整数参数的一条指数向量射线。合并两条射线时可能无交、一个点或一条新射线；不同质因子的指数必须共享同一个参数。

12.2.1 为什么不能对每个素数独立 CRT ?

设素数全集为 $p_1, \dots, p_s$, 把 a, b 分别写成指数向量 A, B 。数列为

$$A + kB, \quad k \in \mathbb{Z}_{\geq 0}.$$

所有坐标里的 k 是同一个数。若分别为每个素数求一个可行指数, 却没有检查它们对应相同的 k , 会生成并不存在于数列中的整数。

12.2.2 两条射线的完整分类

合并 $A + kB$ 与 $C + \ell D$, 需要解所有坐标上的

$$B_j k - D_j \ell = C_j - A_j, \quad k, \ell \geq 0.$$

1. **零方向.** $B = 0$ 或 $D = 0$ 表示原数列公比为一, 是单点。直接检查这个点是否能用另一条射线的同一个非负整数参数表示。
2. **方向不成比例.** 找到两个坐标使对应 2×2 系数矩阵行列式非零。用精确整数消元求出唯一有理数解 (k, ℓ) ; 检查分子能否被分母整除、参数是否非负, 再代入**全部**坐标。全通过则交集是唯一一个点, 否则为空。
3. **方向成比例且均非零.** 选一个 $B_j, D_j > 0$ 的坐标, 解 $B_j k - D_j \ell = C_j - A_j$ 。令 $d = \gcd(B_j, D_j)$ 。若无整数解则为空; 否则所有解形如

$$k = k_0 + (D_j/d)t, \quad \ell = \ell_0 + (B_j/d)t.$$

先检查特解是否满足其余坐标, 再取

$$t_{\min} = \max(\lceil -k_0/(D_j/d) \rceil, \lceil -\ell_0/(B_j/d) \rceil).$$

最小可行参数给出新起点 $E = A + k_{\min}B$, 新方向为 $F = (D_j/d)B$, 交集仍是 $E + tF$ ($t \geq 0$)。成比例情形中, 改变 t 不会破坏其余坐标等式, 因为方向差为零。新方向各分量非负, 因此新射线起点对应最小公共整数。

例子. $2 \cdot 4^k$ 与 $8 \cdot 8^\ell$ 转成 $1 + 2k = 3 + 3\ell$, 最小 $(k, \ell) = (1, 0)$, 公共数列为 $8, 512, 32768, \dots$, 公比 64。 $2 \cdot 6^k$ 与 $12 \cdot 9^\ell$ 则在素数 2, 3 上分别给出 $1 + k = 2, k = 1 + 2\ell$, 唯一 $k = 1, \ell = 0$, 公共项只有 12。

复杂度与实现. 反复合并直到处理所有数列。指数与射线步长可能增长很大, 应使用大整数或严格证明范围; 最后对素因子做快速幂相乘。原题各素因子小于 $10^9 + 7$, 指数可模 $10^9 + 6$ 降幂; Bonus 中若素因子等于输出模数, 必须先判断其指数是否为正, 不能直接当成可逆底数。原范围可先筛到 $\sqrt{10^9}$ 试除分解。Bonus 不能简单把原算法换成 Pollard-Rho 就宣称通过: 稀疏指数向量的合并次数、总维数和高精度运算都需要进一步优化。

题目 / 延伸资料: <https://codeforces.com/problemset/problem/571/E>

12.3 AGC031F：把巨大的状态图缩成每点六个状态

Walk on Graph

无向连通图有 n 点、 m 边，模数 M 为奇数。路径按经过次序的边权为 $w_0, \dots, w_{k-1}$ ，权值为 $\sum_i 2^i w_i$ 。多次询问是否存在从 s 到 t 、权值模 M 为 r 的游走，允许重复边。 $n, m, Q \leq 5 \times 10^4, M \leq 10^6$ 。

怎么做：压成生成树 + 回边

遇到这类图上求权值模 M 的题，通常先把图压成一棵生成树 + 若干回边。树上两点之间的权值差由树唯一决定，回边给出可自由组合的生成元。本题还要额外处理乘 2 不是可逆映射这件事——这就是六状态与奇偶性的来源。

解题方向：反转路径，将状态转移变成 $(u, x) \rightarrow (v, 2x + w)$ ；证明状态可逆和同余类平移，再将乘二的指数只保留奇偶性，配合并查集。

12.3.1 第一步：反向读路径消去长度变量

从 t 出发，倒序处理原路径的边，每走一条边令 $x \leftarrow 2x + w$ ，从 $x = 0$ 得到的最终值正好是原题权值。因此询问等价于 $(t, 0)$ 能否到达 (s, r) 。沿同一条边连续走 h 次，数值变为

$$2^h x + (2^h - 1)w.$$

为什么要反转路径

正着走：从 s 出发到达 t 的权值难合并。反着走：把每条边的贡献写成乘 2 再加权值的仿射变换， s, t 各自到公共点的变换可以复合、比较，于是可达性等价于两个变换像的交集。这就是把路径问题变成状态问题。

M 为奇数，取偶数 $h = 2\varphi(M)$ ，既回到原顶点又回到原数值。所以一次状态转移可通过继续走同一条边反向撤销，状态可达关系可作为无向连通关系处理。 $M = 1$ 直接回答 YES。

12.3.2 第二步：推出同顶点的平移等价

顶点 u 邻接两条权为 a, b 的边，来回走一次分别产生

$$x \mapsto 4x + 3a, \quad x \mapsto 4x + 3b.$$

先撤销前一个变换再执行后一个，得到平移 $x \mapsto x + 3(b - a)$ 。沿其他路径运输这个平移，只会多乘 2 的可逆幂；生成的加法子群不变。连通图中任意两边的权差可由相邻边权差相加得到，故在所有顶点都可进行由全部 $3(w_i - w_1)$ 生成的平移。令

$$g = \gcd(M, w_2 - w_1, \dots, w_m - w_1), \quad M' = \gcd(M, 3g).$$

由裴蜀定理，同一个顶点上模 M' 相同的状态彼此可达，所以可以无损地把模数缩为 M' 。这不表示同顶点状态可达当且仅当模 M' 相同，之后仍有乘四产生的等价。因为 $g \mid M$ ， M' 只能是 g 或 $3g$ 。

12.3.3 第三步：统一边权余数并只保留六类形式

令 $z = w_1 \bmod g$ ，所有边权写为 $w_i = z + c_i g$ 。用新坐标 $X = x + z$ ，转移变成

$$X \mapsto 2X + c_i g \pmod{M'}.$$

来回走同一边给 $X \mapsto 4X + 3c_i g \equiv 4X$ ，所以同一点的 X 与 $4X$ 等价。从初始值 z 出发，任意形式可写成

$$2^e z + kg \pmod{M'}.$$

k 只需模 3；乘四不改变 kg 的模 M' 值，因此指数只需保留奇偶性。为每个顶点建立状态 (u, ϵ, k) ， $\epsilon \in \{0, 1\}, k \in \{0, 1, 2\}$ ，一共 $6n$ 个并查集点。每条边 $u \leftrightarrow v$ 、系数 c 建立两种方向的合并

$$(u, \epsilon, k) \sim (v, \epsilon \oplus 1, (2k + c) \bmod 3).$$

即使 $M' = g$ 导致某些状态数值重复，保留这六种形式并在询问时枚举也不影响正确性。

12.3.4 第四步：把询问数值匹配到形式

预处理布尔表

$$\text{vis}[\epsilon][a] = [\exists e \geq 0, e \bmod 2 = \epsilon, 2^e z \equiv a \pmod{M'}].$$

迭代状态 $(e \bmod 2, 2^e z \bmod M')$ 直到首次重复即可，至多 $2M'$ 个状态。询问 s, t, r 时，枚举六个 (ϵ, k) 。若

$$\text{find}(t, 0, 0) = \text{find}(s, \epsilon, k)$$

且 $\text{vis}[\epsilon][(r + z - kg) \bmod M']$ 为真，就回答 YES，否则 NO。负余数必须规范化；原模数为 M 的 r 在这里自然按 M' 处理。

最小例子. 只有两个顶点、一条边权为 1， $M = 7$ 。从一侧走到另一侧必须走奇数条边，余数为 $2^h - 1$ (h 奇)。可得到 1, 0, 3，不能得到所有余数。这个例子能检查“只看连通图就全可达”的错误直觉。

复杂度与实现. 预处理 $O(M + (n + m)\alpha(n))$ ，每问常数次并查集查询；空间 $O(M + n + m)$ 。这里 α 是反阿克曼函数，不是素数幂指数。

为何这道题适合作为同余代数的结课题

可逆性用于把有向动态改为无向等价；裴蜀用于生成平移子群；取模用于压缩数值；乘四的周期用于压缩状态。四个证明缺一不可，仅写“开六倍点并查集”无法解释算法。

题目 / 延伸资料：https://atcoder.jp/contests/agc031/tasks/agc031_f

题目 / 延伸资料：<https://www.luogu.com.cn/article/hz11gwzl>

12.4 WC2020 猜数游戏：把期望拆成分量贡献

P6730：猜数游戏

给定互不相同的非零余数 $a_1, \dots, a_n$ ，模数 M 为奇素数幂。随机等概率选择一个非空子集。询问一个被选中的 a_i 时，可得知子集中所有能够写成 $a_i^k \bmod M$ (k 为正整数) 的数。题目对每个固定子集考察其最少所需询问次数，求该最小值的期望乘 $(2^n - 1)$ 后模 998244353 的结果。 $n \leq 5000, M \leq 10^8$ 。

解题方向. 建立“一个数的正整数幂能到另一个数”的传递关系；将强连通分量视为一组，统计它被选中且所有严格祖先都未选中的概率。单位部分用阶，非单位部分直接枚举短幂链。

12.4.1 先分清题目中的最优含义

这里求的是每个子集的**最小覆盖询问数**的平均值，不是在未知答案下设计一个自适应策略、使平均猜测次数最小。原题样例中子集只含 1 时最少次数为一，正体现了这一口径。把它误建成在线决策问题会得到不同答案。

建边 $i \rightarrow j$ 当 $a_i^k \equiv a_j \pmod{M}$ 对某个 $k \geq 1$ 成立。幂再取幂仍为幂，所以该关系传递。缩点后是一个传递 DAG。对一个固定选中子集，只有“被选中且没有任何被选中的严格祖先”的分量必须各询问一次；其他选中分量都由某个上方的选中分量覆盖。同一强连通分量中只需选一个被选中的点询问。

12.4.2 期望线性性给出封闭计数式

对分量 C ，设其大小为 c_C ，能够到达它但不属于它的原图点数为 u_C 。它贡献一次询问，当且仅当自己至少选一个点、全部严格祖先都不选，其余任意选。满足该事件的子集数是

$$(2^{c_C} - 1)2^{n-c_C-u_C}.$$

口径：两个平均询问数不是一回事

对所有子集，求最小询问数的平均与设计一个策略，使平均询问数最小是两个不同的量。前者是固定对象的平均，可以直接换序计数；后者要证明策略最优。读题时先把这句话写完，再决定用哪种概率语言。

为什么不需要独立

期望线性性： $\mathbb{E}[\sum_i X_i] = \sum_i \mathbb{E}[X_i]$ ，对任意随机变量都成立，不要求独立。这也正是它能化简互相牵连的覆盖关系的原因：把每个强连通分量当成一个指示变量，只需算它被计人的概率，不必管分量之间是否相关。

这些子集一定非空，因此把所有分量贡献相加就是题目要求的已乘分母结果：

$$\sum_C (2^{c_C} - 1)2^{n-c_C-u_C} \pmod{998244353}.$$

这里不用再求 $(2^n - 1)^{-1}$ 。

12.4.3 不用真的建出平方规模的图

单位的幂仍是单位；非单位在素数幂模数下的正整数幂始终为非单位，故两部分之间没有边。

- 单位部分： M 有原根，所以 $i \rightarrow j$ 等价于 $\text{ord}_M(a_j) \mid \text{ord}_M(a_i)$ 。阶相同的元素构成一个分量，按阶计数后， u_C 等于所有出现的**严格倍数阶**的元素总数。直接两两检查阶的整除关系也只有 $O(n^2)$ 。
- 非单位部分：写 $M = p^\alpha$ ， $\nu_p(a_i) = v > 0$ 。 a_i^k 的赋值在变为零之前为 kv ，严格递增，因此不同非单位不可能相互可达，每个点自成分量。从每个 a_i 枚举 $k = 1, \dots, \lceil \alpha/v \rceil - 1$ 的非零幂，用值到下标的哈希表查找；命中其他点就给它祖先数加一。 $k = 1$ 的自己不计入严格祖先。

样例一完整核对。 $M = 7$ ，数为 1, 3, 4, 6，阶分别为 1, 6, 3, 2。对应祖先数为 3, 0, 1, 1，分量都为单点，贡献为 1, 8, 4, 4，总计 17。

样例二核对. $M = 9$, 数为 $1, \dots, 8$ 。单位的阶 $6, 3, 2, 1$ 的组大小为 $2, 2, 1, 1$, 贡献分别为 $192, 48, 32, 4$; 非单位 $3, 6$ 各贡献 128 , 总计 532 。

复杂度与实现. 分解并求阶后, 关系计数 $O(n^2 + n\alpha)$, 额外空间可做到 $O(n + \alpha)$, 无需存全部边。若显式建图再缩点, 虽然概念清楚, 也应把 $O(n^2)$ 存储写出来。

题目 / 延伸资料: <https://www.luogu.com.cn/problem/P6730>

第 13 章

素性判定与因数分解

章节导读

上一章的综合题告诉我们，很多数论问题最后都绕不开“这个数的结构到底是什么”。这一章把问题直接拉回素数与因数分解：先从试除和费马测试开始，再进入 Miller-Rabin 与 Pollard-Rho，建立从“判断是不是素数”到“真正拆出因子”的完整工具链。

13.1 试除、费马测试与 Carmichael 数

试除只需检查到 $\sqrt{n}$ ：若 n 合数，至少有一个非平凡因子不超过 $\sqrt{n}$ 。对单个 64 位整数，试除通常太慢；对一批不太大的数，预先筛素数仍然有效。费马小定理给出必要条件：素数 n 、 $\gcd(a, n) = 1$ 时 $a^{n-1} \equiv 1 \pmod{n}$ 。但其逆命题不成立。Carmichael 数是对所有与 n 互质的 a 都通过费马测试的合数。这里不能把条件改成 $n \nmid a$ 。

例：561. $561 = 3 \cdot 11 \cdot 17$ ，2, 10, 16 都整除 560。对所有与 561 互质的 a ，费马小定理分别给出 $a^{560} \equiv 1$ 模 3, 11, 17，CRT 得到模 561 也为 1。所以随机重复费马测试无法可靠识别这一类合数。

从必要条件寻找更强必要条件

对奇素数，平方等于一的元素只有 ± 1 。不仅要检查最终 a^{n-1} ，还要检查通过连续平方到达一的过程里是否出现了非法的“非平凡平方根”。这正是 Miller-Rabin 的核心。

词义：非平凡平方根

$x^2 \equiv 1 \pmod{n}$ 的解里， $x \not\equiv \pm 1$ 的那个叫非平凡平方根。它的价值在于： $n \mid (x-1)(x+1)$ 但 n 不整除任何一个因子，于是 $\gcd(x-1, n)$ 必是真因子——一句话把求平方根变成求 gcd。

13.2 Miller-Rabin 的强伪素数测试

对奇数 $n > 2$ ，写

$$n-1 = 2^s d, \quad d \text{ 为奇数.}$$

选取底数 a ，先算 $x = a^d \bmod n$ 。本轮通过，当且仅当

$$a^d \equiv 1 \pmod{n} \quad \text{或} \quad \exists j \in \{0, \dots, s-1\}, a^{d2^j} \equiv -1 \pmod{n}.$$

不通过就能确定 n 为合数；通过只说明该底数未发现合数证据。

为何素数一定通过？素数情形最终 $a^{d2^s} = a^{n-1} = 1$ 。若起点已为一，第一种情况通过；否则看首次由非一平方得到一的地方，前项是 $X^2 = 1$ 的根，只能为 -1 。因此上述两种情况必有一个成

立。

□

完整示例：底数二检测 561. $560 = 2^4 \cdot 35$, 连续平方链为

$$2^{35} \equiv 263, \quad 263^2 \equiv 166, \quad 166^2 \equiv 67, \quad 67^2 \equiv 1 \pmod{561}.$$

没有出现 $-1 \equiv 560$, 起点也不是一, 所以发现合数。67 是非平凡平方根, $\gcd(67 - 1, 561) = 33$, 还能顺便得到因子。

复杂度与实现. 一轮只做一次快速幂和至多 $s - 1$ 次平方, 为 $O(\log n)$ 次模乘, 不要为每个指数重新快速幂而变成 $O(\log^2 n)$ 。

13.3 随机误差与确定性底数

经典强伪素数计数定理给出: 对固定奇合数 n , 均匀随机底数的一轮误判概率不超过 $1/4$ 。独立测试 k 轮, 误判概率不超过 4^{-k} 。这是对每个固定输入、每轮重新独立抽样的概率结论。在 $0 \leq n < 2^{64}$ 范围内, 可用固定底数

2, 325, 9375, 28178, 450775, 9780504, 1795265022

口径：随机的是底数还是模数

对固定 n 、随机底数和对固定底数、随机 n 是两个不同的陈述, 不能混用。固定底数 (如 2) 时存在反例, 所以本书模板用一组固定的小素数当底数, 在 2^{64} 范围内是**确定**正确的判定, 而不是概率判定。

读一条测试链的方法

先算 a^d , 再一路平方到 $a^{d2^s} = a^{n-1}$ 。判定规则是: 起点是 1 (通过), 或首次到达 1 的前一项是 -1 (通过); 否则 n 是合数, 而且那个前一项就是非平凡平方根, 顺手 $\gcd$ 出因子。561 是最小的反例 (Carmichael 数), 所以费马测试单独用不安全。

得到确定性判定。每个底数先对 n 取模, 余数为零时跳过该轮; 该有限范围保证不能外推到任意大整数。

13.3.1 课堂可证明的较弱上界: $1/2$

为了理解误差来源, 下面证明一个后文也会用到的引理; $1/4$ 的更紧界作为经典结论使用。

引理 13.1 (已知群指数倍数时的平方根分裂). 设奇数 n 至少含两个不同素因子, $T = 2^s d$ 为 $\varphi(n)$ 的倍数, d 为奇数。在模 n 单位中均匀选择 a , 序列 $a^d, a^{2d}, \dots, a^{2^s d} = 1$ 中, 最后一个非一元素存在且不为 -1 的概率至少为 $1/2$ 。

证明. 先把这一段用到的每个词落到具体计算上, 读者可按行核对。

设定。写 $n = \prod_i q_i^{e_i}$ (q_i 为奇素数, 因为固定底数情形只处理奇合数)。模 $q_i^{e_i}$ 的单位群是循环群, 阶为

$$\varphi(q_i^{e_i}) = q_i^{e_i-1}(q_i - 1) =: 2^{s_i} t_i, \quad t_i \text{ 奇}, s_i \geq 1.$$

d 由 $n - 1 = 2^s d$ (d 奇) 给出, T 是 $\varphi(n)$ 的倍数, 而 $\varphi(n) = \prod_i 2^{s_i} t_i$, 故每个奇部分 t_i 都整除 d 。

单个素数幂上的失败概率。设 a_i 在单位群中均匀分布, 群为循环群 $C_{2^{s_i} t_i} \cong C_{2^{s_i}} \times C_{t_i}$ 。 a_i^d 的奇部分被 d (它是 t_i 的倍数) 消掉, 所以 a_i^d 落在 2-部分 $C_{2^{s_i}}$ 上且均匀。定义 J_i 为从 a_i^d 开始连续平

方, 第一次得到 1 所需的次数 (0 表示它已经是 1)。在 $C_{2^{s_i}}$ 中, 2^{s_i-j} 阶元素恰有 $\varphi(2^{s_i-j}) = 2^{s_i-j-1}$ 个 ($j \geq 1$), 故

$$\Pr(J_i = 0) = 2^{-s_i}, \quad \Pr(J_i = j) = 2^{-1} \cdot 2^{-s_i+j-1} \cdot 2 = 2^{j-1-s_i} \quad (1 \leq j \leq s_i).$$

两式都 $\leq 1/2$, 因为 $s_i \geq 1$ 。这正是一个局部测不出问题的概率至多一半的来源。

多个素数幂同时失败。中国剩余定理说各素数幂上的余数可以独立选取, 故 $J_1, \dots, J_r$ 相互独立。测试通过要求: 要么某个 $a_i^d = 1$ (全局 $a^d = 1$), 要么所有局部序列在同一个 j 处首次到达 -1 后再平方变 1——即诸 J_i 必须全部相等。固定 $J_1 = j$ 后, 其余 J_i 各自以不超过 $1/2$ 的概率落到这个值, 因此全相等的概率至多 $(1/2)^{r-1}$ 。当 n 是素数幂 ($r = 1$) 时, 用 $X^2 \equiv 1$ 在奇素数幂模数下只有 ± 1 两个根直接给出同一个界。

合起来: 一轮通过的概率 $\leq 1/2$, 而经典计数定理给出更强的 $1/4$; 本书按 $1/4$ 叙述, 因为它对任意奇合数成立且证明可查 (下一段的 $1/2$ 是无条件版本, 二者不矛盾, 只是强弱不同)。□

若 $n = q^e$ 为奇素数的真幂, 费马测试通过的单位比例仅 $1/q^{e-1} \leq 1/3$ 。若 n 至少有两个不同素因子, 取 $T = (n-1)\varphi(n)$ 。通过 Miller-Rabin 的底数, 其各局部二幂阶同步; 把奇数部分再乘 $\varphi(n)$ 的奇数部分不会改变这些二幂阶, 所以也属于上面引理中“不分裂”的集合。于是得到不超过 $1/2$ 的较弱误判上界。原稿的长证明中模数 q_i 与 $q_i^{e_i}$ 、二进制指数与奇数部分多次混淆, 这里统一用局部循环群的分布表示。

随机结论与安全工程

随机底数不能固定为随意挑出的几个小素数后仍声称对任意大整数有 4^{-k} 的误差界。64 位的固定底数集是另一种有适用范围的结论。竞赛用随机数发生器也不是密码学随机源; 本讲义模板不直接用作安全协议实现。

题目 / 延伸资料: https://cp-algorithms.com/algebra/primality_tests.html

13.4 Pollard-Rho: 在未知模素因子下找碰撞

若 n 合数, 寻找 $1 < d < n, d | n$, 再递归分解 $d, n/d$ 。寻找非平凡因子前先做素性判定, 并处理 1、偶数与小素因子。设 p 是 n 的最小素因子。若找到 $x \not\equiv y \pmod{n}$ 但 $x \equiv y \pmod{p}$, 则 $\gcd(|x-y|, n)$ 就是非平凡因子。采用伪随机迭代

$$x_{i+1} = x_i^2 + c \pmod{n}.$$

模 p 的投影也遵循同样的迭代。用 Floyd 快慢指针或 Brent 判环, 在不知道 p 的情况下通过 gcd 检查是否发生局部碰撞。

13.4.1 生日悖论如何给出规模估计

若抽取近似均匀的模 p 状态, r 个状态没有碰撞的概率近似

$$\prod_{j=0}^{r-1} (1 - j/p) \approx \exp(-r(r-1)/(2p)).$$

所以 r 达到 $\sqrt{p}$ 量级便有常数碰撞概率。但若把全部数对都检查, 需要 $O(r^2) = O(p)$ 次 gcd; 原稿把这一朴素两两比较的成本写小了。Floyd 每轮更新慢指针一次、快指针两次, 检查两者差的 gcd。只要投影迭代进入循环, 两指针在一个环长内相遇, 因此用线性于序列长度的检查数代替了全部数对。

为什么有时会得到 n ? 如果两个状态模 n 也相同, gcd 就是 n , 并没有得到真因子。应换常数 c 、初始值后重试, 不能把 n 原样递归下去, 否则无限递归。

13.4.2 批量 gcd 与故障回退

累积一批差值的乘积

$$P = \prod_i |x_i - y_i| \pmod{n},$$

再求一次 $\gcd(P, n)$ 。结果为 1 说明整批没有发现因子; 在 $(1, n)$ 之间就成功; 等于 n 则回退到该批起点逐项求 gcd, 或重新启动。如果两项分别含不同素因子, 乘起来也可能让 gcd 变成 n , 所以“乘积 gcd 为 n ”不等于这一批没有可用因子。设批长为 B , 常见估计为

$$O\left(\sqrt{p} + \frac{\sqrt{p}}{B} \log n + B \log n\right).$$

取 $B \asymp \log n$ 后应先写成 $O(\sqrt{p} + \log^2 n)$, 恢复成本不能不加说明就消失。实际实现常用固定批长减少常数。

13.4.3 应怎样表述复杂度

在把多项式迭代视作足够随机的启发式假设下, 常见的寻找因子规模为 $\tilde{O}(\sqrt{p})$, 因 $p \leq \sqrt{n}$, 通常用约 $n^{1/4}$ 的经验 / 启发式量级描述。这不是已证明对每个输入和每个随机参数成立的最坏界。每个递归子问题变小也不单独证明总成本由第一轮支配, 需要结合因子的大小与各层成本分析。

给学生的实验任务

比较 n 为小素数乘大素数、两近似相等素数的乘积、素数幂时的运行情况。固定并记录随机种子用于调试; 统计模乘、gcd、重启次数, 而不是仅报告某次运行耗时。判素与因数分解验证应独立: 输出因子必须都通过判素, 乘回必须等于输入。

13.5 位数模型与大整数题的边界

输入一个整数 n 需要 $\Theta(\log n)$ 位。通常意义下的多项式时间是关于位数的多项式, 而不是关于数值 n 的多项式。对一般整数分解, 目前没有已知的经典多项式时间算法; 量子计算模型下存在 Shor 算法, 这与竞赛环境中的经典算法不同。素性判定则已有确定性多项式算法, 不能把“判素难”和“分解难”混为一谈。

大整数实现提醒。 64 位模乘可用无符号 128 位中间值; 150 位输入不能套用该实现。模乘、快速幂、开整数根、gcd 都要换成高精度版本, 并把它们的位复杂度计算进去。下一章有额外信息的分解题可获得随机多项式算法, 关键在于已知一个与真实因子群阶相关的倍数, 而不是 Pollard-Rho 的直接加速。

第 14 章

特殊信息如何把分解变成随机多项式问题

章节导读

上一章我们讨论的是在信息有限时怎样主动寻找因子；如果题目额外告诉我们 (n) 等信息，问题的结构就会发生变化。利用这些额外信息，可以先恢复与因子有关的代数关系，再把分解转化成随机选择参数、寻找非平凡平方根的过程。

14.1 QOJ 4920: 已知 n 与 $\varphi(n)$

给定整数与欧拉函数值，分解质因数

已知 N 和 $\varphi(N)$, $N \leq 2^{150}$ 。不要求使用固定宽度整数实现，设计关于输入位数的随机多项式时间分解算法。

这一节为什么特殊信息是关键

$N \leq 2^{150}$ 不可能用 $O(\sqrt{N})$ 枚举。但已知 $\varphi(N)$ 时, $T = (N-1)\varphi(N)$ 是单位群阶的倍数，于是求平方根再 gcd 的每一步都在 $\text{poly}(\log N)$ 时间内完成。缺了这个倍数，就退回 Pollard-Rho 的启发式，不能宣称多项式时间。

解题方向. 已知 $T = \varphi(N)$ ，因此对每个递归因子 $n \mid N$ 的单位 a 都有 $a^T \equiv 1 \pmod{n}$ 。通过连续平方找非平凡平方根，再用 gcd 分裂。

14.1.1 先理解一个二素数例子

若已知 $N = pq$, p, q 为不同素数，则 $\varphi(N) = N - p - q + 1$ ，所以

$$p + q = N - \varphi(N) + 1, \quad pq = N.$$

直接解一元二次方程即可。例如 $N = 77, \varphi(N) = 60$ ，和为 18、积为 77，两根为 7, 11。一般 N 可能含很多素因子，不能再把它当成二次方程，但“已知群阶的倍数”依然可以使用。

14.1.2 保持原始的群阶倍数

在所有递归中固定 $T = \varphi(N)$ 。若 $n \mid N$ ，则 $\varphi(n) \mid \varphi(N)$ ：按素数幂公式比较，删除素因子或降低指数后，欧拉函数都是原值的约数。因此 T 仍适用于每个子问题。先处理 $n = 1$ 、素数、二的幂因子；若 $n = b^e$ 是完全幂，则递归分解 b 并将结果指数乘 e 。整数开根可枚举 $e \leq \log_2 n$ ，用二分或整数根算法验证，不使用浮点近似作最终判定。剩余 n 为奇合数且至少有两个不同素因子。

14.1.3 一次随机尝试

1. 随机 $1 \leq a < n$, 若 $1 < \gcd(a, n) < n$, 已经得到因子。
2. 否则 a 为单位, 写 $T = 2^s d$, d 奇, 计算 $a^d, a^{2d}, \dots, a^{2^s d} = 1$ 。
3. 若在链中发现 $V \not\equiv \pm 1$ 且 $V^2 \equiv 1 \pmod{n}$, 取 $g = \gcd(V - 1, n)$ 。

由于 $n \mid (V - 1)(V + 1)$ 而 n 不整除两因子中的任何一个, g 是非平凡因子。上一章的平方根分裂引理保证, 在单位上一次尝试成功概率至少为 $1/2$ 。

数值示范. $n = 77, T = 60 = 4 \cdot 15$, 取 $a = 2$, $2^{15} \equiv 43 \pmod{77}$, $43^2 \equiv 1$, 且 $43 \not\equiv \pm 1$, 于是 $\gcd(42, 77) = 7$, 得到另一个因子 11。

复杂度与实现. 每次尝试的模幂、平方、gcd 都是关于 $\log N$ 的多项式操作, 每个有效分裂严格减小子问题, 递归树规模为 $O(\log N)$ 。可使用确定性多项式判素保证理论正确性; 实践中常使用高精度 Miller–Rabin, 并对整体误差预算单独分配。截断重试时, 用联合界控制全树失败概率。

题目 / 延伸资料: <https://qoj.ac/problem/4920>

14.2 QOJ 1860: 由 $M = N\varphi(N)$ 还原 N

构造满足 $M = N\varphi(N)$ 的整数

给定正整数 $M \leq 10^{36}$, 保证存在正整数 N 使 $M = N\varphi(N)$, 求这个 N 。有多组数据。

解题方向. 先证明 N 唯一, 再选择性地寻找 N 所含素因子。未知 N 时仍知道 M 是 $\varphi(N)$ 的倍数, 但它不一定是递归整数 m 的欧拉函数的倍数。

14.2.1 第一步: 解确实唯一

若 $N = \prod q_i^{e_i}$, 则

$$M = \prod_i (q_i - 1) q_i^{2e_i - 1}.$$

为什么要先证唯一性

构造类题目 (求满足条件的 N) 如果解不唯一, 输出格式就必须规定取哪一个; 题面只让输出一个数, 等于在提示你解唯一。先证唯一 (这里靠最大素因子的指数 $2e - 1$ 被唯一确定), 算法就能按从大到小逐个恢复写, 不必回溯搜索。

设 q 为 N 的最大素因子。因为 $q_i - 1 < q_i \leq q$, 它们不会引入比 q 更大的素因子, 也不会额外贡献 q ; 所以 q 同时是 M 的最大素因子, 并且

$$\nu_q(M) = 2e_q - 1.$$

q 的指数唯一确定。除去 $(q - 1)q^{2e_q - 1}$ 后, 又得到剩余部分的同型问题, 依次从大到小恢复即可。 $M = 1$ 时 $N = 1$ 。

示例. $M = 48$, 最大素因子 3 的指数为一, 故 N 中有一个 3; 除去 $3(3 - 1) = 6$ 后剩 8。2 的指数为三, 所以 N 中有两个 2; 最终 $N = 3 \cdot 4 = 12$ 。

14.2.2 第二步：不必完全分解原数

M 中可能含仅来自 $q_i - 1$ 、却不整除 N 的素因子。例如 $N = 15$ 时 $M = 120$ ，其中 $2 \nmid 15$ 。算法只需要收集到 N 的所有不同素因子，允许额外收集少量不需要的素数。递归处理当前整数 $m \mid M$ ，始终固定指数 $T = M$ 。

用于证明的量 $d_0 = \gcd(m, N)$ 是未知的，**算法不能计算它**。不过 $\varphi(d_0) \mid \varphi(N) \mid T$ ，因此对所有与 m 互质的 a ，

$$a^T \equiv 1 \pmod{d_0}.$$

这就是能够安全剪枝的原因。

14.2.3 一次递归怎么处理？

1. $m = 1$ 返回； m 为素数就记录；提出因子 2 并记录它。若 $m = b^e$ 为完全幂，只需递归 b 以收集不同素因子。
2. 对剩余奇合数随机 $a \in [1, m)$ 。若 $\gcd$ 已为非平凡因子，直接分裂并递归。
3. 否则计算 $D = \gcd(a^T - 1, m)$ 。因为 $d_0 \mid D$ ：
 - $D = 1$ ：说明 $\gcd(m, N) = 1$ ，整个 m 中没有需要的素因子，可以安全丢弃；
 - $1 < D < m$ ：得到非平凡分裂，递归 $D, m/D$ ；
 - $D = m$ ：说明本次 $a^T \equiv 1 \pmod{m}$ ，进入连续平方链寻找非平凡平方根。
4. 在第三种情况中，写 $T = 2^s d$ 、 d 奇，从 a^d 不断平方。若得到 $V^2 \equiv 1$ 、 $V \not\equiv \pm 1$ ，使用 $\gcd(V - 1, m)$ 分裂；若未得到，重新随机。

原稿中一个使算法直接失效的符号错误

当 $V \equiv -1 \pmod{m}$ 时， $\gcd(V + 1, m) = m$ ，并非非平凡因子。必须要求 V 是既非 1 也非 -1 的平方根，才用 $V \pm 1$ 分裂。

14.2.4 第三步：为什么仍有常数成功概率？

这里不能直接说 $\varphi(m) \mid T$ ，因为它未必成立。正确做法是对通过 $a^T = 1$ 的底数集合进行条件分析。模 m 的单位群中，满足 $a^T = 1$ 的元素构成子群。按每个奇素数幂分量来看，这个子群的大小为 $\gcd(T, \varphi(q^e))$ 。当 $M > 1$ 时 $T = M$ 为偶数，因此每个局部子群的二幂部分至少有两个元素。条件于 $a^T = 1$ 后，CRT 仍使不同局部分量独立均匀；平方链首次变为一的层数分布与上一章引理相同，每项概率至多 $1/2$ 。至少有两个不同素因子时，找到非平凡平方根的条件成功率至少 $1/2$ 。若不满足 $a^T = 1$ ，前面的 $D < m$ 已经使算法安全剪枝或分裂。因此每次尝试无法推进的概率至多为 $1/2$ ，不需要错误地假设 T 是整个 m 的群阶倍数。

14.2.5 最后按降序恢复答案

将收集到的不同素数从大到小排序，令剩余值 $R = M$ 、答案 $A = 1$ 。遇到 $q \nmid R$ 就跳过；若 $q \mid R$ ，则当前最大的尚未处理相关素数一定是剩余 N 的最大素因子。计算 $e = \nu_q(R)$ ，应为奇数，令 $k = (e + 1)/2$ ，更新

$$A \leftarrow Aq^k, \quad R \leftarrow \frac{R}{(q-1)q^e}.$$

直到 $R = 1$ 。最终用收集到的质因数分解计算 $\varphi(A)$ ，验证 $A\varphi(A) = M$ 。剪枝不会遗漏 N 的素因子；额外收集的素数要么已随更大的 $q - 1$ 被消去，要么最终不整除 R ，因此不会造成错误恢复。

复杂度与实现. 这是关于 $\log M$ 的随机多项式路线，依赖高精度运算、整数完全幂检测与可靠判素。理论上可以采用确定性多项式判素并无限重试形成期望多项式算法；实践中若用概率判素和有限重试，应声明错误预算并验证输出。它不是直接对 10^{36} 运行 64 位 Pollard-Rho。

题目 / 延伸资料: <https://qoj.ac/problem/1860>

第 15 章

韦达跳跃与构造不定方程的解

章节导读

前面的分解算法依赖“把一个整体拆成因子”；这一章换成另一类离散结构：从一个已经存在的整数解出发，如何构造出另一个更小的解。韦达跳跃把二次方程的两个根联系起来，再结合正性与最小性建立递降，从而把许多看似只能试答案的构造题变成系统的证明。

15.1 把一个变量看成二次方程的根

若 $at^2 + bt + c = 0$ 的两个根为 t_1, t_2 ，则

$$t_1 + t_2 = -b/a, \quad t_1 t_2 = c/a.$$

韦达跳跃到底跳什么

把方程看成关于某个变量（比如 z ）的一元二次方程： $z^2 - k(x+y)z + (x^2 + y^2 - 1) = 0$ 。已知一个根 z ，用韦达定理 $z + z' = k(x+y)$ 直接写出另一个根 $z' = k(x+y) - z$ ——它就是解，且通常更小（降）或更大（升）。构造题里我们反向用它从小解生成无穷多个不同解。

当方程各系数为整数、首项系数为一且已知一根为整数时，另一根也为整数。这种“固定其余变量、替换一根”的操作称为韦达跳跃。若新根比旧根小且仍满足约束，可用于无穷递降；反过来可从基本解生成更多解。

P1400: Easy Equation

给定 $2 \leq k \leq 1000$ 、 $1 \leq n \leq 1000$ ，输出 n 组正整数解

$$x^2 + y^2 + z^2 = k(xy + yz + zx) + 1.$$

全部 $3n$ 个输出整数必须互不相同，每个数最多 100 位，即小于 10^{100} 。

解题方向. 固定两变量，第三变量可替换为 k 倍的另外两数之和减去自身；从基本解建立递降树，反向广度优先生成并检查全局重复值。

15.1.1 推导跳跃公式而不是记变换

固定 x, y ，将方程改写为

$$z^2 - k(x+y)z + x^2 + y^2 - kxy - 1 = 0.$$

如果 z 为一根，另一根为

$$z' = k(x+y) - z.$$

因此 (x, y, z') 仍满足原方程。该操作是对合：再替换一次就回到 z 。但另一根可能为零或负数，不能无条件说它仍是正整数解。

15.1.2 严格递降与停止条件

设 $1 \leq x \leq y \leq z$ ，定义上面的二次多项式 $F(t)$ 。有

$$F(y) = (2 - k)y^2 + x^2 - 2kxy - 1 < 0.$$

首项系数为正，故两根分别在 y 两侧； z 是较大根，另一个根满足 $z' < y < z$ 。若 $z' > 0$ ，则排序后得到和更小的正整数解；反复操作不可能无限下降。若 $z' \leq 0$ ，在正整数解空间中停止。这定义了以边界解为根的递降森林。

易错点

“不停得到更小整数”只有同时保证仍处于研究对象的集合里，才能组成无限递降矛盾。整数性、正性和严格变小是三个独立检查。对本题我们把越过正数边界的点视为基本解，而不是强行继续下降。

15.1.3 从基本解向上生成：两个孩子

从 $1 \leq x < y < z$ 的解出发，替换两个较小的变量，排序后的两个孩子为

$$(y, z, k(y + z) - x), \quad (x, z, k(x + z) - y).$$

由于 $k \geq 2$ ，两个新产生的数都严格大于 z ，所以仍是严格递增的正整数三元组。孩子再替换最大数就回到父亲；父亲由此唯一，所以这棵树不会出现同一三元组从不同父亲重复生成的情况。然而三元组不重复并不意味着其中的整数不重复：父子自然共享两个数，因此输出时还要维护一个全局已使用整数集合。

15.1.4 一个适用于所有合法 k 的种子

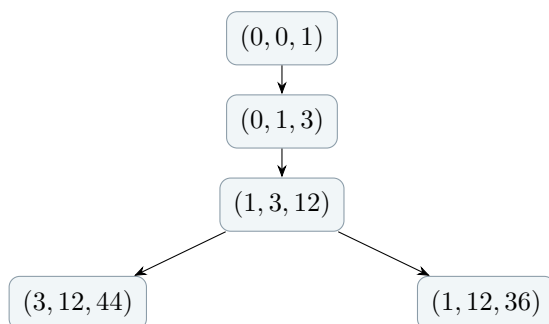
非负整数解 $(0, 0, 1)$ 通过两次跳跃可得到

$$(0, 1, k) \longrightarrow (1, k, k(k + 1)).$$

为什么需要统一种子

BFS 从一个固定的最小三元组出发，能保证不会漏掉某个 k ；同时要用全局去重（同一数不能出现在两组解里）来筛掉重复。少了任一条，程序要么少解要么重解，而对拍很难发现——因为它在小数据上经常是对的。

最后一组是严格递增的正整数解，可作为统一的搜索起点。这样无需先枚举几千万对基本解。



图为 $k = 3$ 的局部生成关系，由 TikZ 内嵌绘制，无需原稿缺失的 `tree_k3.png`。

15.1.5 BFS 与全局去重的可实现策略

1. 队列初始放入 $(1, k, k(k+1))$ ，已使用集合为空。
2. 取出一个三元组。若其三个值均未出现在已使用集合，就输出它并登记三个值。
3. **无论该三元组是否被输出**，都将两个孩子入队；否则会错误截断可用分支。
4. 获得所需组数后停止。

采用大整数，例如 Python 整数或 C++ 的 `boost::multiprecision::cpp_int`。计算判定等式时需要平方，不能因为坐标本身似乎较小就假定中间值也适合 128 位。

15.1.6 正确性证明与规模证据分开陈述

前面的代数推导严格保证：每个生成三元组满足方程、三个数为正且不同、父子关系无重复；全局集合保证输出的 $3n$ 个值两两不同。仍需说明在位数限制内能否找到足够多组，不能用“树是无限的”代替定量保证。

本次整理对本题**全部 999 个合法参数** $k = 2, \dots, 1000$ 运行统一种子的 BFS，均生成了 1000 组，并逐组用大整数验算方程与全局去重条件。有限检查结果为：最多访问 7132 个节点，最深输出层为 12，输出整数最大为 43 位。任意更小的 n 直接取相应前缀，因此覆盖原题全部参数组合。验证脚本及逐 k 记录附于资料包。这是可复现的**有限范围计算验证**，不是关于任意 k, n 的渐近定理；教师应明确区别。可让提高层学生尝试补充适用于无界参数的定量树分析。

一个独立的位数上界核对。 每次孩子的最大数小于原最大数的 $2k$ 倍，因此深度 d 时最大数小于

$$k(k+1)(2k)^d.$$

结合有限验证得到的 $d \leq 12, k \leq 1000$ ，该式也远小于 10^{100} 。实际出现的 43 位数已经可能超过有符号 128 位的范围，因此本方案不能沿用原稿“`__int128` 一定足够”的笼统说法。

题目 / 延伸资料：<https://www.luogu.com.cn/problem/P1400>

附录 A

分层训练与参考解析

章节导读

前十五章建立的是一条从基本数论工具一路走到高级专题的主线；从这里开始，正文不再继续引入新的知识体系，而是转向训练与教学资源。附录 A 把前面学过的方法重新组合进分层练习中，重点从“会不会公式”转向“能不能选对工具、检查条件并写出完整推导”。

本章是新增训练，不替代前文原题。学生版只保留题目；教师版在每题后给出结果、关键步骤与评价要点。建议先独立完成，再检查条件和推导，而不是仅比对最后一个数。

A.1 A 组：整除、通解与基础建模

A1：从 gcd 回代到不定方程

求 $\gcd(414, 662)$ ，给出 $414x + 662y = 6$ 的一组整数解与通解。

讲解：先看这题到底在做什么

先把 gcd 的回代结果写成一个线性组合，再整体乘到右端。这里练的是“exgcd 的输出怎么变成方程特解”，而不是重新证明裴蜀定理。

解析. gcd 为 2。欧几里得回代得 $2 = 8 \cdot 414 - 5 \cdot 662$ ，乘三后得到 $(x_0, y_0) = (24, -15)$ 。通解为

$$x = 24 + 331t, \quad y = -15 - 207t, \quad t \in \mathbb{Z}.$$

三档练习的用法

A 组只用到第 1–3 章，做完应该能独立写出通解区间；B 组对应第 4–6 章，重点是**先列周期再套公式**；C、D 组要求你识别工具而不是回忆公式。建议每题限时 15 分钟，写不出来就读解析的第一句，仍然卡住就先跳到下一组，回头再战。

评价时应分别检查：整除条件、缩放方向、通解步长；只写一个特解不能算完成。

A2：正整数解的计数

求 $14x + 21y = 147$ 的全部正整数解、解数以及两个变量的极值。

讲解：先看这题到底在做什么

先写出全部整数解，再把 $x > 0, y > 0$ 各自产生的条件变成同一个参数 t 的上下界。这样“数多少组正整数解”就完全变成了整数区间计数。

解析. 化为 $2x + 3y = 21$ 。取 $x = 3t, y = 7 - 2t$ ，约束给 $1 \leq t \leq 3$ ，所以解为 $(3, 5), (6, 3), (9, 1)$ 。解数 3， x 最小 3、最大 9， y 最小 1、最大 5。强调两个最小值不来自同一组解。

A3: 约分会改变模数

解 $18x \equiv 30 \pmod{42}$ ，并在区间 $[0, 42)$ 中列出全部解。

讲解：先看这题到底在做什么

同余式两边约掉公共因子以后，模数也必须同步除掉这个因子。先得到约分后的最小模数，再列出全部同余解；只写一个代表元会漏掉区间里的其他解。

解析. 除 $\gcd 6$ 得 $3x \equiv 5 \pmod{7}$ ，故 $x \equiv 4 \pmod{7}$ 。区间内解为 4, 11, 18, 25, 32, 39，共有六个。只给 4 是漏解；把约分后模数仍写成 42 是错误。

A4: CRT 与下界

求满足 $x \equiv 1 \pmod{4}$ 、 $x \equiv 3 \pmod{6}$ 且 $x \geq 40$ 的最小整数。

讲解：先看这题到底在做什么

CRT 负责找到所有满足同余条件的数，即一个同余类；题目还有“至少多大”的额外限制，所以最后还要从这个同余类中找第一个不小于下界的代表元。

解析. 令 $x = 1 + 4t$ ， $4t \equiv 2 \pmod{6}$ ，故 $t \equiv 2 \pmod{3}$ ，合并为 $x \equiv 9 \pmod{12}$ 。不小于 40 的首项为 45。直接输出最小非负代表元 9 忽略了大小约束。

A5: 一个质因数分解对应多种函数

计算 $\varphi(72), \mu(72), \tau(72)$ 与 $\sum_{d|72} \varphi(d)$ 。

讲解：先看这题到底在做什么

把同一个质因数分解同时喂给 φ, μ, au ，可以很好地训练“各函数到底看分解的什么信息”： φ 看质因子及其幂次， μ 只关心是否出现平方因子， au 直接把各指数加一后相乘。

解析. $72 = 2^3 \cdot 3^2$ ，所以 $\varphi(72) = 72(1 - 1/2)(1 - 1/3) = 24$ ， $\mu(72) = 0$ ， $\tau(72) = (3+1)(2+1) = 12$ 。最后一项直接由欧拉恒等式等于 72，无需枚举十二个约数。评价重点是区分“有平方因子”与“不同素因子个数”。

A6: 单次逆元与批量逆元

求 $37^{-1} \pmod{101}$ ；再说明模 17 同时求 2, 3, 5 的逆元时怎样只做一次一般求逆。

讲解：先看这题到底在做什么

先用一次 exgcd 得到基准逆元，再观察前缀积之间的乘积逆元关系。批量法的节省之处不在于“ exgcd 更快”，而在于原本很多次求逆被一次逆元和若干乘法替代。

解析. $1 = 11 \cdot 101 - 30 \cdot 37$ ，故逆元为 71。批量问题先算前缀积 1, 2, 6, 13，求 $13^{-1} = 4$ ，再逆推得到 $5^{-1} = 7, 3^{-1} = 6, 2^{-1} = 9$ 。每一步应用的是乘积逆元关系，而不是将三个独立 exgcd 称为“批量”。

A7: 环的连通分量

在 18 个点的环上每次加 12，有几个循环？从点 3 出发能否到点 4？列出点 3 所在循环。

讲解：先看这题到底在做什么

每走一步就是给编号加 k 并模 n 。能走到哪些点只由 $\gcd(n, k)$ 决定；因此先算 $\gcd$ 就能直接知道环被分成多少个分量，而不是逐点模拟。

解析。 $\gcd$ 为 6，所以有六个循环，每个长三。每次保持模 6 余数，3 与 4 不在同一循环。对应循环为 $3 \rightarrow 15 \rightarrow 9 \rightarrow 3$ 。可以进一步追问：到某个点的最少步数与“是否同环”不是同一个问题。

A8: 无整数解与无正整数解

比较 $6x + 10y = 7$ 与 $6x + 10y = 2$ 。第二个方程中，单独可取得的最小正 x 、最小正 y 分别是多少？它们能否同时取得？

讲解：先看这题到底在做什么

先区分两个问题： $d \nmid c$ 时连整数解都没有； $d \mid c$ 时整数解存在，但正整数限制可能让参数区间为空。即使没有同时为正的解，最小正 x 和最小正 y 仍可能分别存在。

解析。前者因为 $2 \nmid 7$ 而无整数解。后者有解 $(-3, 2)$ ，通解 $x = -3 + 5t, y = 2 - 3t$ ，没有同时为正的参数。最小正 $x = 2$ （对应 $y = -1$ ），最小正 $y = 2$ （对应 $x = -3$ ），不能同时取得。这正是 P5656 特殊输出分支的含义。

A.2 B 组：指数、赋值与组合数**B1: 欧拉指数与真实周期**

求 $7^{123456789} \bmod 40$ ，并分别使用欧拉函数与阶解释降幂依据。

讲解：先看这题到底在做什么

欧拉定理给出的 $\varphi(m)$ 只是一个一定有效的指数，不一定是最小周期。先通过小次幂找到真实的阶，再决定指数应该对什么数取模，能避免把“周期”和“可用上界”混为一谈。

解析。 $\gcd(7, 40) = 1$ ， $\varphi(40) = 16$ ，指数模 16 为 5；而 $7^2 \equiv 9, 7^4 \equiv 1$ ，真实阶为 4。指数模 4 为一，答案 7。欧拉定理给出一个周期，未必给最小周期。

B2: 非互质大指数

求 $6^{10^{100}} \bmod 72$ ，并说明不计算任何大指数即可得到答案的原因。

讲解：先看这题到底在做什么

这里不要机械套欧拉降幂。直接看各素因子的赋值什么时候达到模数所需的指数，一旦整个数已经被模数整除，答案就立即是 0。这是非互质底数时最直接的判定方式。

解析。 $72 = 2^3 \cdot 3^2$ ， 6^e 中两个素数的赋值均为 e 。当 $e \geq 3$ 即同时被两个素数幂整除，故答案为零。此处直接按赋值判零，比构造欧拉链更简单。不要把所有巨大指数问题都机械套用递归降幂。

B3: 先列周期，再写离散对数通解

求 $\text{ord}_{17}(4)$ ，并解 $4^x \equiv 13 \pmod{17}$ 。

讲解：先看这题到底在做什么

先把 4 的幂循环写出来，阶自然就出现了。找到最小解以后，所有解都只是在这个周期上平移，因此通解就是“最小解 + 阶的倍数”。

解析. $4^2 \equiv 16 \equiv -1, 4^4 \equiv 1$ ，所以阶为四。周期为 1, 4, 16, 13，最小非负解为三，全部非负解为 $x = 3 + 4t$ ($t \geq 0$)。这里模 17 的阶不是 $\varphi(17) = 16$ 。

B4: 离散对数的有限前缀

分别求 $6^x \equiv 12 \pmod{18}$ 与 $6^x \equiv 0 \pmod{18}$ 的最小非负解。

讲解：先看这题到底在做什么

底数与模数不互质时，序列可能先快速坍缩，再进入一个短周期。此时不能定义一个正常的乘法阶，因此要先观察前缀行为，再判断是否存在目标值。

解析. 序列为 1, 6, 0, 0, ...，第一式无解，第二式最小解为二。这一序列不是从指数零开始的纯周期；也不存在 $\text{ord}_{18}(6)$ 。学生若写“指数对阶取模”，应先让其指出阶为何不存在。

B5: 奇素数 LTE

求 $\nu_3(10^{81} - 1)$ ，并列出的全部前提。

讲解：先看这题到底在做什么

LTE 不是只背最后一个公式。讲解时先逐条检查素数、整除关系和指数条件，再把赋值直接代入。学生能明确说出“为什么这条 LTE 能用”，比记住结果更重要。

解析. 素数三为奇数， $3 \mid 10 - 1$ ， $3 \nmid 10$ 、 $3 \nmid 1$ ，指数为正。故结果为 $\nu_3(9) + \nu_3(81) = 2 + 4 = 6$ 。满分答案必须写出这些条件，而不是只写六。

B6: 二的赋值

求 $\nu_2(5^{12} - 1)$ ，用一般偶指数形式与 $4 \mid x - y$ 的简化形式各算一次。

讲解：先看这题到底在做什么

模 2 的 LTE 与奇素数版本不一样，最稳妥的做法是先写通式，再看额外条件能否简化。最后两种形式得到相同结果时，可以把它当作一次条件自检。

解析. 一般式为 $\nu_2(4) + \nu_2(6) + \nu_2(12) - 1 = 2 + 1 + 2 - 1 = 4$ 。由于 $4 \mid 5 - 1$ ，简化式为 $\nu_2(4) + \nu_2(12) = 4$ 。两种形式一致是一次很好的条件检查。

B7: 阶乘赋值与组合数赋值

求 $\nu_3(100!)$ 与 $\nu_2\left(\binom{100}{50}\right)$ 。

讲解：先看这题到底在做什么

先把阶乘中的 p 因子数量分别算出来，再做减法得到组合数的赋值。Legendre 公式和二进制数位和公式是两种不同视角，适合让学生验证同一个结果。

解析. 第一项为 $33 + 11 + 3 + 1 = 48$ 。第二项为 $\nu_2(100!) - 2\nu_2(50!) = 97 - 2 \cdot 47 = 3$ 。也可用二进制数位和：100、50 都有三个一，所以 $3 + 3 - 3 = 3$ 。结果说明该组合数能被八整除，不能被十六整除。

B8：一次 Lucas 计算

求 $\binom{20}{6} \bmod 7$ 。若把下标改成八，结果是否仍非零？

讲解：先看这题到底在做什么

Lucas 定理的关键是把 n, k 写成 p 进制，然后逐位取组合数。只要出现某一位的下标超过上标，对应因子就是 0；没有越界时才继续相乘。

解析. $20 = (26)_7, 6 = (06)_7$ ，余数为 $\binom{2}{0}\binom{6}{6} = 1$ 。 $8 = (11)_7$ ，余数为 $\binom{2}{1}\binom{6}{1} = 12 \equiv 5$ ，仍非零。这两题都没有位上越界；改成 7 进制某位超过上标的数才会自动为零，不能仅凭下标“变大”判断。

B9：合数模数不能直接除阶乘

求 $\binom{12}{4} \bmod 18$ ，要求按模二和模九分别解释。

讲解：先看这题到底在做什么

组合数取模时最危险的地方就是把普通除法机械搬进模环。先分别处理素数幂中的 p 因子和单位部分，再用 CRT 合并；尤其要明确“分母没有逆元”时不能直接除。

解析. 模二： $\nu_2 = 10 - 3 - 7 = 0$ ，组合数为奇数，余数一。模九： $\nu_3 = 5 - 1 - 2 = 2$ ，至少含 3^2 ，余数零。CRT 合并“奇数且为九的倍数”，模十八余九。实际 $\binom{12}{4} = 495$ 。不允许把 $4! \bmod 18 = 6$ 的“逆元”代入，因为它根本不存在。

B10：辨认计数对象

(1) P7488 的三角形模型中 $n = 4, m = 2$ 的组数是多少？(2) 五件不同礼物分别给两位有编号的人两件、一件，有多少方案？

讲解：先看这题到底在做什么

两道题表面都是“选一些对象”，但排列是否算不同、接收者是否有身份，决定了计数公式。先明确对象是什么、哪些选择不能重复，再决定是否需要乘或除阶乘。

解析. (1) $\binom{4}{2}^2 \cdot 2! = 72$ ，也可由 $R(3, 2) + (8 - 2)R(3, 1) = 18 + 6 \cdot 9 = 72$ 。(2) $\binom{5}{2}\binom{3}{1} = 30$ 。第一题三角形组无序，第二题接收者有身份，但两题最终都通过恰好计数避免重复；不能见到“组”就随意除阶乘。

A.3 C 组：反演与亚线性求和

C1: 手算一次杜教筛

计算 $S_\varphi(12)$ 和 $S_\mu(12)$ ，并写出前者按商分块后的递归式。

讲解：先看这题到底在做什么

先把递推式中的每一个块看成一个整体，而不是逐项展开。 $\lfloor n/j \rfloor$ 相同的连续 j 会共享同一个状态，所以真正要算的是块长和对应的递归状态；手算一次后再看代码会清楚很多。

解析.

$$S_\varphi(12) = 78 - S_\varphi(6) - S_\varphi(4) - S_\varphi(3) - 2S_\varphi(2) - 6S_\varphi(1) = 46.$$

$S_\mu(10) = -1, \mu(11) = -1, \mu(12) = 0$ ，所以 $S_\mu(12) = -2$ 。本题检查的是块长与从 $j = 2$ 开始，漏掉这两处任意一处都会出错。

C2: 矩形上的 gcd 与互质计数

求 $\sum_{i \leq 4, j \leq 6} \gcd(i, j)$ ，以及其中互质有序数对的个数。

讲解：先看这题到底在做什么

这两种计数都能拆成“公共因子 d ”，但权重分别来自 φ 和 μ 。先把计数对象说清楚，再写权重，能避免把两个看起来相似的公式混在一起。

解析. gcd 和为 $24 + 6 + 4 + 2 = 36$ ；互质数对为 $24 - 6 - 2 = 16$ 。前者权重是 $\varphi(d)$ ，后者是 $\mu(d)$ ，两式商因子相同，不能把权重混用。也不能在矩形问题中将两个商都写成 $\lfloor 4/d \rfloor$ 。

C3: gcd 的平方应该使用哪个系数？

推导 $\sum_{i, j \leq n} \gcd(i, j)^2$ 的一维求和式，给出系数在素数幂处的值，并计算 $n = 3$ 的答案。

讲解：先看这题到底在做什么

平方 gcd 对应的是 Jordan 型卷积系数，而不是把 φ 直接平方。最稳妥的解释是从 $\text{id}_2 = J_2 * 1$ 出发，再用同样的公共因子计数套路。

解析. 令 $J_2 = \text{id}_2 * \mu$ ，则 $\text{id}_2 = J_2 * 1$ ，所以答案是

$$\sum_{d \leq n} J_2(d) \lfloor n/d \rfloor^2, \quad J_2(p^e) = p^{2e} - p^{2(e-1)}.$$

$n = 3$ 时为 $9 \cdot 1 + 1 \cdot 3 + 1 \cdot 8 = 20$ 。系数不是 $\varphi(d)^2$ ，因为卷积不保持这种逐点平方关系。

C4: PN 思想的最简单计数应用

求不超过三十的平方自由数个数，要求不逐个判定。

讲解：先看这题到底在做什么

这个例子只是在数没有平方因子的整数，但它完整体现了 PN 的套路：先做局部卷积消去主要贡献，再留下平方数支撑，最后只枚举 $d \leq \sqrt{n}$ 。

解析. 使用 $\sum_{d \leq \sqrt{30}} \mu(d) \lfloor 30/d^2 \rfloor$ ，得到 $30 - 7 - 3 + 0 - 1 = 19$ 。讲解时可让学生说明为什么含多个平方因子的数会通过莫比乌斯系数正确容斥。

C5: PN 卷积商的交叉项

设积性函数满足 $f(p^k) = p^{2k} + [k \geq 2]$ 。取 $g(n) = n^2$ 并令 $f = g * h$ ，求 $h(p), h(p^2), h(p^3)$ ，设计 $S_f(n)$ 的稀疏求和式。

讲解：先看这题到底在做什么

卷积商在 p^3 处会混入低次项的贡献，所以不能直接写成“原函数减基准函数”。先按素数幂递推把前面的项扣干净，再看剩余支撑为什么仍然稀疏。

解析. $h(p) = 0, h(p^2) = 1$ ，而

$$h(p^3) = f(p^3) - g(p^3) - g(p)h(p^2) = 1 - p^2.$$

故不能对所有幂次都写成 $f(p^k) - g(p^k)$ 。有

$$S_f(n) = \sum_{d \in \text{PN}, d \leq n} h(d) \frac{q(q+1)(2q+1)}{6}, \quad q = \lfloor n/d \rfloor.$$

在局部系数及枚举准备完成后，只需 $O(\sqrt{n})$ 个支撑项；若取模且分母不可逆，要整数约分后再取模。

C6: 验证 P4240 的简化系数

计算 $f(6), f(12)$ ，其中 $f(k) = \sum_{d|k} d\mu(k/d)/\varphi(d)$ ；再用 G 形式算 $\sum_{i,j \leq 3} \varphi(ij)$ 。

讲解：先看这题到底在做什么

这一题适合专门检查“局部系数到底有没有算对”。先在整数或有理数层面验证几个小 k ，确认系数结构正确，再把它们放回模意义下的实现。

解析. 6 平方自由， $f(6) = 1/\varphi(6) = 1/2$ ；12 含平方因子， $f(12) = 0$ 。后一个求和中， $k = 1, 2, 3$ 的贡献为 16, 1, 2，合计 19。分数可先在有理数中理解，再在题目模数中用逆元表示。

A.4 D 组：证明辨析与综合拓展**D1: 非平凡平方根为何能分解？**

列出 $x^2 \equiv 1 \pmod{15}$ 的所有解。用其中一个非平凡解找到十五的两个素因子。

讲解：先看这题到底在做什么

CRT 告诉我们模素因子分别取 1 或 -1，组合起来可以得到非平凡平方根。只要它既不是 1 也不是 -1，那么与 $V-1, V+1$ 的 gcd 就会分别捞出不同因子。

解析. CRT 分别在模三、模五选择 ± 1 ，得 1, 4, 11, 14。选 $V = 4$ ，则 $\gcd(V-1, 15) = 3$ ， $\gcd(V+1, 15) = 5$ 。若选 $V = 14 = -1$ ，后一 gcd 等于十五，不会分裂；这正是随机分解步骤为何排除 -1。

D2: 相同阶不代表同一个子群

比较模十五的 2, 7 的阶。是否存在 $a \geq 0$ 使 $2^a \equiv 7 \pmod{15}$ ？说明 P5605 中原根条件不可删除。

讲解：先看这题到底在做什么

“阶相同”和“生成同一个子群”是两件事。先列出两个元素的幂集合，再直接比较集合是否相同，就能看到为什么在非循环单位群里不能只靠阶判断子群包含。

解析. 两者阶均为四，但 2 的幂集合为 $\{1, 2, 4, 8\}$ ，7 的幂集合为 $\{1, 7, 4, 13\}$ ，后者并不属于前者生成的子群。模十五的单位群不循环；阶的整除关系不能替代子群包含关系。

D3：一步边也可能产生多个路径余数

两个点仅通过一条边权二的无向边相连，模数为七。允许反复走边，从一端到另一端能得到哪些权值余数？

讲解：先看这题到底在做什么

先把路径权值写成一个几何级数，再研究 2^h 在允许的奇数 h 上能出现哪些余数。这里必须同时关注图上的路径结构和数值状态，不能只看其中一个。

解析. 路径长度 h 必须为奇数，原权值为 $2(1 + 2 + \cdots + 2^{h-1}) = 2(2^h - 1)$ 。 2^h 在奇数 h 上可取 2, 1, 4，所以余数为 2, 0, 6。图的拓扑连通性与数值状态连通性必须同时考虑。

D4：二维卷积的一次完全消去

取 $S(n) = \tau(n)$ 。证明

$$\tau(xy) = \sum_{d|\gcd(x,y)} \mu(d)\tau(x/d)\tau(y/d),$$

并写出 $\sum_{i,j \leq n} \tau(ij)$ 的一维表达式。

讲解：先看这题到底在做什么

这题真正要看的不是最后那一行公式，而是局部生成函数里的 $1 - XY$ 。它意味着所有坐标轴上的主要贡献都被消掉，只剩对角线上的稀疏支撑，于是二维求和又降成一维。

解析. 固定素数， $S(p^k) = k + 1$ 。二元局部级数为

$$\sum_{a,b \geq 0} (a + b + 1) X^a Y^b = \frac{1 - XY}{(1 - X)^2 (1 - Y)^2}.$$

分母对应 $\tau(x)\tau(y)$ ，剩余 $1 - XY$ 表示卷积商只在局部 (p, p) 为 -1 ，因此全局只在 $x = y = d$ 为 $\mu(d)$ 。答案为

$$\sum_{d \leq n} \mu(d) S_\tau(\lfloor n/d \rfloor)^2.$$

$n = 3$ 时为 $25 - 1 - 1 = 23$ 。这展示了“算法二”在某些函数上把所有非对角残差完全消掉，而不是只得到一个较稀疏的支撑。

D5：从最大素因子开始还原

已知 $M = 1080 = N\varphi(N)$ 且解存在，求 N ，并解释为什么不是直接取 $\sqrt{M}$ 。

讲解：先看这题到底在做什么

已知 $N\varphi(N)$ 后，可以从最大的素因子开始拆，因为 N 和 $\varphi(N)$ 中各素因子的指数关系很受限制。每还原出一个素因子，就更新剩余量，再继续处理下一层。

解析. $1080 = 2^3 \cdot 3^3 \cdot 5$, 最大素因子五的指数为一, 对应 N 中一个五; 除去 $5 \cdot 4 = 20$ 后剩五十四。三的指数为三, 对应 N 中两个三; 除去 $3^3 \cdot 2 = 54$ 后结束。所以 $N = 45$, $\varphi(45) = 24$, 乘积为 1080。 $\varphi(N)$ 一般不等于 N , 平方根只能提供量级, 不能给答案。

D6: 构造题还需要全局去重

当 $k = 2$, 从种子 $(1, 2, 6)$ 写出两个孩子。为什么它们都不能作为该种子之后的第二组输出? 沿 BFS 继续找一组可用输出。

讲解: 先看这题到底在做什么

构造出一个满足方程的三元组只是局部正确; 题目通常还要求不同组之间不能重复使用整数。因此搜索状态必须同时记录“这个组是否合法”和“这些数是否已经在全局出现过”。

解析. 两个孩子是 $(2, 6, 15)$ 、 $(1, 6, 12)$, 都与种子共享已使用整数。不能因为“三元组不相同”就输出。按正文孩子顺序 BFS, 继续可找到 $(15, 40, 104)$; 它与 $1, 2, 6$ 无交, 且

$$15^2 + 40^2 + 104^2 = 12641 = 2(15 \cdot 40 + 40 \cdot 104 + 15 \cdot 104) + 1.$$

把“单组合法”和“所有组之间合法”分别验证, 是构造题的通用习惯。

附录 B

核心模板与接口

章节导读

附录 A 训练的是分析与推导，接下来需要把这些结论真正落到程序接口上。本附录集中整理高频数论模板，并把输入前提、溢出边界、返回值以及空间成本写在代码旁边，方便把数学证明转化为可靠实现。

B.1 模板使用说明

附录代码怎么读

先读注释里的 Contract（前提条件：模数范围、是否互质、输入上限），再读函数体。凡是本书在正文里写**必须先检查**的地方，模板里都有对应的 `assert` 或返回值；提交时要按题面删掉 `assert` 以免被卡常数。

完整 C++17 实现在资料包 `code/number_theory.hpp`。本附录仅摘录关键部分，依赖同文件中的类型别名、`exgcd` 与相关函数；摘录不是可单独编译的提交文件。测试程序为 `code/selftest.cpp`。

模板	限制
<code>mul_mod / pow_mod</code>	无符号 64 位整数，模数必须大于零，乘积使用无符号 128 位。
<code>inverse_mod</code>	不可逆时返回空 optional；模数一返回零是程序约定，不用于通常单位群讨论。
<code>exgcd</code> <code>merge</code>	参数非负且不全为零；返回 gcd 及其一组整数线性组合系数。输入及最终模数必须放入正的有符号 64 位。合并最小公倍数超界时抛异常，不静默溢出。
<code>bsgs / exbsgs</code>	返回最小非负解或空值；哈希表空间为平方根级，不能因为数值能放入 64 位就对 10^{18} 直接开表。
<code>PrimePowerBinom</code>	素数参数需预先保证；默认最多建 2×10^6 个量级的表项。支持非负 n, k , $k > n$ 返回零。
<code>is_prime / factor</code>	判素适用于完整无符号 64 位；分解使用可复现的默认随机种子，允许调用方修改，不是密码学实现。
<code>order</code>	需要 a 与 m 互质，并提供正确的 $\varphi(m)$ 及其有序素因子列表。

B.2 安全模乘、快速幂与逆元

```

inline u64 mul_mod(u64 a, u64 b, u64 m) {
    assert(m > 0);
    return (u128)a * b % m;
}

inline u64 pow_mod(u64 a, u64 e, u64 m) {
    assert(m > 0);
    u64 r = 1 % m; a %= m;
    for (; e; e >>= 1, a = mul_mod(a, a, m))
        if (e & 1) r = mul_mod(r, a, m);
    return r;
}

inline std::optional<u64> inverse_mod(u64 a, u64 m) {
    assert(m > 0);
    // The residue for modulus 1 is returned as a programming convention.
    if (m == 1) return 0;
    i128 x, y;
    if (exgcd(a % m, m, x, y) != 1) return std::nullopt;
    return (u64)norm(x, m);
}

```

B.3 最小非负离散对数

```

inline std::optional<u64> bsgs(u64 a, u64 b, u64 m) {
    assert(m > 0);
    if (m == 1) return 0;
    a %= m; b %= m;
    if (std::gcd(a, m) != 1) return std::nullopt;
    if (b == 1) return 0;
    if (std::gcd(b, m) != 1) return std::nullopt;
    u64 B = (u64)std::sqrt((long double)m);
    while ((u128)B*B < m) ++B;
    while (B && (u128)(B-1)*(B-1) >= m) --B;
    // Caller is responsible for the O(sqrt(m)) memory budget.
    std::unordered_map<u64, u64> baby;
    baby.reserve((std::size_t)(B*2+1));
    u64 v = 1;
    for (u64 j=0; j<B; ++j) {
        baby.emplace(v, j); // Keep the earliest/smallest j.
        v = mul_mod(v, a, m);
    }
    u64 step = *inverse_mod(v, m), giant = b;
    for (u64 i=0; i<=B; ++i) {
        auto it = baby.find(giant);
        if (it != baby.end()) {
            u128 x = (u128)i*B+it->second;
            if (x < m) return (u64)x;
        }
        giant = mul_mod(giant, step, m);
    }
    return std::nullopt;
}

```

```

inline std::optional<u64> exbsgs(u64 a, u64 b, u64 m) {
    assert(m > 0);
    if (m == 1) return 0;
    a %= m; b %= m;
    if (b == 1) return 0;
    if (a == 0) return b == 0 ? std::optional<u64>(1) : std::nullopt;
    u64 count = 0, k = 1;
    while (true) {
        u64 d = std::gcd(a,m);
        if (d == 1) break;
        if (b == k) return count;
        if (b % d) return std::nullopt;
        b /= d; m /= d; ++count;
        if (m == 1) return count;
        k = mul_mod(k,a/d,m);
    }
    auto inv = inverse_mod(k,m);
    assert(inv.has_value());
    auto tail = bsgs(a,mul_mod(b,*inv,m),m);
    if (!tail) return std::nullopt;
    return *tail + count;
}

```

B.4 素数幂模数下的组合数

```

struct PrimePowerBinom {
    u64 p, q = 1;
    unsigned alpha;
    std::vector<u64> pref;
    // Contract: p is prime. Table size must be affordable.
    PrimePowerBinom(u64 prime, unsigned exponent,
        u64 table_limit=2000000):p(prime),alpha(exponent) {
        assert(p>=2 && alpha>=1);
        for (unsigned i=0;i<alpha;++i) {
            if (q>table_limit/p) throw std::length_error("prime-power table too large");
            q*=p;
        }
        pref.assign(q+1,1);
        for (u64 i=1;i<=q;++i)
            pref[i]=(i%p) ? mul_mod(pref[i-1],i,q) : pref[i-1];
        assert(pref[q]==1 || pref[q]==q-1);
    }
    u64 valuation(u64 n) const {
        u64 e=0;
        while (n) { n/=p; e+=n; }
        return e;
    }
    u64 unit(u64 n) const {
        u64 r=1;
        while (n) {
            r=mul_mod(r,pref[n%q],q);
            if ((n/q)&1) r=mul_mod(r,pref[q],q);
            n/=p;
        }
    }
}

```

```

    return r;
}
u64 choose(u64 n,u64 k) const {
    if (k>n) return 0;
    u64 e=valuation(n)-valuation(k)-valuation(n-k);
    if (e>=alpha) return 0;
    u64 r=unit(n);
    r=mul_mod(r,*inverse_mod(unit(k),q),q);
    r=mul_mod(r,*inverse_mod(unit(n-k),q),q);
    return mul_mod(r,pow_mod(p,e,q),q);
}
};

```

B.5 完整无符号 64 位的 Miller–Rabin

```

inline bool is_prime(u64 n) {
    if (n<2) return false;
    for (u64 p:{2ULL,3ULL,5ULL,7ULL,11ULL,13ULL,
        17ULL,19ULL,23ULL,29ULL,31ULL,37ULL})
        if (n%p==0) return n==p;
    u64 d=n-1; unsigned s=0;
    while (!(d&1)) { d>>=1; ++s; }
    for (u64 base:{2ULL,325ULL,9375ULL,28178ULL,
        450775ULL,9780504ULL,1795265022ULL}) {
        u64 a=base%n;
        if (!a) continue;
        u64 x=pow_mod(a,d,n);
        if (x==1 || x==n-1) continue;
        bool pass=false;
        for (unsigned j=1;j<s;++j) {
            x=mul_mod(x,x,n);
            if (x==n-1) { pass=true; break; }
            if (x==1) break;
        }
        if (!pass) return false;
    }
    return true;
}

```

B.6 怎样复现验证

怎么做：对拍的最小可行版本

这四条命令是**对拍**的最小可行版本：一份正确但慢的实现 + 一份你写的实现 + 随机数据。竞赛里 90% 的 **WA** 能被它抓出来，包括取模负号、最小解、边界 $n = 1$ 这类。

在资料目录中运行：

```

g++ -std=c++17 -O2 -Wall -Wextra -fsanitize=undefined \
    code/selftest.cpp -o nt_selftest
./nt_selftest
python validation/verify_math.py

```

```
python validation/check_happy.py
python code/vieta_construct.py --verify
```

C++ 测试程序以 Boost 的大整数作为组合数的独立对照；模板本身无需 Boost。Python 的数学验证仅依赖标准库。

提醒：Boost 只出现在自测里

Boost 只在自测里当**独立裁判**（用 `cpp_int` 精确算组合数再取模）。提交的代码只依赖 `number_theory.hpp`。这个区分很重要：对拍用的实现要**笨且显然正确**，被测的实现要**快且接口干净**。

附录 C

教师使用：课堂设计、追问与评量

章节导读

附录 A 解决“学生怎样练”，附录 B 解决“程序怎样写”；最后的附录 C 则把这两部分重新放回课堂。这里不新增一套数学主线，而是给出课程安排、追问、评分与验证框架，用来说明同一份数论材料怎样分别服务于讲授、训练和复习。

C.1 如何安排一轮而不是一节“数论课”

建议把本资料看作课程资源库，而不是一次讲完的幻灯片。每讲至少留下一个可验证的学习产出：一段具有输入输出契约的代码、一份没有跳步的推导、一组能击穿错误解法的反例。下面给出六个 90 分钟课堂包；高级章节可另设专题阅读。

阶段	通常的时间分配	教师的观察对象
诊断与情境	0–10 分钟：前置小测、具体难题	学生卡在数学概念，还是只忘了实现？
提出结构	10–30 分钟：换元、等价条件、不变量	能否说出“为什么这样设”？
证明与手算	30–50 分钟：核心证明、小数值验算	每一步是否依赖互质、素数、正性？
引导练习	50–70 分钟：同伴推导与纠错	先让学生指出错误，再补正确算法。
实现与迁移	70–85 分钟：代码接口、变式	能否从结论转换为边界安全的实现？
离堂检查	85–90 分钟：一道短题、一句解释	不只收答案，应收关键条件与理由。

C.2 课堂包一：exgcd 从“能解”到“求最小解”

学习目标. 学生能区分整数解与正整数解；能从一组特解生成全部解；能把两个正性约束转成同一个整数参数区间。

课前诊断. 让学生回答 $6x + 10y = 7$ 为什么无解、 $6x + 10y = 2$ 为什么可能有整数解却没有正整数解。不必先出现代码。

板书主线.

$$\text{公约数集合不变} \longrightarrow au + bv = d \longrightarrow (x_0, y_0) = \frac{c}{d}(u, v) \longrightarrow t \text{ 的区间.}$$

引导顺序. 先用 252, 198 手算 gcd；把余数倒代为整数线性组合。让学生自己发现乘 c/d ，再用两组解相减证明 Δx 必须是 b/d 的倍数。最后讲 P5656，不要从通解公式直接开始背诵。

预设错误与回应. 学生说“有解当且仅当 $c \mid d$ ”时, 要求把 $a = 6, b = 10, c = 32$ 代入判断, 再找出 $(2, 2)$; 学生说“求到一个负数解说明无解”时, 要求展示参数平移; 学生把 `ceil` 写成 C++ 除法时, 使用 $-5/3$ 进行对照。

同伴活动. 一人负责给出一个特解, 另一人只利用通解在区间内数解, 再交换检查。两人使用不同特解时, 最终正整数解集合应相同; 这可以检验“特解选择不影响答案”的理解。

离堂题与作业. 离堂做 A4; 作业做 A1、A2、A8, 再实现 P5656 的数学核心。评阅时要求附至少一个“有整数解但无正整数解”的测试。

C.3 课堂包二：指数取模的三个不同问题

学习目标. 区分“算 a^E 的余数”“求指数 x ”“求最小周期”三种问题; 知道欧拉函数是可用周期但不一定最小; 知道非互质指数不能只保留余数。

课前诊断. 写出模八的 2^x 前五项, 模七的 2^x 前七项。请学生观察一个有尾巴、一个从开头就循环。

板书分栏.

问题	核心工具	不能丢的信息
计算幂余数	快速幂、扩展欧拉	指数是否达到阈值
求最小指数	BSGS/exBSGS	零指数、最小命中方式
求周期	阶与素因子试除	单位条件、正整数最小值

证明重点. 欧拉定理只讲清一次“乘法置换剩余系”, 不反复背费马与欧拉两套相同证明。扩展欧拉在素数幂上分类, 把“不互质”变成“足够多素因子时归零”。

引导练习. 先计算 $2^4 \bmod 8$, 再对照指数只取模四的错误结果; 接着让学生设计二元组接口 $(E \bmod q, E \geq q)$ 。随后手算 $2^x \equiv 5 \pmod{13}$, 画出 baby/giant 的匹配表。

预设错误与回应. “两个连续塔高余数相同就稳定”需要完整状态论证; “固定模数就能复用 BSGS 表”还要检查底数; “求阶时 BSGS 返回零”说明忘记阶要求最小正指数。

离堂题与作业. 做 B3、B4, 要求先写能否使用阶的判断。提高作业为 P6736: 只实现截断塔和模塔两个函数, 再解释它们为什么返回不同信息, 暂不要求最优时限提交。

C.4 课堂包三：组合数中不能取逆的部分去哪了？

学习目标. 将组合数分为素数指数与单位部分; 能解释 Lucas 只适合素数模数; 能独立完成一个小合数模数的组合数计算。

情境. 给出 $\binom{10}{4} \bmod 12$, 先允许学生尝试阶乘逆元法。等有人发现分母不能求逆时, 再提出“并非不能除, 而是不能在取模后直接除”。

板书主线.

$$n! \longrightarrow p^{\nu_p(n!)} U_p(n) \longrightarrow p^e \cdot \text{单位分式} \longrightarrow \text{各素数幂余数} \longrightarrow \text{CRT}.$$

证明顺序. 先用倍数计数讲 Legendre; 再用数位和讲 Kummer; 最后讲 Lucas 的生成函数证明。扩展 Lucas 不是在原 Lucas 公式中把素数改成合数, 而是更换为素数幂单位阶乘路线。

操作活动. 将 $1, \dots, 10$ 中每个数写成二的幂乘奇数, 排成两行。让学生亲自看见“删掉偶数”与“除去二的因子”得到不同乘积, 然后推 $U_p(n)$ 的递归。

预设错误与回应. $e \geq \alpha$ 可直接判零, 但 $e < \alpha$ 时单位部分才决定剩余答案。单独的 $k! \bmod M = 0$ 并不能说明组合数为零。若模数为一, 整个计算都可提前结束。

离堂题与作业. 离堂做 B9; 基础作业做 B7、B8, 提高作业做 P2183 和 P7488。要求附上一个最终余数为零但计数对象确实存在的例子。

C.5 课堂包四：反演是为了分离变量

学习目标. 学生能将 $C(\gcd(i, j))$ 写成药数贡献和; 能明确交换求和后每个因子的上界; 能从反演过渡到前缀和递归。

情境. 先画 3×3 的 gcd 矩阵并算总和十二。提出: “直接处理每个点, 需要平方次; 能否让每个公约数统一贡献?”

板书主线.

$$\gcd(i, j) = \sum_{d \mid i, d \mid j} \varphi(d) \implies \sum_d \varphi(d) \lfloor n/d \rfloor \lfloor m/d \rfloor.$$

再把 φ 改为 $D = C * \mu$, 得到一般加权公式。最后从 $\varphi * 1 = \text{id}$ 推导杜教筛, 使同一卷积关系既用于拆点值, 也用于拆前缀。

同伴活动. 每组在一张纸上只写求和指标的条件, 不写函数值: $1 \leq i \leq n, 1 \leq j \leq m, d \mid i, d \mid j$ 。另一组把它改成先枚举 d 的形式。这样可以暴露上界与整除条件的丢失, 而不是让错误藏在长公式中。

预设错误与回应. 把 gcd 拆成药数和后不能继续乘原 gcd; 把 $i = da, j = db$ 后要同步修改两边上界; 分块时两个商变化不同, 必须取共同右端点。 $S_\mu(n)$ 为零不代表它未被计算。

离堂题与作业. 做 C2、C3; 上机先写朴素 gcd 双循环和反演公式对拍, 再实现记忆化杜教筛。提高学生可分析 P4240 的变长表项数。

C.6 课堂包五：PN 与二维积性函数的“消主项”

学习目标. 理解为什么素数处匹配会让卷积商的支撑稀疏；理解二维坐标轴与对角线是两个不同层次的主要贡献；能正确叠加预处理和查询成本。

前置要求. 必须已经熟练进行卷积的素数幂递推，知道形式幂级数常数项为一时可逆。若学生尚不能算出 C5 的三个局部系数，不宜直接讲 UOJ 885。

板书主线.

$$f(p) = g(p) \Rightarrow h(p) = 0 \Rightarrow \text{PN 支撑};$$

$$f(p^a, 1) = g(p^a, 1) \Rightarrow \text{去掉坐标轴};$$

$$h = g_3 * h' \Rightarrow h'(p^k, p^k) = 0 \Rightarrow \text{再去掉对角线}.$$

引导练习. 先做平方自由数计数，看到 $1 - z^2$ ；再做 D4，看到 $1 - XY$ 。这两个可完全手算的局部级数，是一般稀疏支撑论证的最好入口。

证明分层. 基础提高班证明 PN 只有 $O(\sqrt{n})$ 个；进一步证明同质因子对数为 $O(n)$ ；专题班再用收敛的局部乘积证明 $O((AB)^{1/3})$ 。不要同时把三层计数压缩成一句“显然很少”。

离堂题与作业. 要求学生写出一个函数匹配素数处但 $h(p^3) \neq f(p^3) - g(p^3)$ 的例子，并指出总复杂度中哪项是前缀表预处理。研究作业为算法三中 $\rho < 2/3$ 的收敛阈值，不直接要求完整最优实现。

C.7 课堂包六：概率算法的答案为什么可信？

学习目标. 区分单边错误与零错误重试；知道概率的随机对象是底数而不是输入；能通过非平凡平方根解释判素和分解的联系。

情境. 先用 561 展示费马测试的失败，再展示 263, 166, 67, 1 的平方链。让学生自己指出 67 为什么在素数模数下不可能出现。

板书对照.

工具	成功得到什么	失败时意味着什么
Miller–Rabin 见证	确定合数	未见证不自动证明为素数
Pollard–Rho 真因子	可直接验算的因子	更换参数重试
已知群阶倍数分裂	非平凡平方根	若全为 ± 1 ，重新随机

课堂实验. 固定一个合数，枚举所有小底数，统计哪些会通过；再换合数，观察比例。必须先固定输入再讨论底数比例。让学生将输出因子乘回原数，理解可验证结果与概率判定结果的不同。

预设错误与回应. “做了十次就绝对正确”需要明确随机误差；“这些七个底数适用任意大整数”要指出 64 位范围；“得到 gcd 等于 n 就递归 n ”会无限循环；“发现 -1 就得到真因子”用 gcd 立即反驳。

离堂题与作业. 做 D1、D5。上机只对受控大小整数实现判素与分解，记录固定种子以便复现。进阶阅读 QOJ 1860 时，要求专门证明条件于 $a^T = 1$ 的子群分布，而不是偷用 $\varphi(m) \mid T$ 。

C.8 一次完整作业的评分框架

项目	建议分值	应检查的内容
建模与条件	20	变量范围、互质 / 素数条件、是否允许零指数、对象是否有序。
正确性推导	30	必要性与充分性、无漏解、无重复计数、不变量、递归终止。
算法与实现	20	数据结构与推导一致，表的合法索引，模数一与负余数。
复杂度	15	分解、预处理、每问及空间分开写；随机与最坏界不混淆。
验证与反例	15	独立朴素程序、小范围穷举、边界、输出回代、失败案例。

建议允许学生用一个准确的反例修复先前不成立的结论，并对此给分。教师版的目标是建立可审查的推理，不是让学生把所有长公式背到最后。

C.9 上机作业的最低测试清单

1. **exgcd/CRT:** 不相容模数、一个模数整除另一个、余数为负、模数一、只有下界不限制余数、最小公倍数超出类型。
2. **离散对数:** 最小解为零、底数零、目标零、gcd 不为一、剥离后模数一、同余类重复导致的非最小命中。
3. **组合数:** $k = 0, n, k > n, n = 0, M = 1$ ，模八的单位块乘积为一，赋值刚好等于模数指数。
4. **函数求和:** $n = 1, S_\mu(n) = 0$ ，两个商区间错位，多询问缓存，矩形两边差距极大。
5. **判素分解:** 0, 1, 2，偶数、素数幂、Carmichael 数、强伪素数、接近 2^{64} 的数、批量 gcd 等于 n 的回退。
6. **构造:** 每组回代、组内大小范围、跨组全局去重、所有允许参数的规模验证。